

Binary Trees

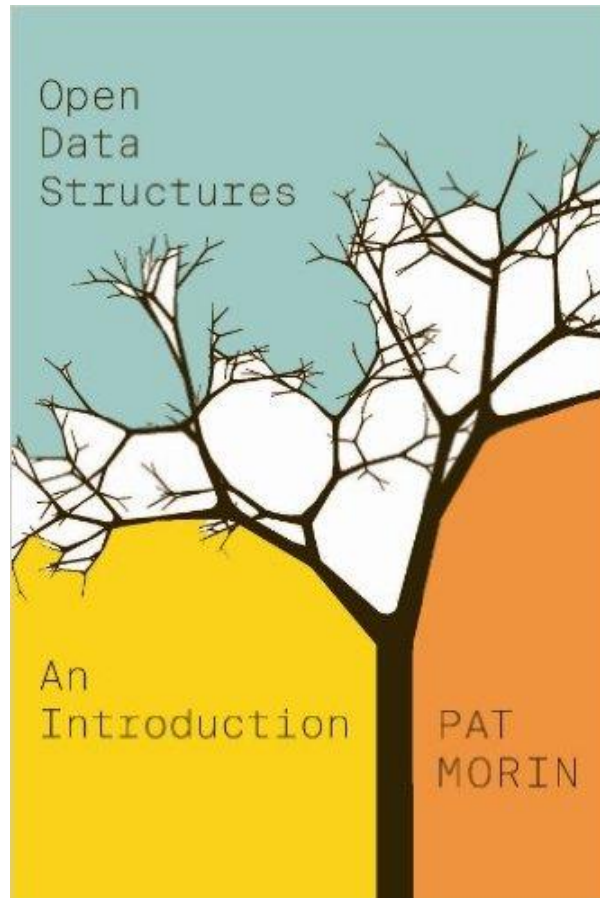
COMP 2402

Abstract Data Types & Algorithms

Readings

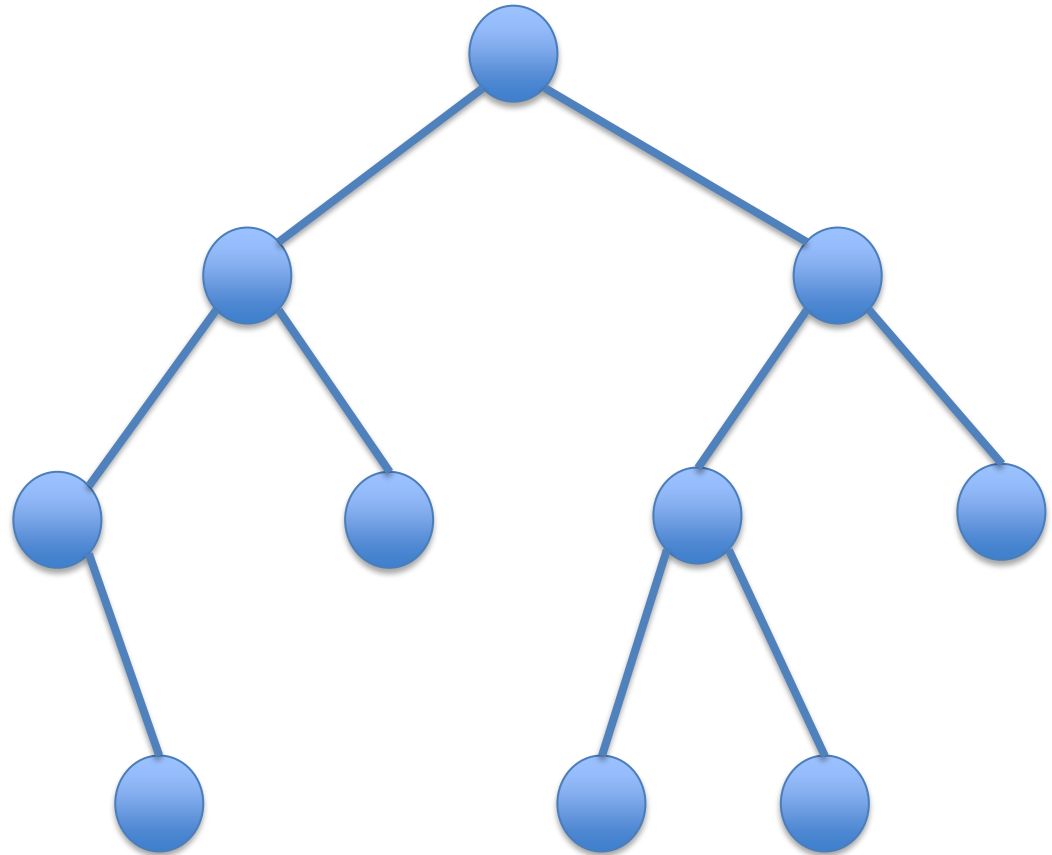
Binary Trees

ODS - Chapter 6



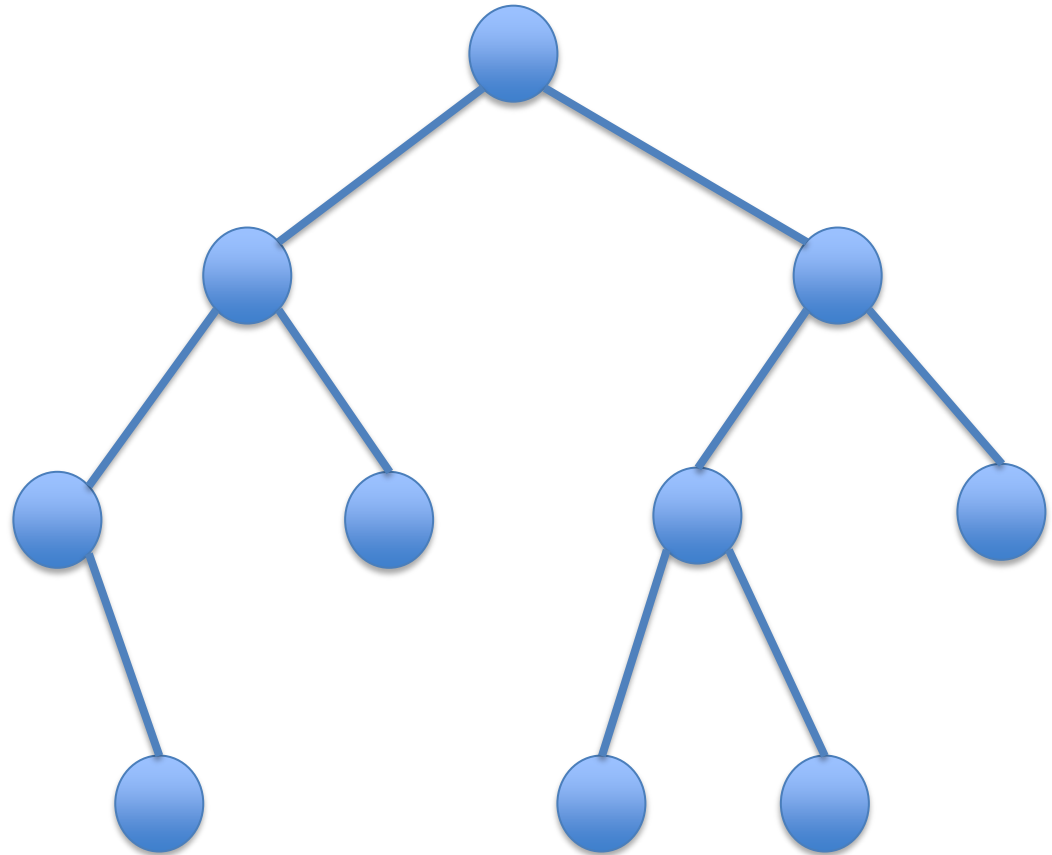
Binary Trees

A **binary tree** is a connected, undirected, finite graph with no cycles and no vertex with degree greater than 3



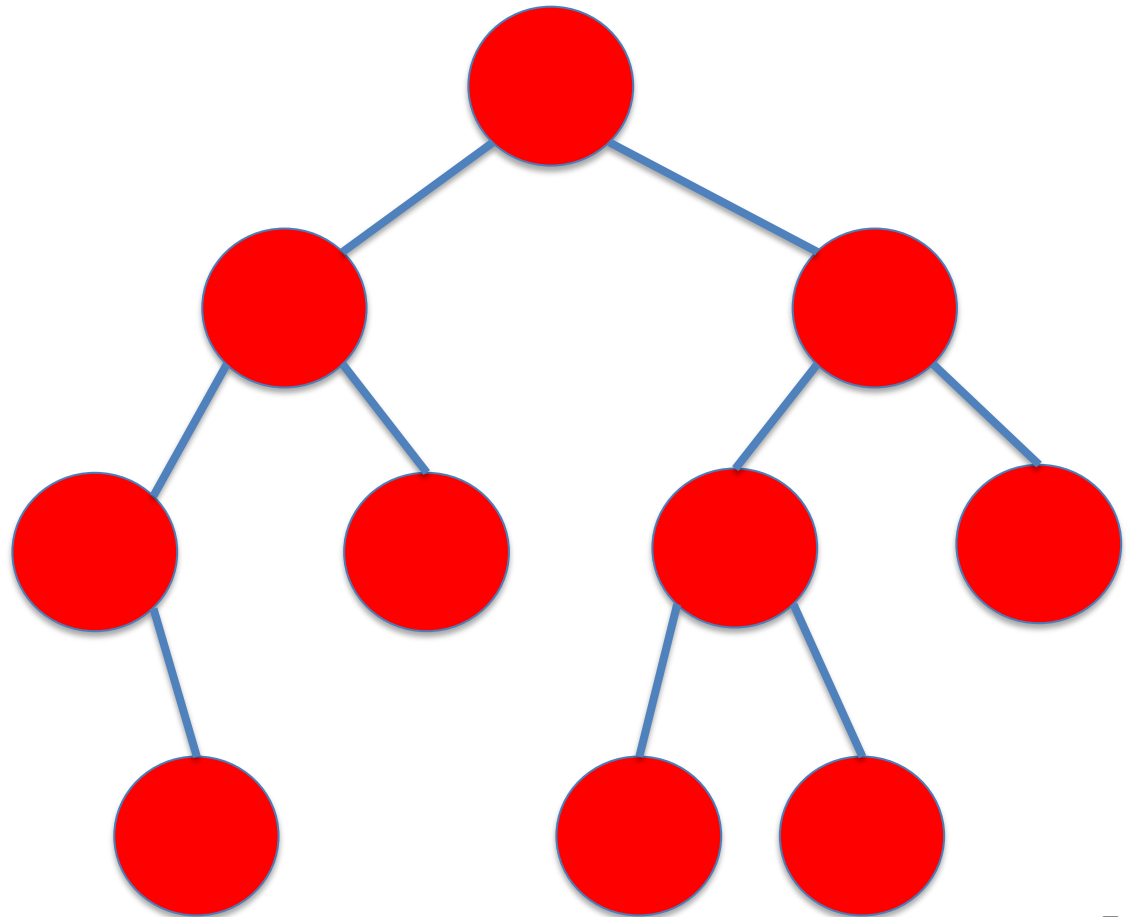
Binary Trees

A **graph** is a collection of **vertices** (nodes) and **edges** that join vertices.



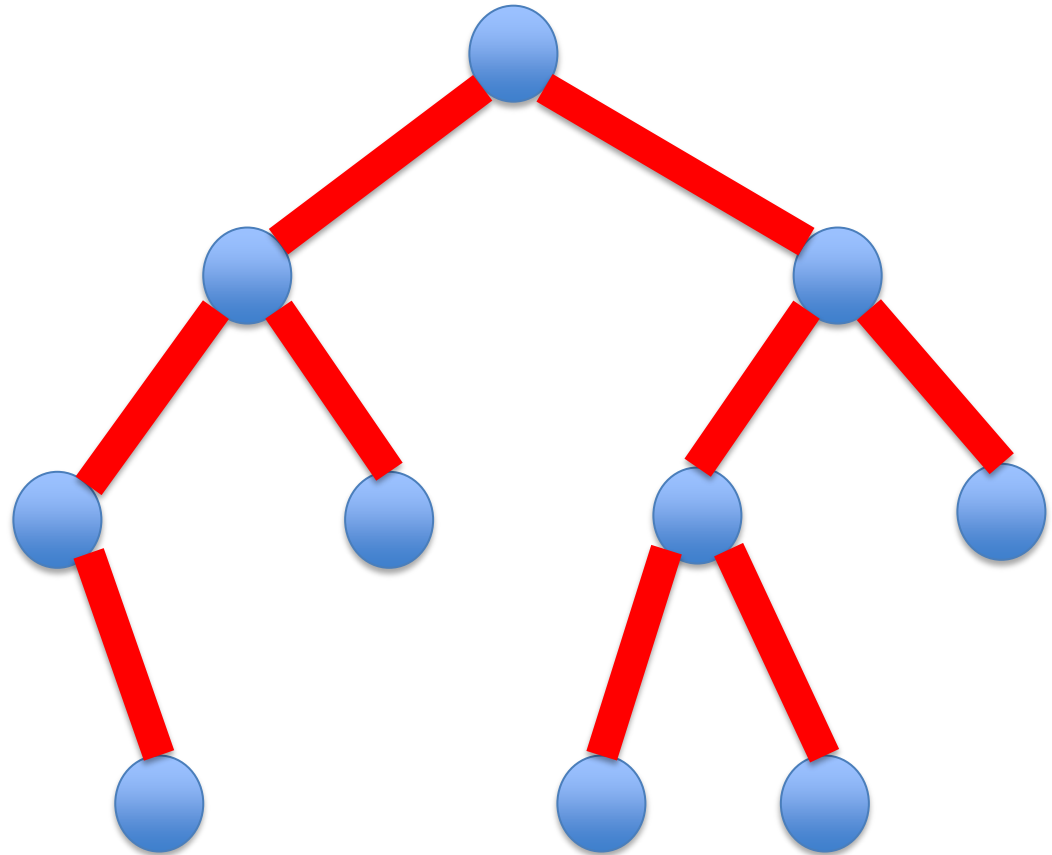
Binary Trees

A **graph** is a collection of **vertices** (**nodes**) and **edges** that join vertices.



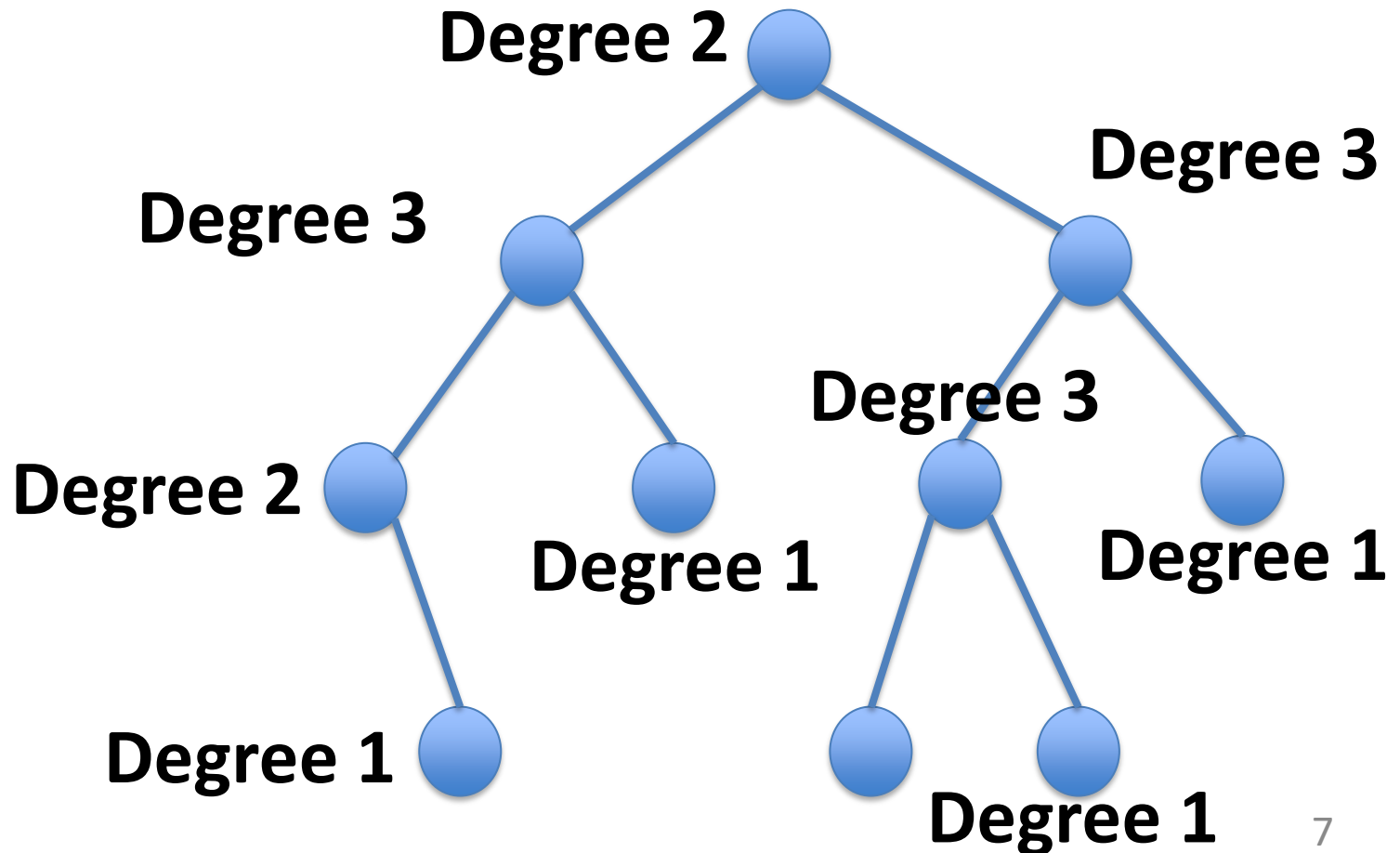
Binary Trees

A **graph** is a collection of **vertices** (nodes) and **edges** that join vertices.



Binary Trees

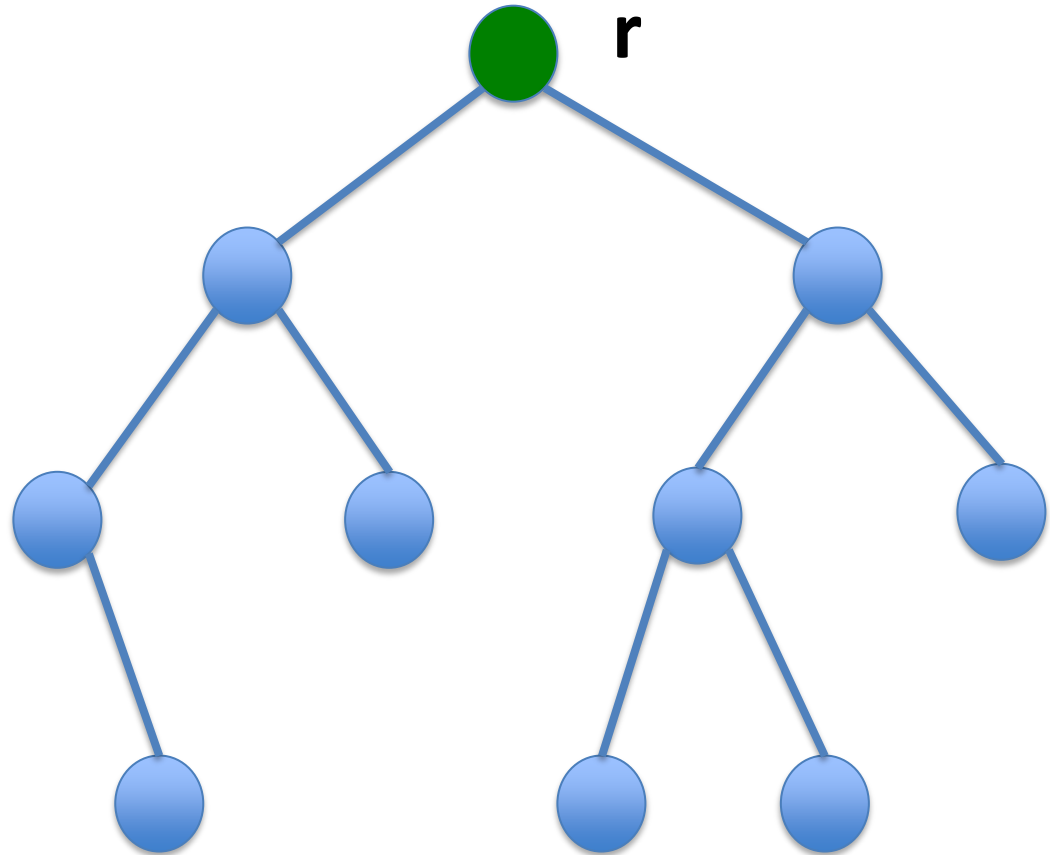
The **degree** of a vertex is the number of edges that are incident to it.



Binary Trees

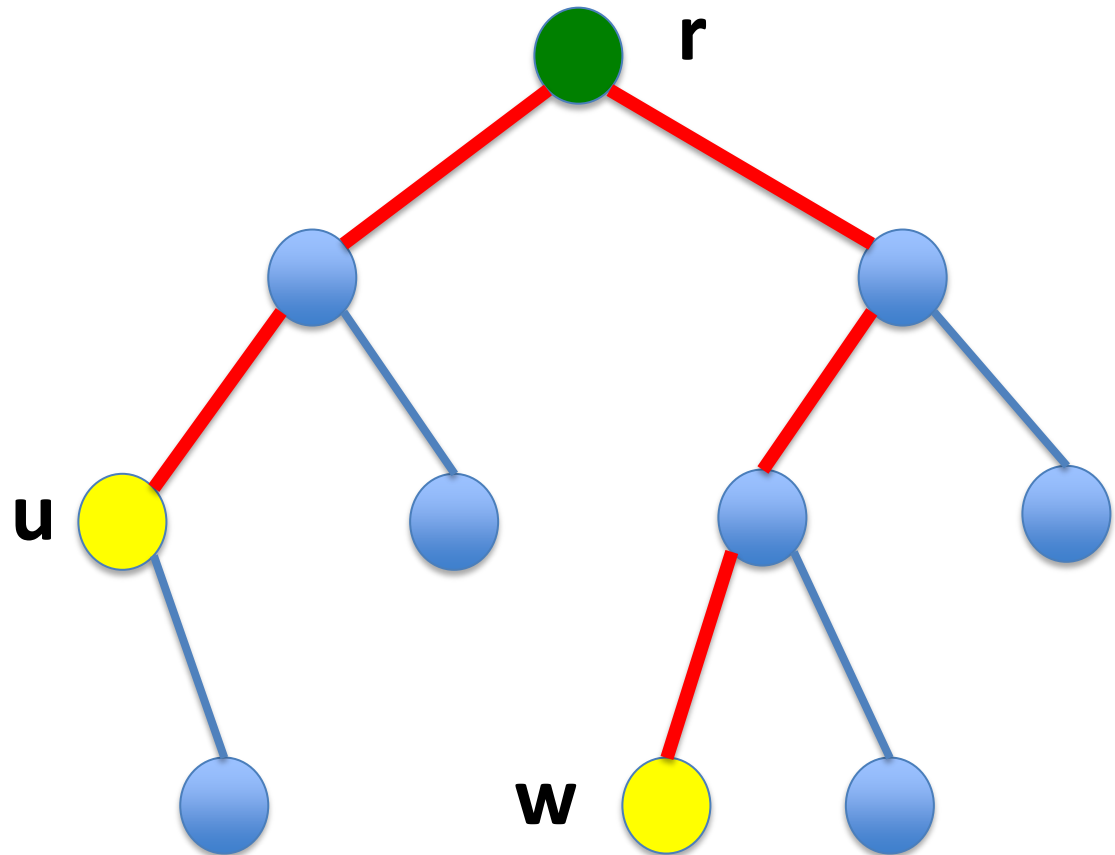
Binary trees are typically **rooted**.

The **root**, denoted **r**, is a special vertex with degree at most 2



Binary Trees

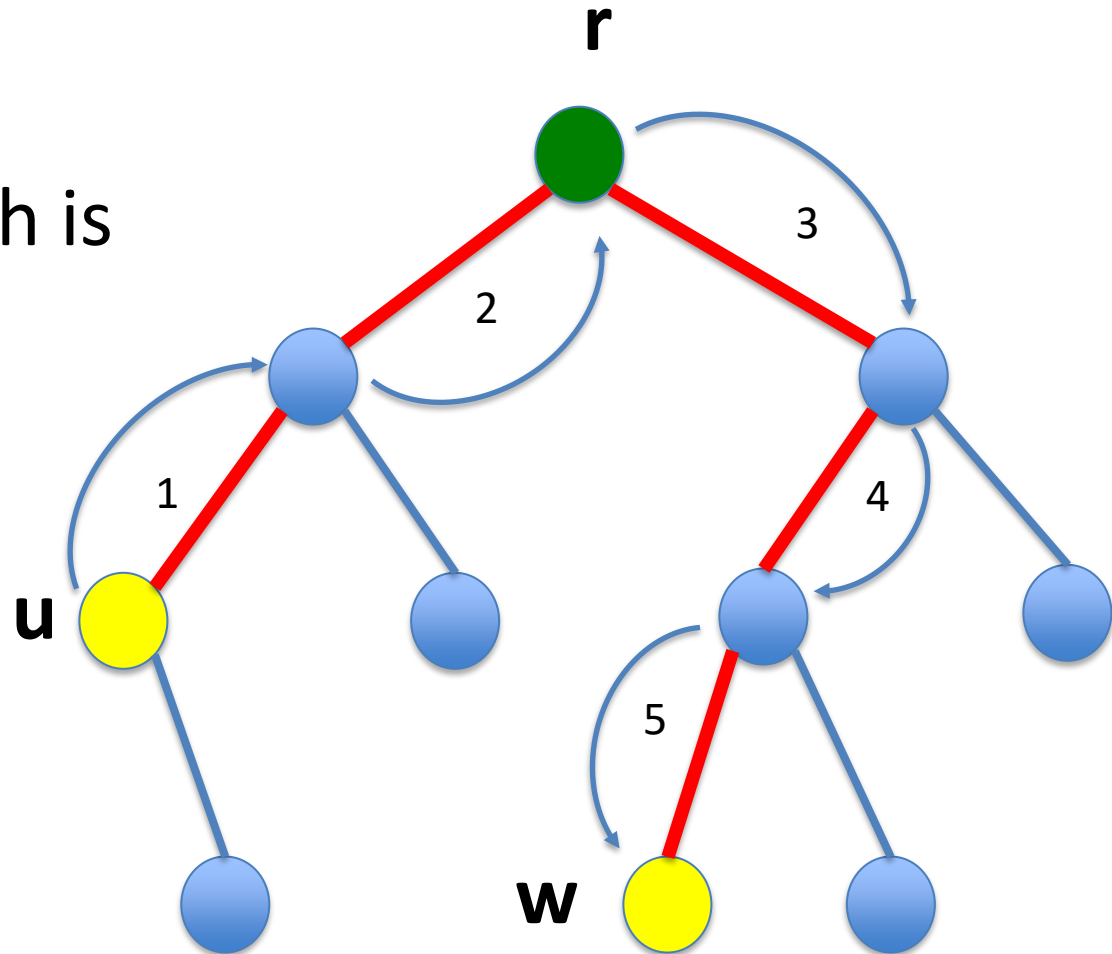
There is a unique path between any two nodes in a binary tree



Binary Trees

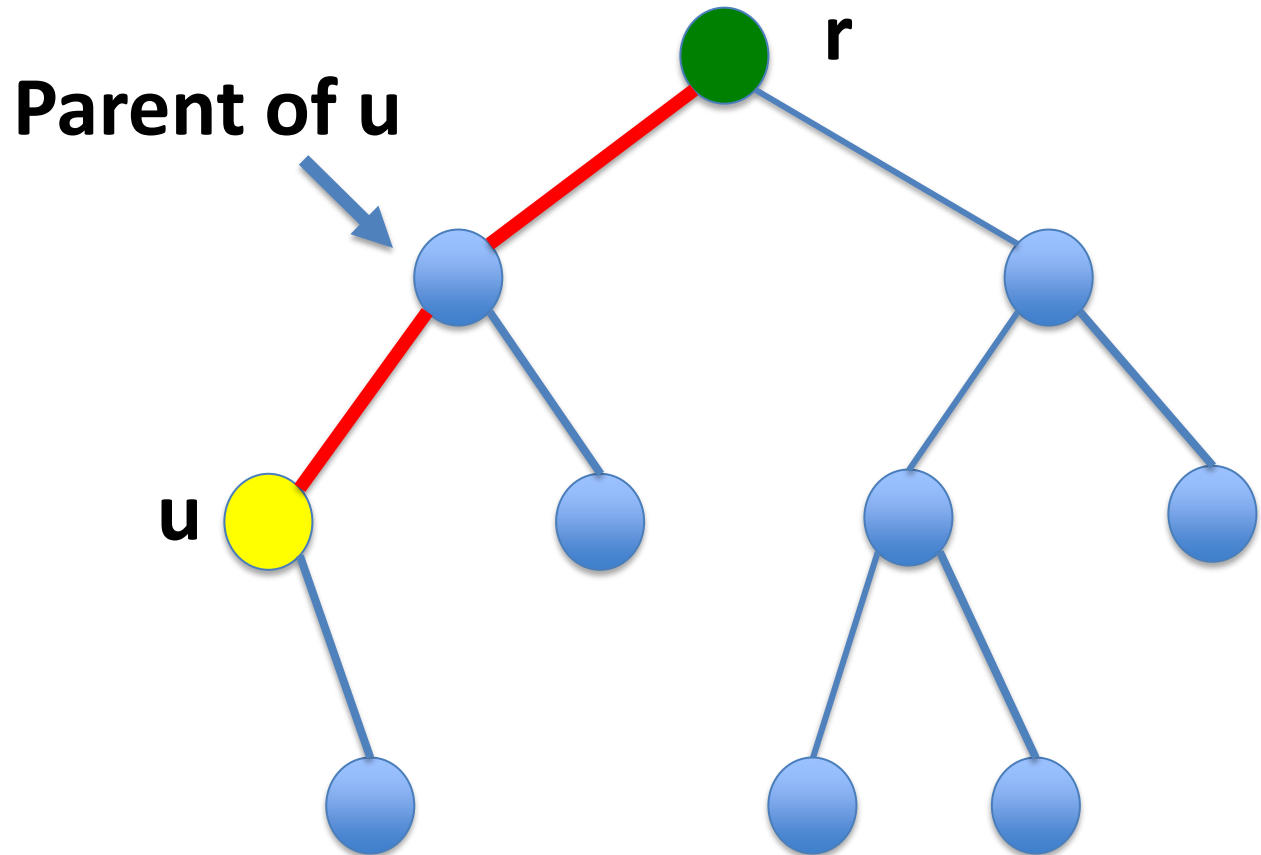
There is a unique path between any two nodes in a binary tree.

Length of a path is the number of edges on the path



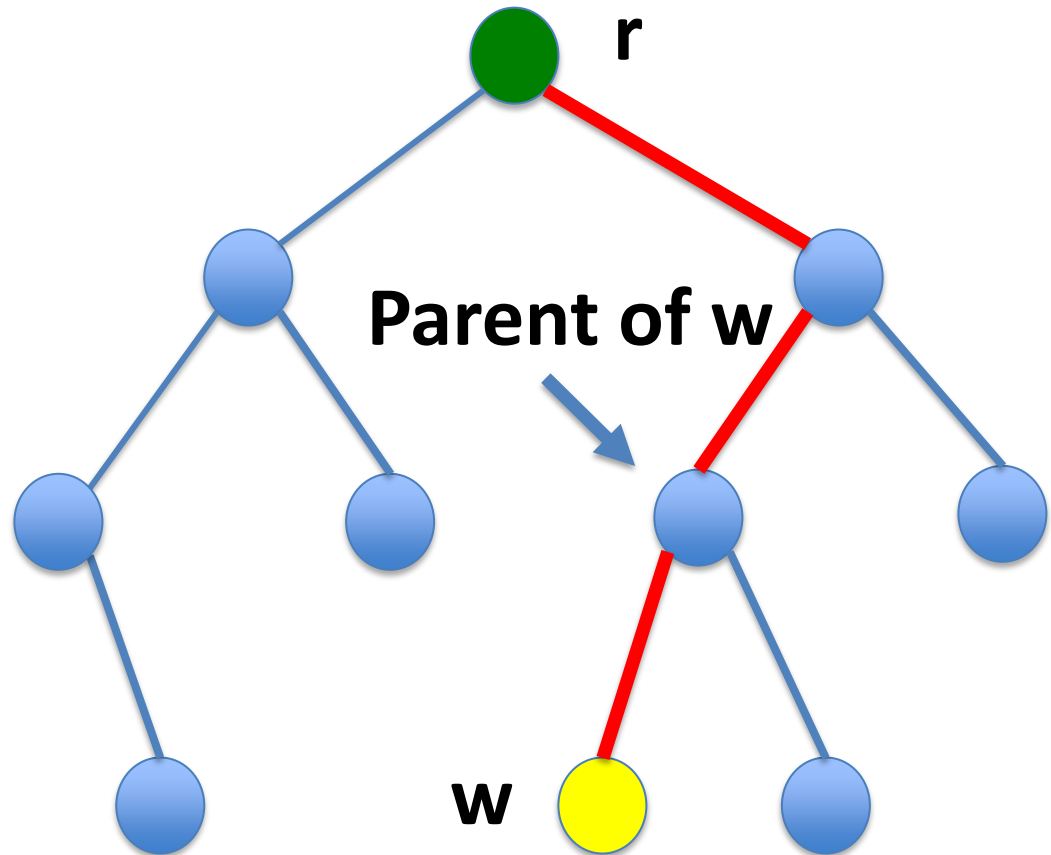
Binary Trees

The second node on the path from any node u to the root is called the **parent** of u



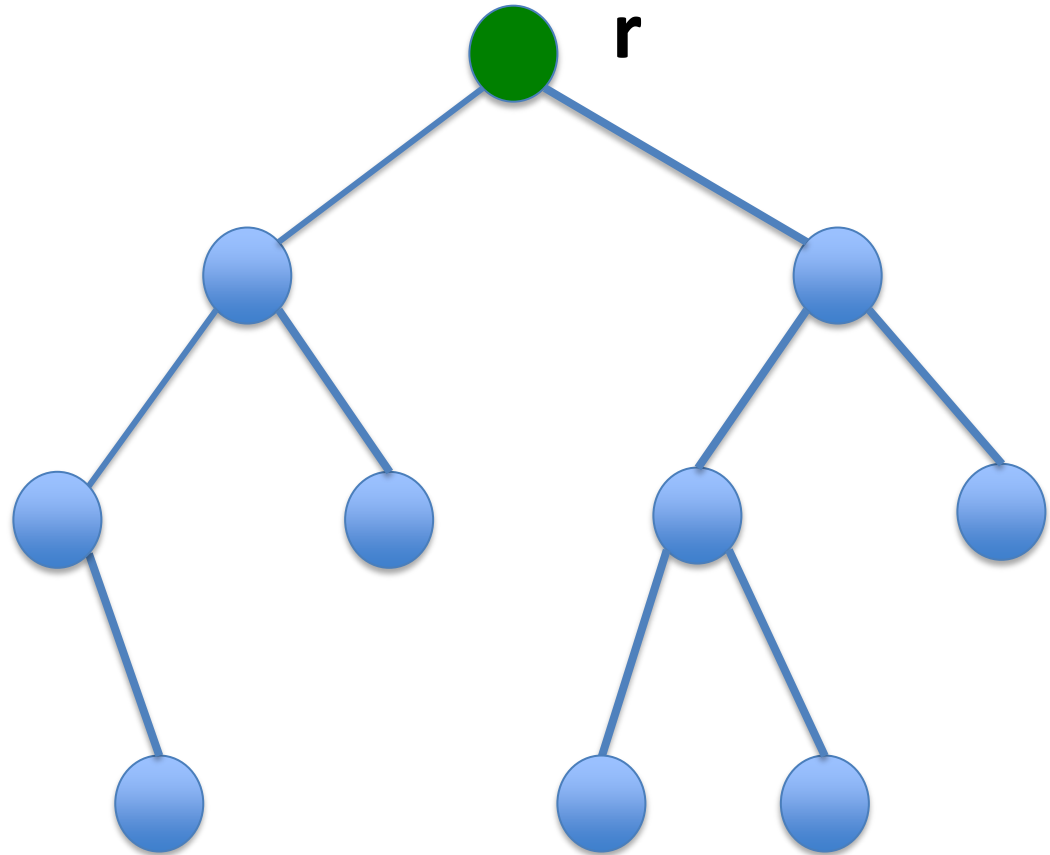
Binary Trees

The second node on the path from any node w to the root is called the **parent** of w



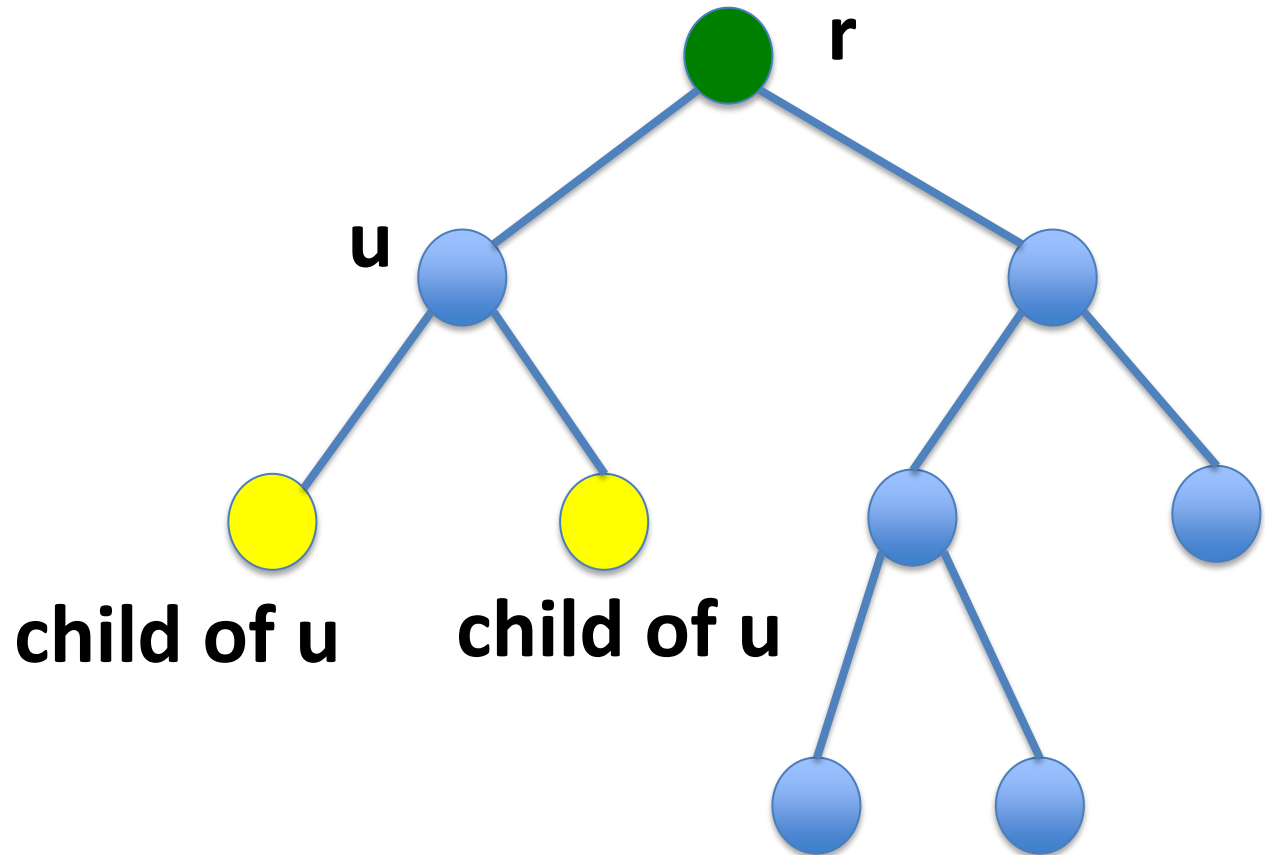
Binary Trees

Every node in a binary tree, except the root, has a parent node.



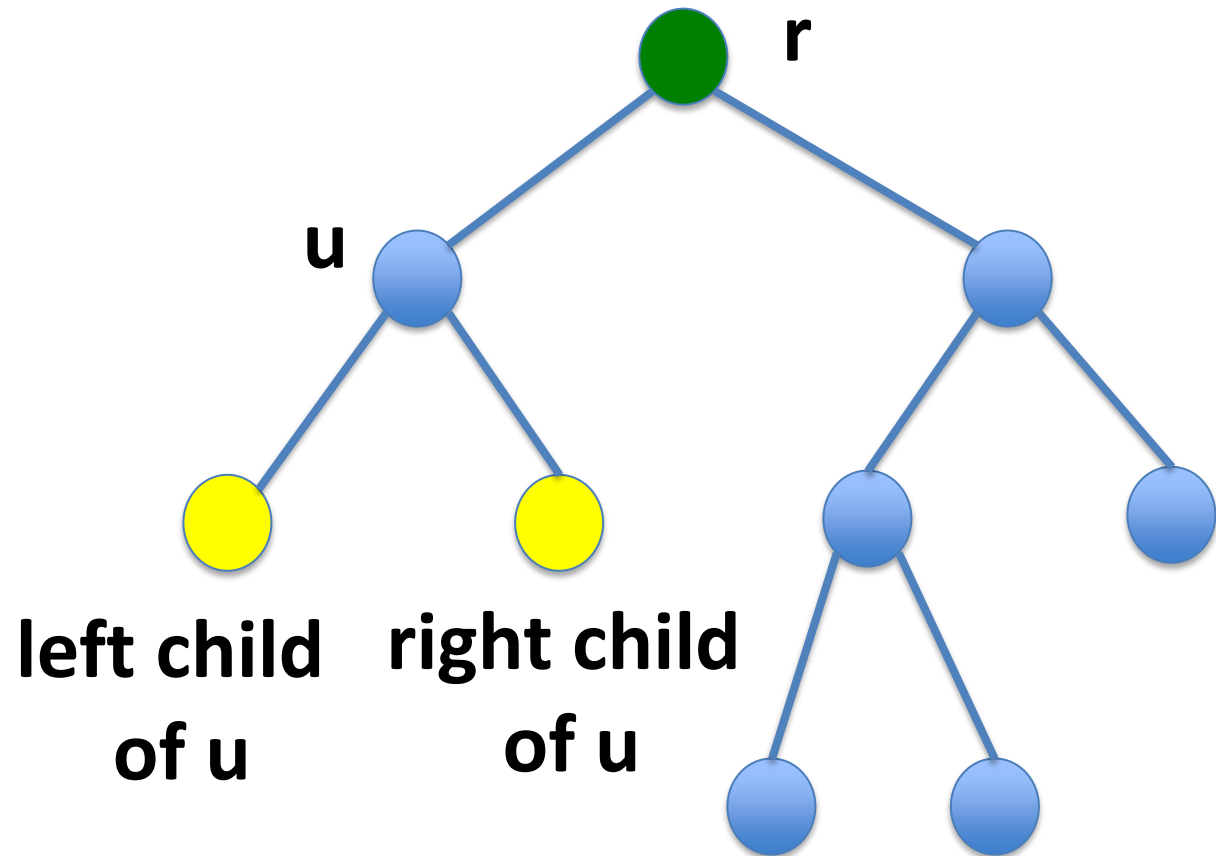
Binary Trees

For a node u , all adjacent nodes, not including the parent of u , are the **children** of u .



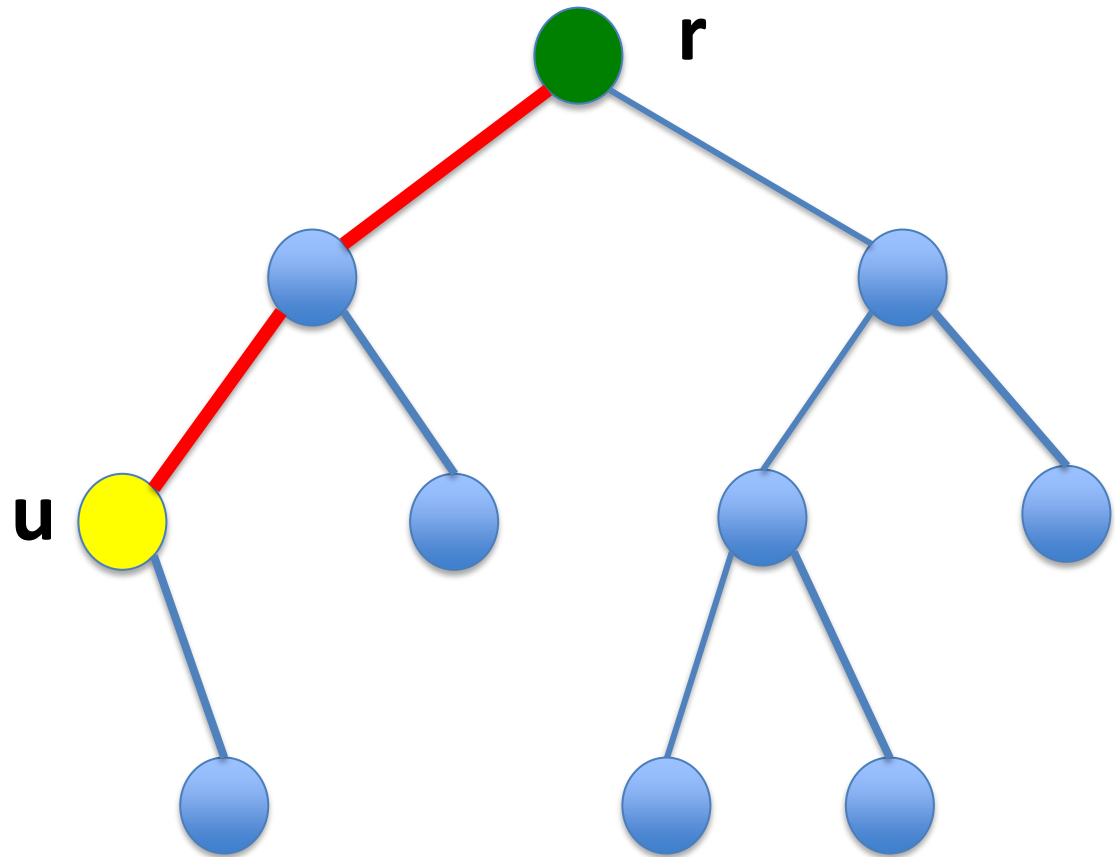
Binary Trees

There is usually an ordering of the trees, so the children are labelled **left** and **right**.



Binary Trees

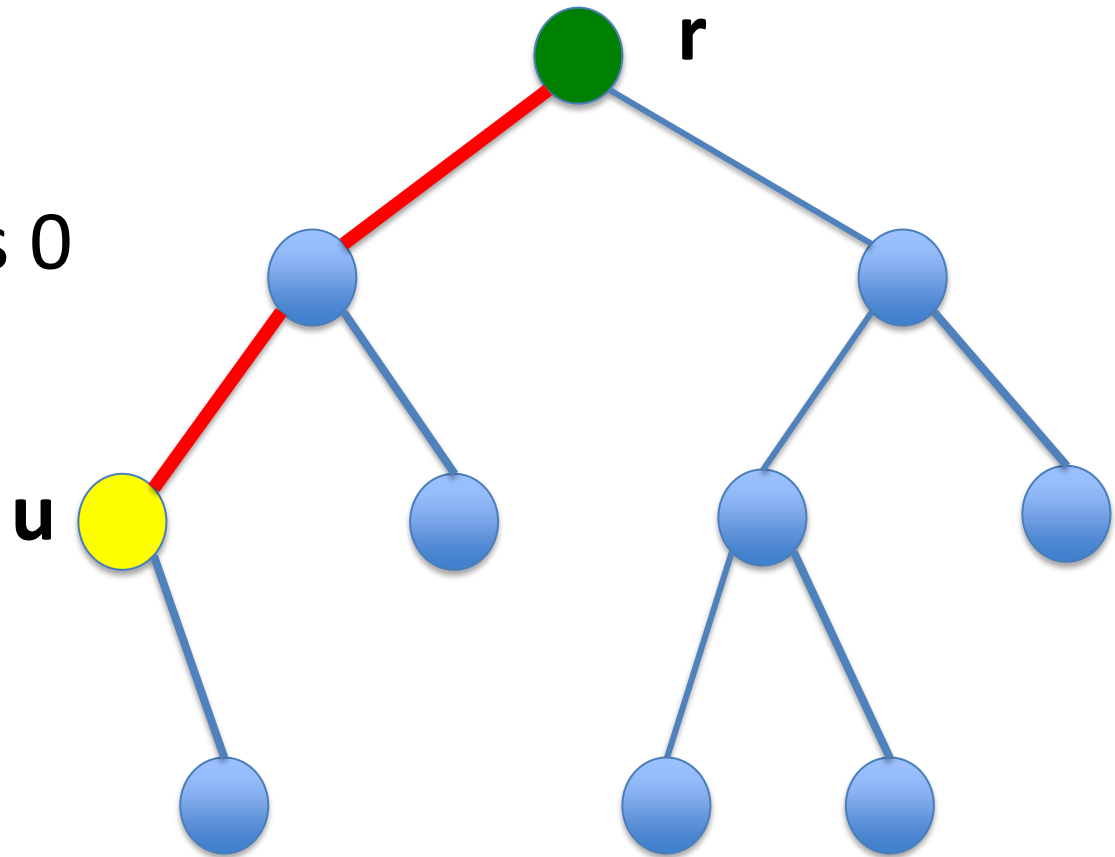
The **depth** of node u is the length of the path from u to the root.



Binary Trees

The **depth** of node u is the length of the path from u to the root.

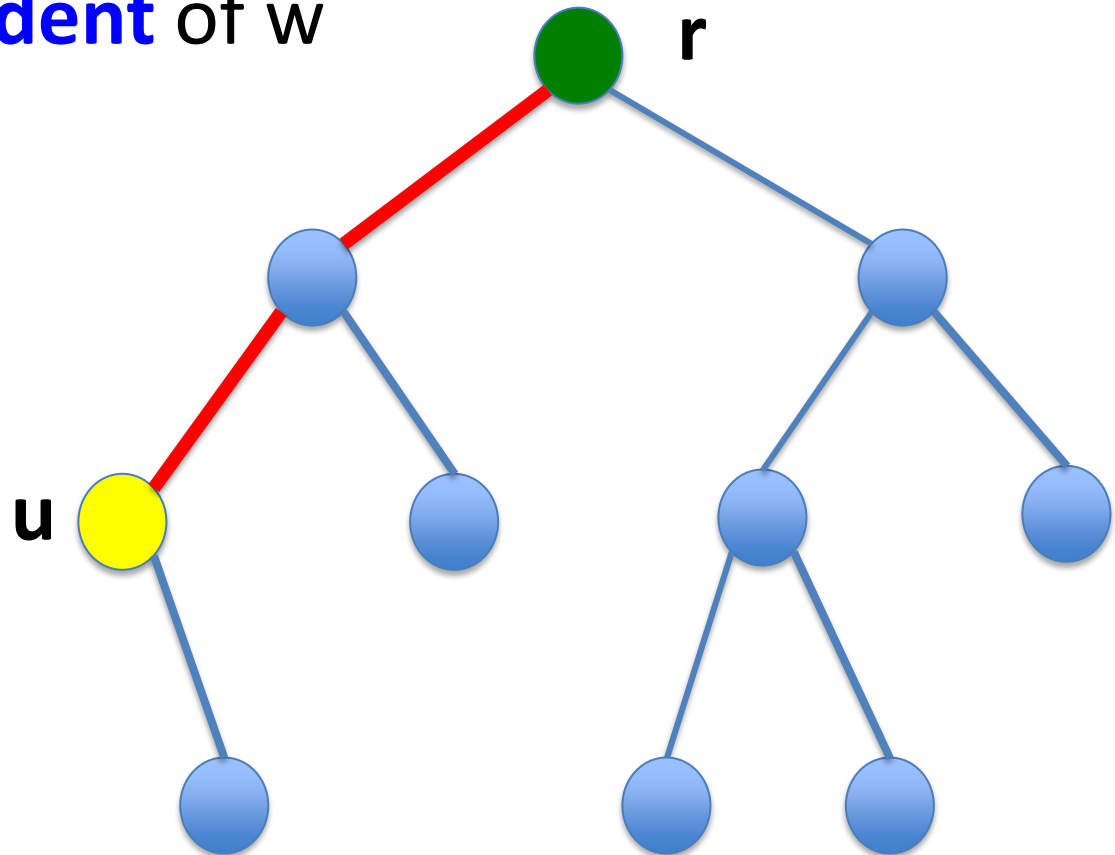
depth of u is 2
depth of root is 0



Binary Trees

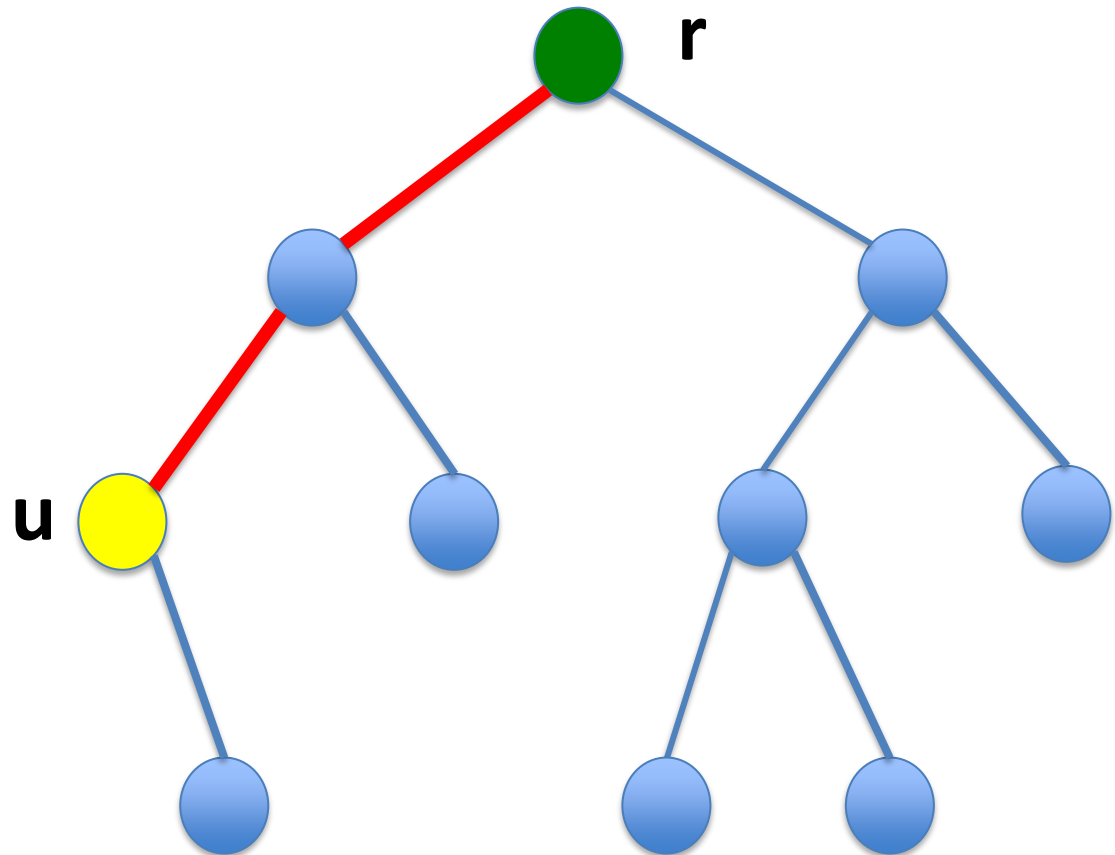
For every node w on the path from u to the root

- w is an **ancestor** of u
- u is a **descendent** of w



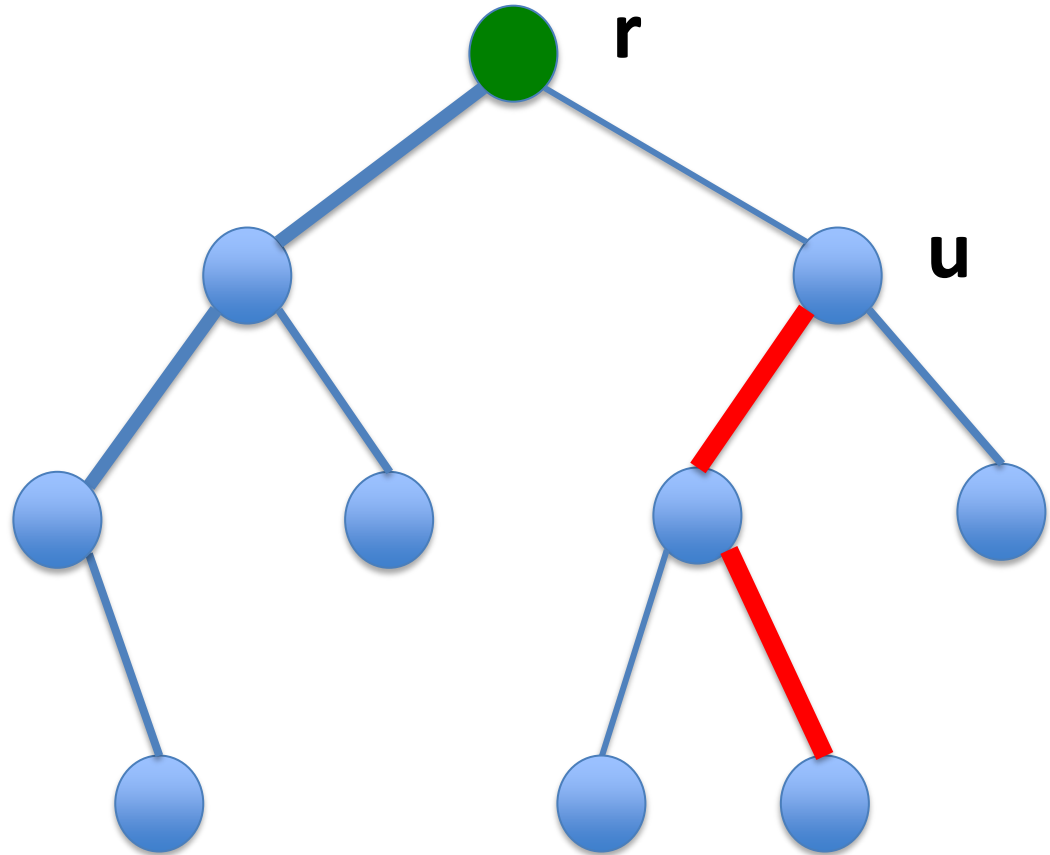
Binary Trees

Every node is both an ancestor and a descendent of itself.



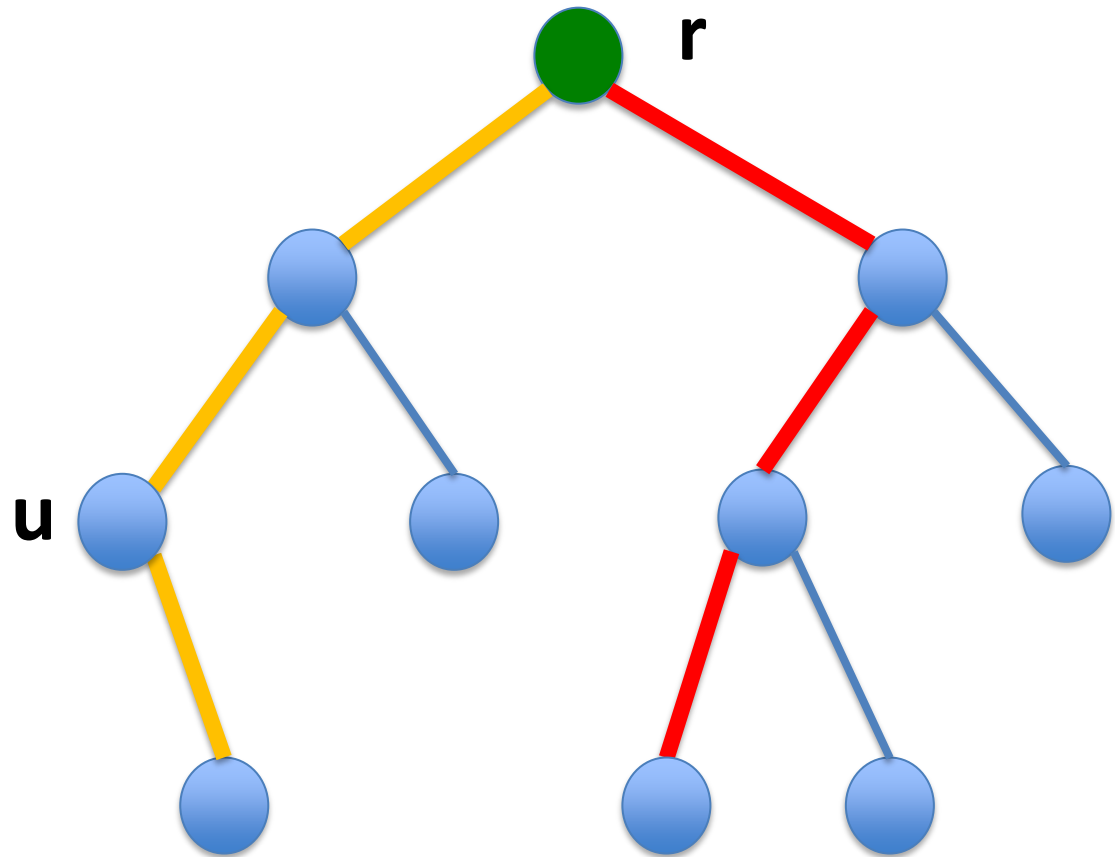
Binary Trees

The **height** of a node u is the length of the **longest** path from u to one of its descendants.



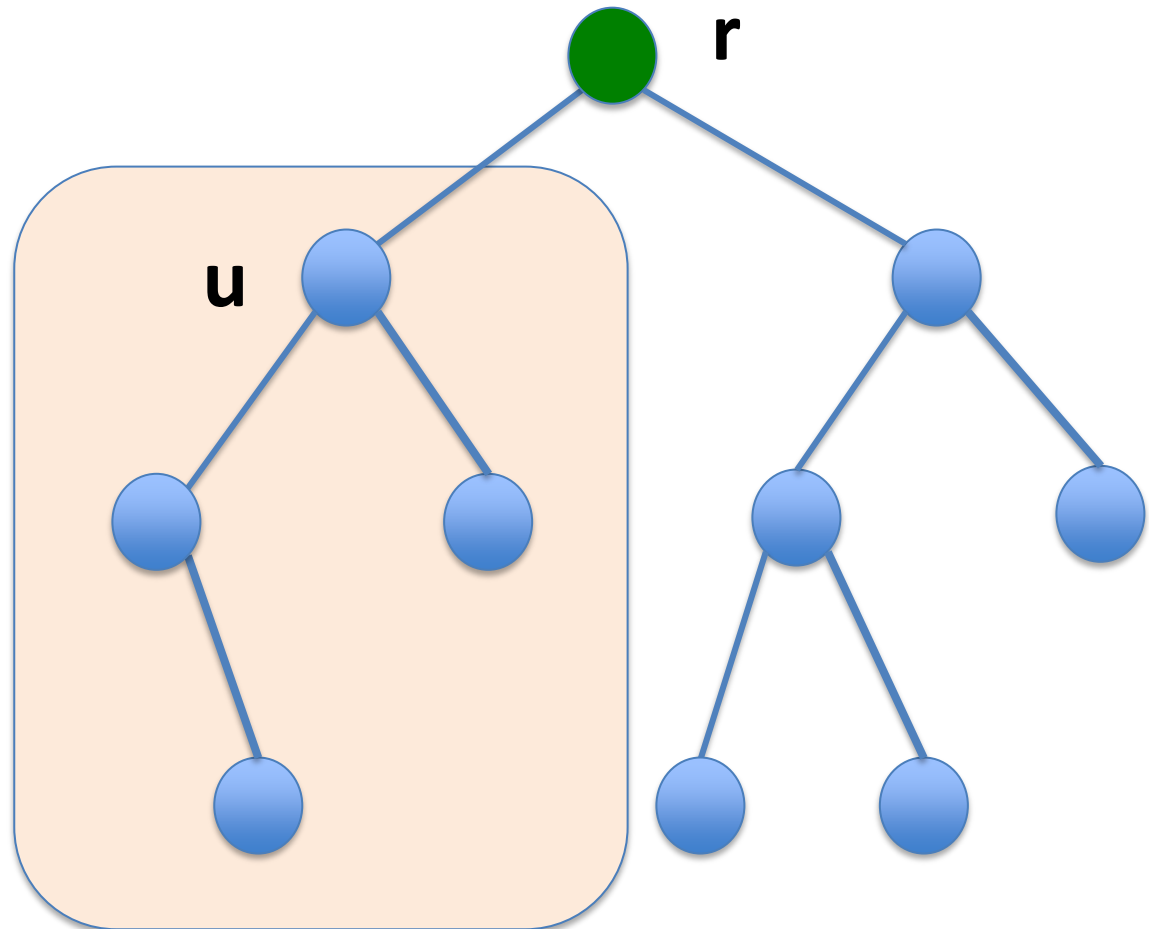
Binary Trees

The **height of a tree** is the height of the root.



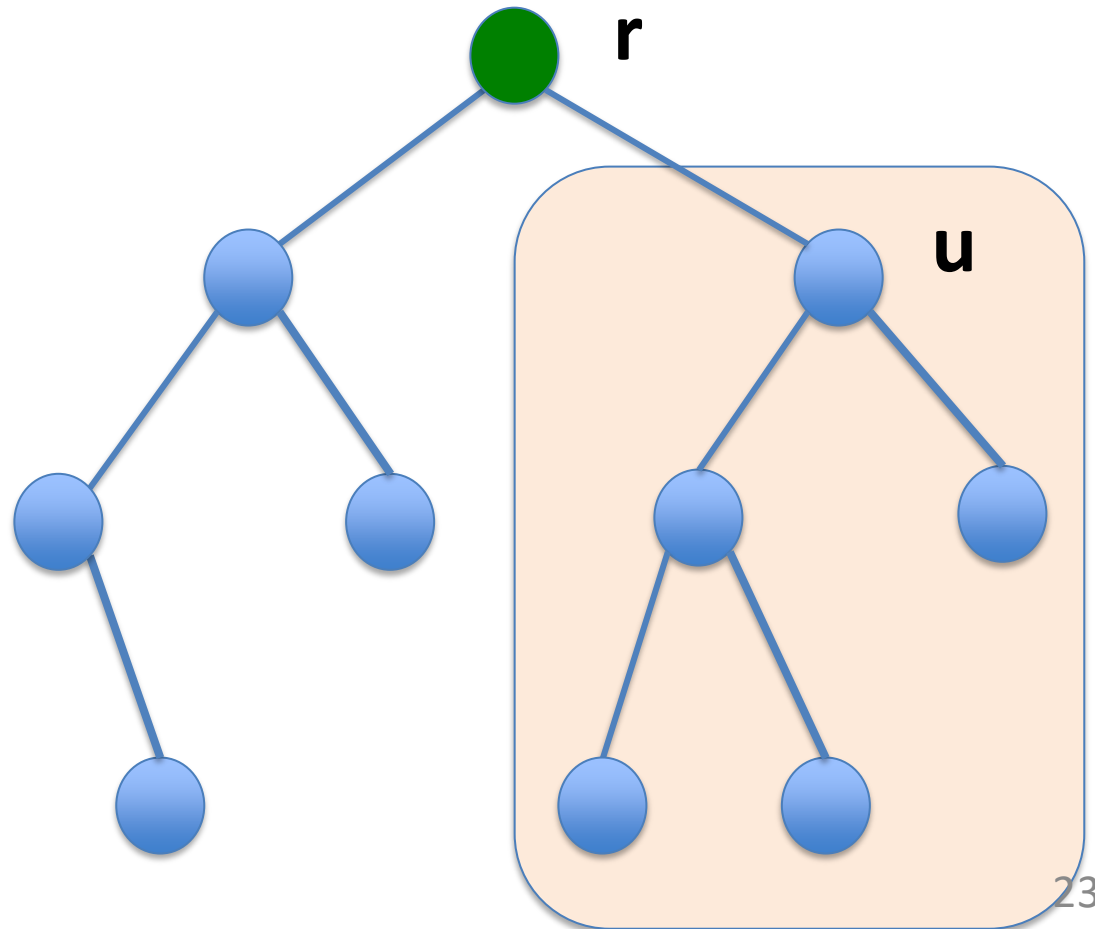
Binary Trees

The **subtree** of a node u is the binary tree rooted at u that contains all of u 's descendants



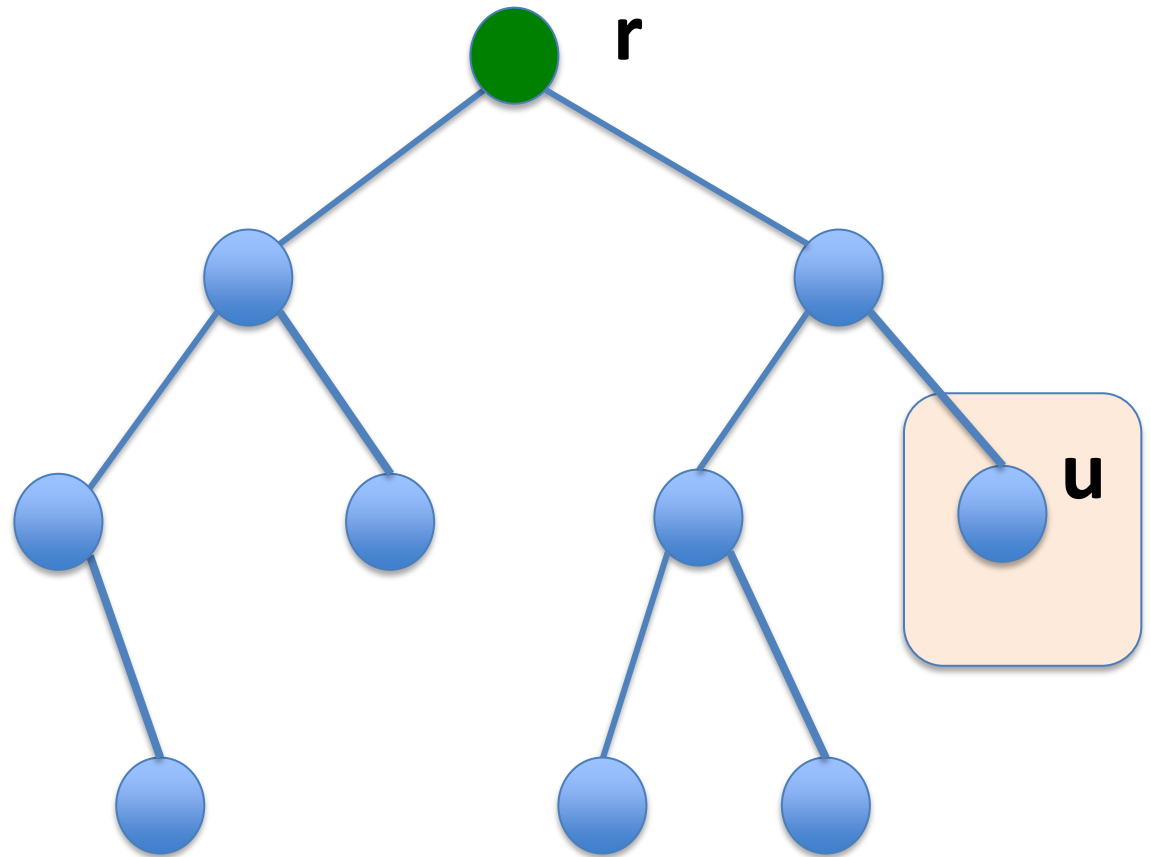
Binary Trees

The **subtree** of a node u is the binary tree rooted at u that contains all of u 's descendants



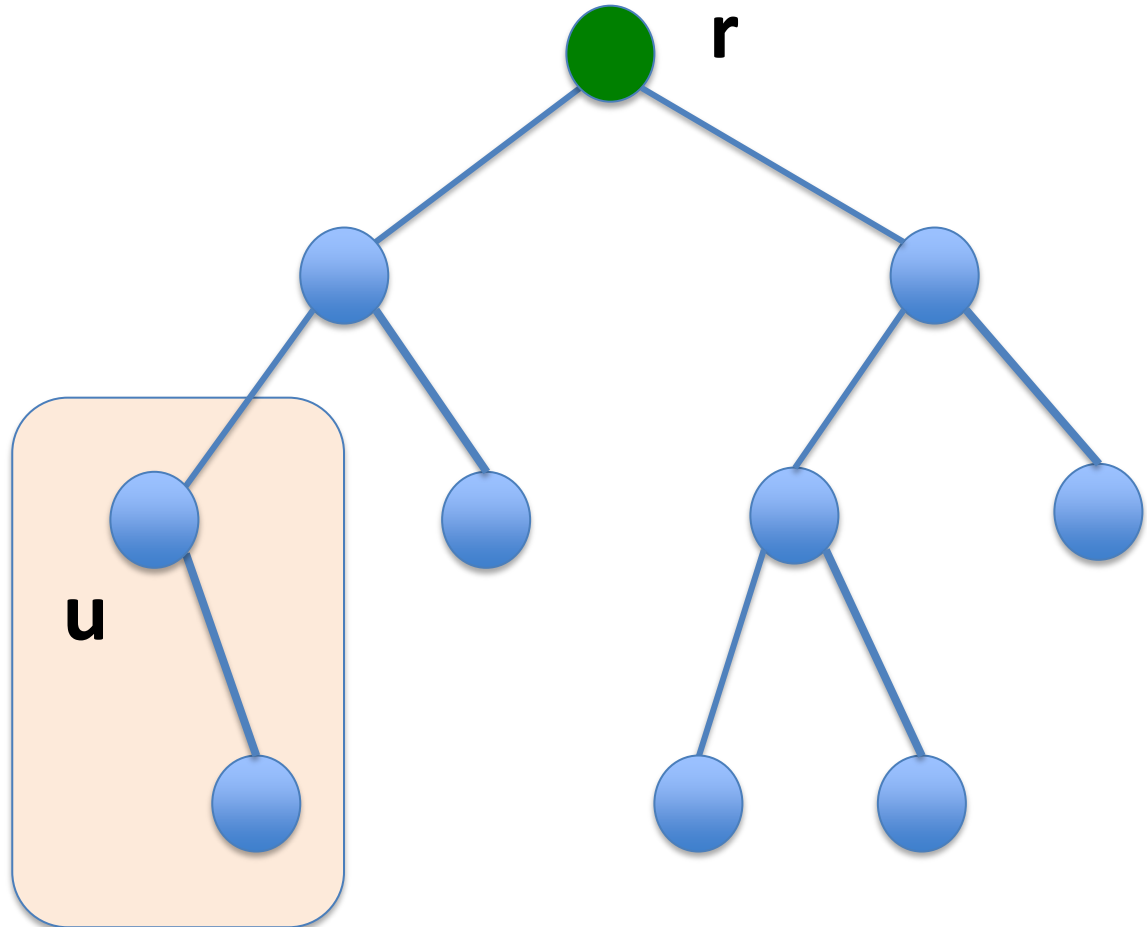
Binary Trees

The **subtree** of a node u is the binary tree rooted at u that contains all of u 's descendants



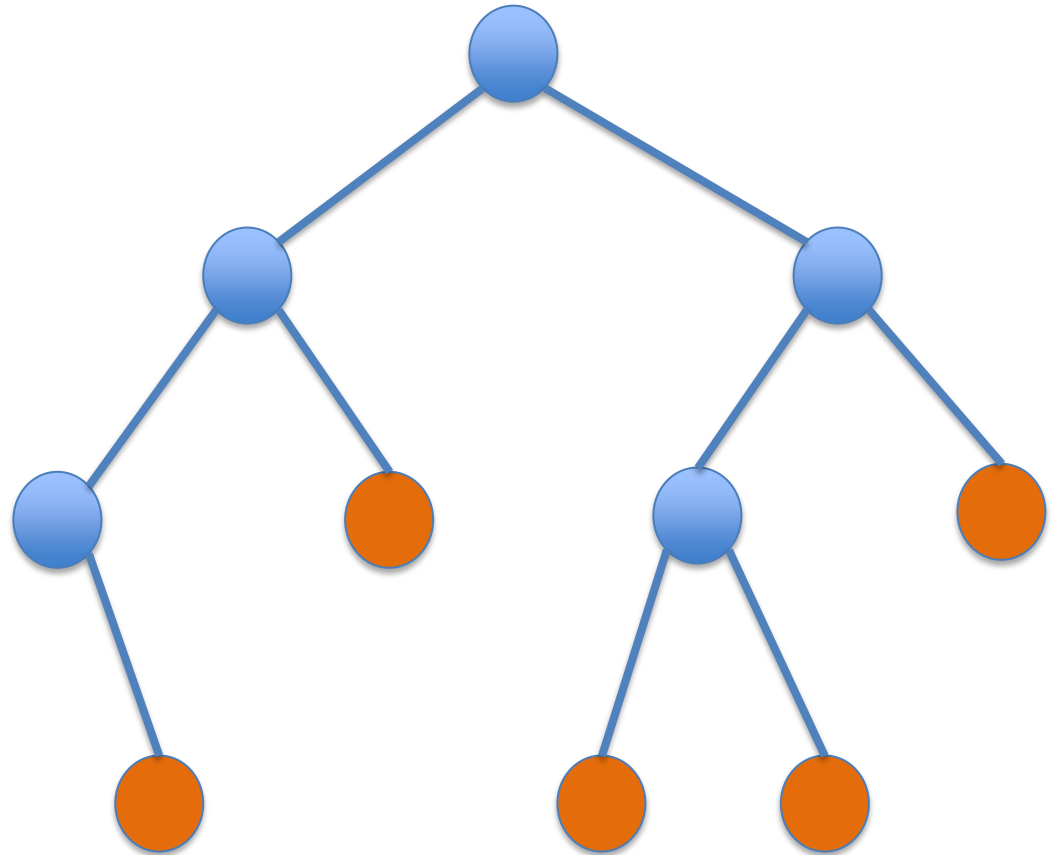
Binary Trees

The **subtree** of a node u is the binary tree rooted at u that contains all of u 's descendants



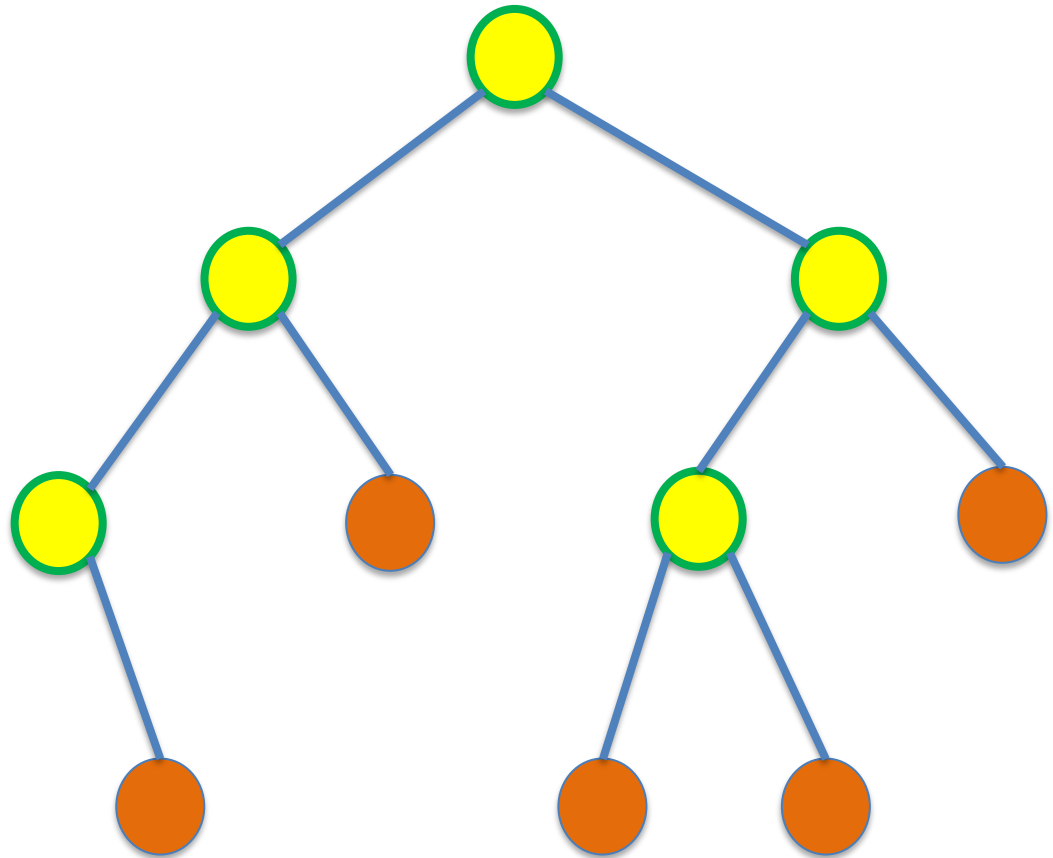
Binary Trees

A node u is called a **leaf** if it has no descendants. Each leaf has height zero.



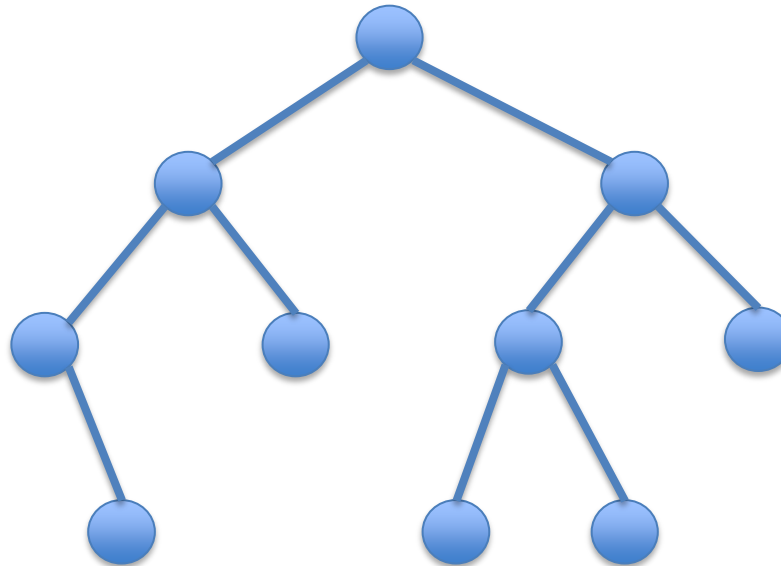
Binary Trees

Non-leaf nodes are called **internal** nodes.



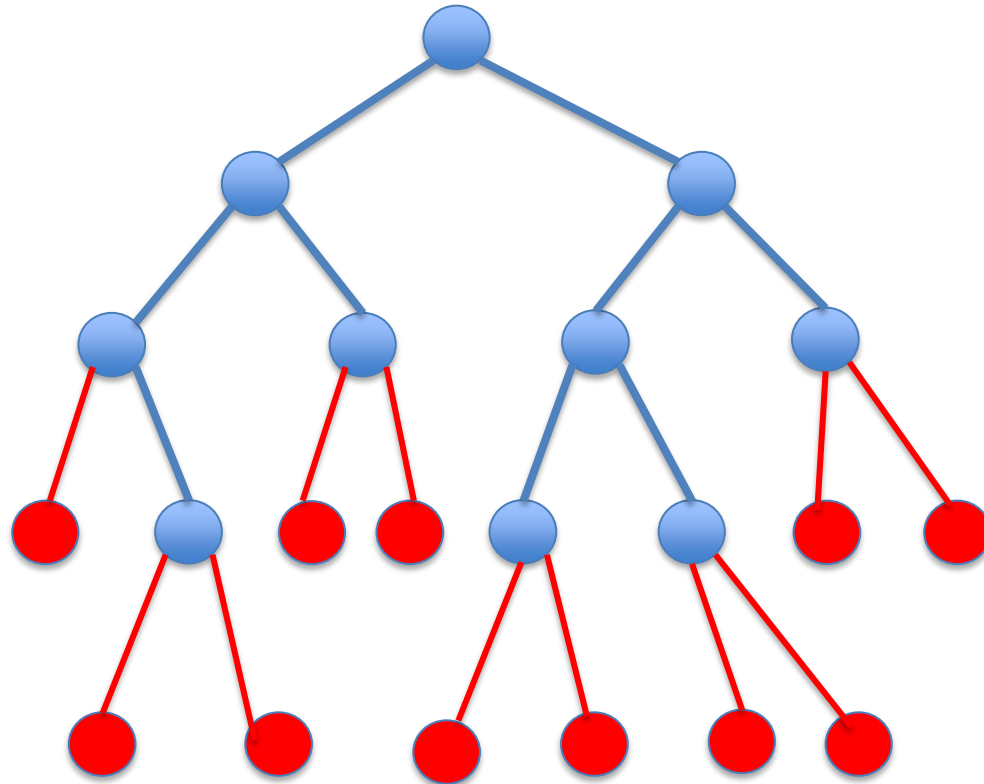
Binary Trees

It is sometimes convenient to augment a tree with external nodes.



Binary Trees

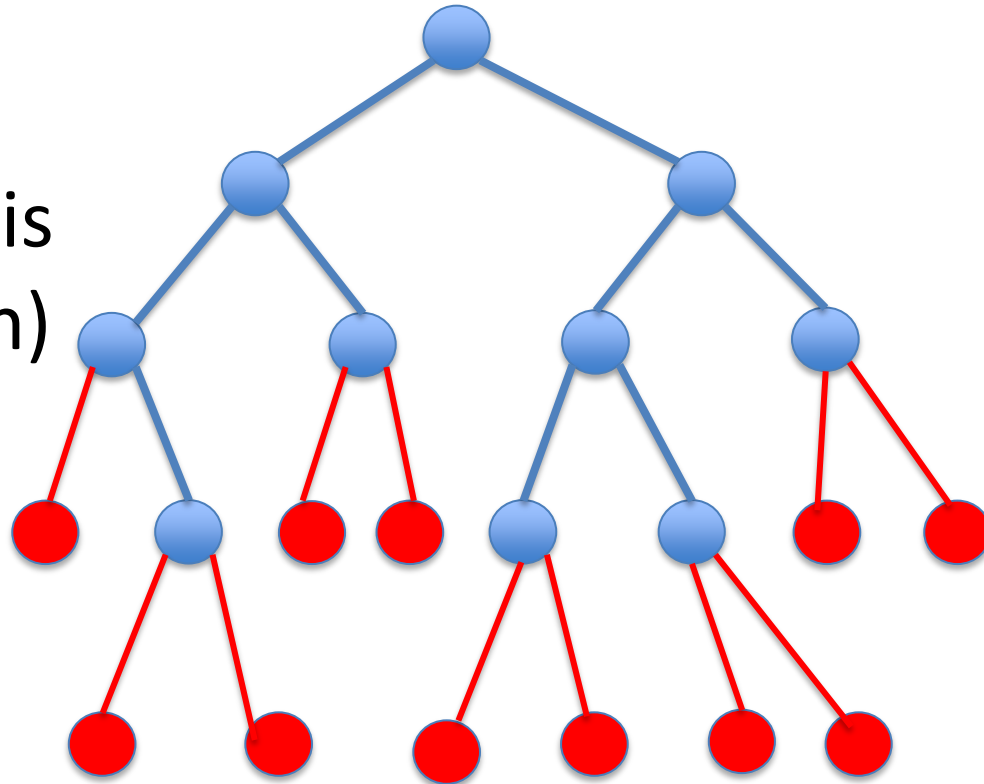
It is sometimes convenient to augment a tree with external nodes. (creating a full/proper binary tree)



Binary Trees

In this augmented binary tree, if there are $n \geq 1$ internal (real) nodes then there are always $n + 1$ leaves.

(try to prove this
using induction)



Binary Trees

Just like a linked list is a collection of **nodes** (data and a reference to another node), we can implement a binary tree with **nodes** that have data and one, two or three references to other nodes.

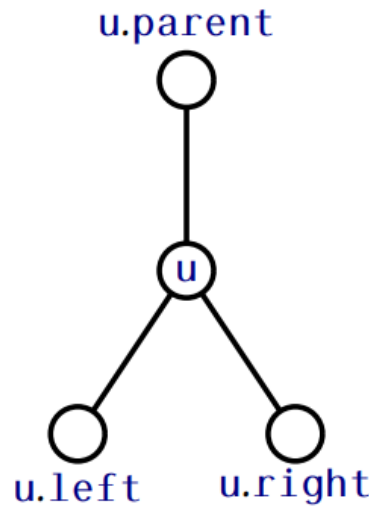
Binary Trees

Just like a linked list is a collection of **nodes** (data and a reference to another node), we can implement a binary tree with **nodes** that have data and one, two or three references to other nodes.

What can we say about any given node based on the number of non-null (non-nil) references it has?

Binary Trees

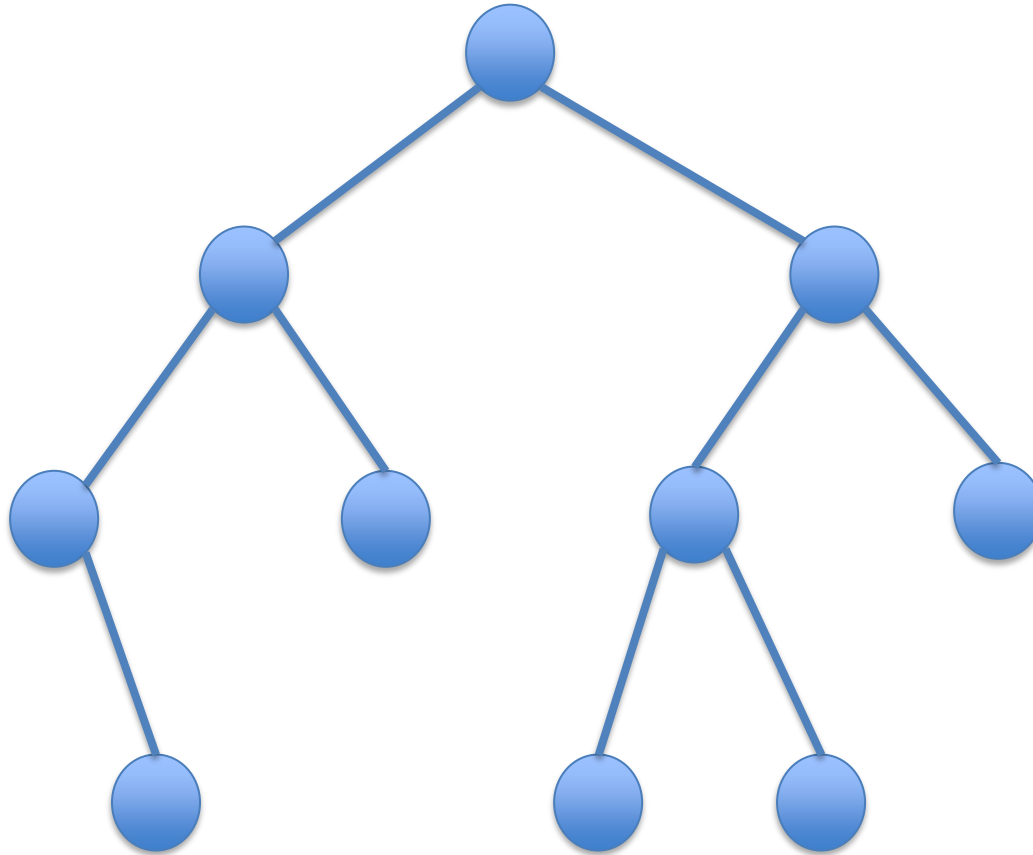
```
class BTreeNode <Node extends BTreeNode<Node>>{  
    Node left;  
    Node right;  
    Node parent;  
}
```



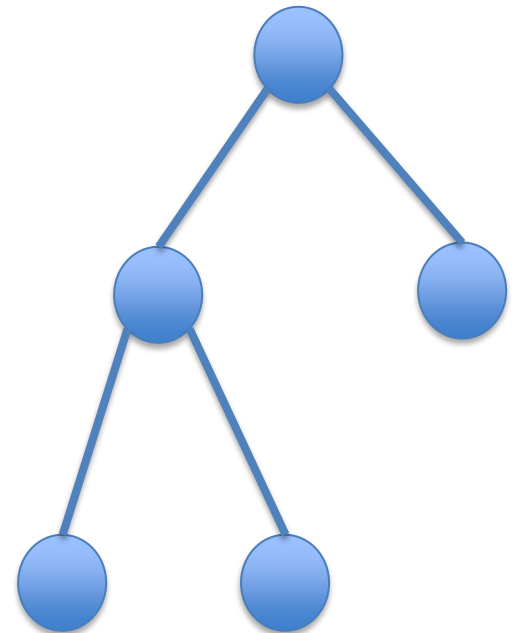
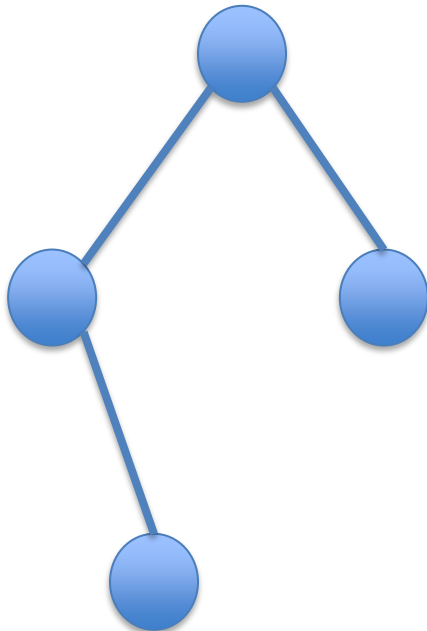
We use **nil** to denote the absence of an adjacent node (parent or child)

nil is a special instantiation of the class (it is not null)

Recursive Nature of Binary Trees



Recursive Nature of Binary Trees



Recursive Nature of Binary Trees

When solving a problem recursively we always have

1. A **base case**, which is an instance of the problem that we can solve (we know the answer)
2. A **recursive case** in which we try to solve an easier (typically smaller) instance of the problem

Recursive Nature of Binary Trees

Find the depth (distance from the root) of a node.

1. use recursion
2. use iteration

Recursive Nature of Binary Trees

Find the depth of a node using recursion.

```
int depth(Node u) {  
    if (u == r) { // r is the root  
        return 0;  
    } else {  
        return 1 + depth(u.parent);  
    }  
}
```

Recursive Nature of Binary Trees

Find the depth of a node using iteration.

```
int depth(Node u) {  
    int d = 0;  
    while ( u != r ){    // r is the root  
        u = u.parent;  
        d += 1;  
    }  
    return d;  
}
```

Recursive Nature of Binary Trees

Using recursion, find

1. The depth of a given node
2. The height of a given node
3. The shortest path length from a given node to any leaf
4. The size of a binary tree
(number of real/internal nodes)

Recursive Nature of Binary Trees

What happens if we don't have a parent reference in each node?

Traversing a Binary Trees

We can use recursion to visit every node in a binary tree. This is called a **traversal**.

Traversing a Binary Trees

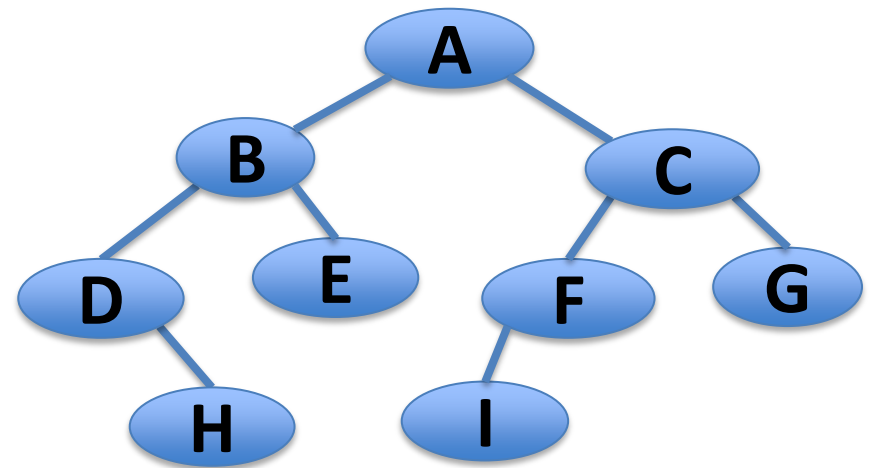
```
void traverse(Node u) {  
    if (u == nil) return;    // base case  
  
    traverse(u.left);  
  
    traverse(u.right);  
  
}
```

Traversing a Binary Trees

```
void traverse(Node u) {  
    if (u == nil) return;    // base case  
    // pre-order  
    traverse(u.left);  
    // in-order  
    traverse(u.right);  
    // post-order  
}
```

Binary Trees

- pre-order?
- inorder?
- post-order?



Binary Trees

Whatever we can compute using recursion we can also compute without using recursion.

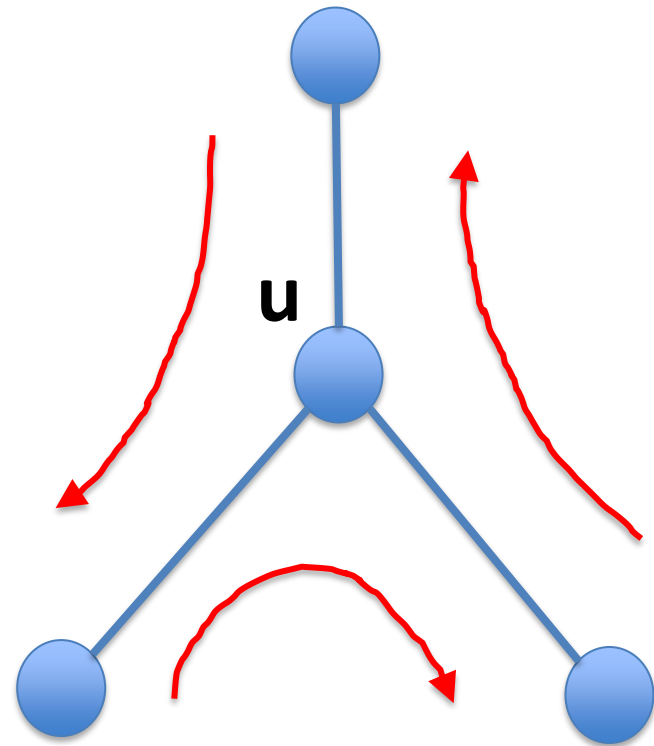
For this we need to keep track of where we came from when coming to a node. We'll follow the same pattern as the basic recursive traversal (as illustrated on the next slide).

Note: If we don't have a parent reference (not all implementations have a parent) we'll need to keep other information stored as well.

Binary Trees

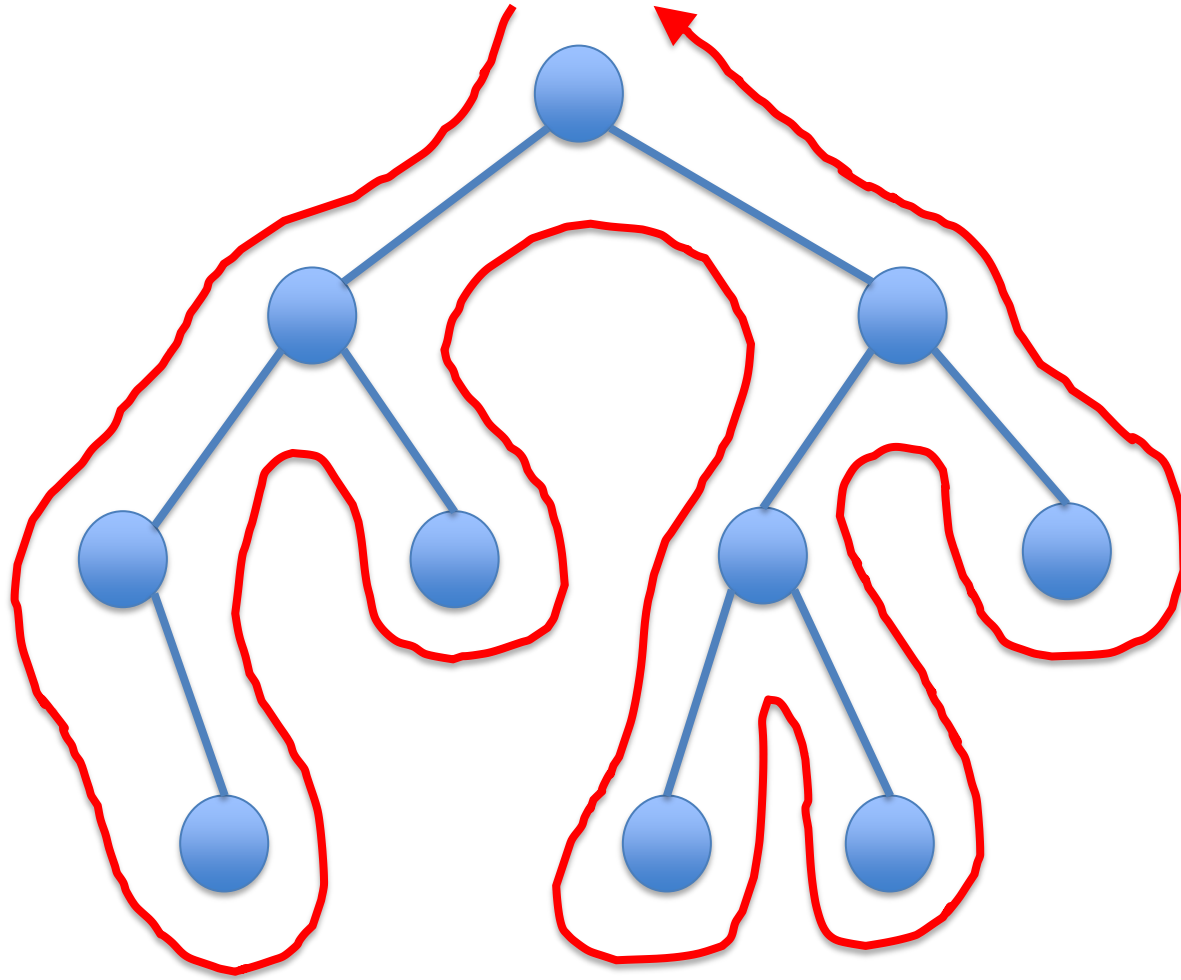
Non-recursive traversals of a binary tree. Three cases to consider...

- visit left child if coming from parent
- visit right child if coming from left child
- go back to parent if coming from right child



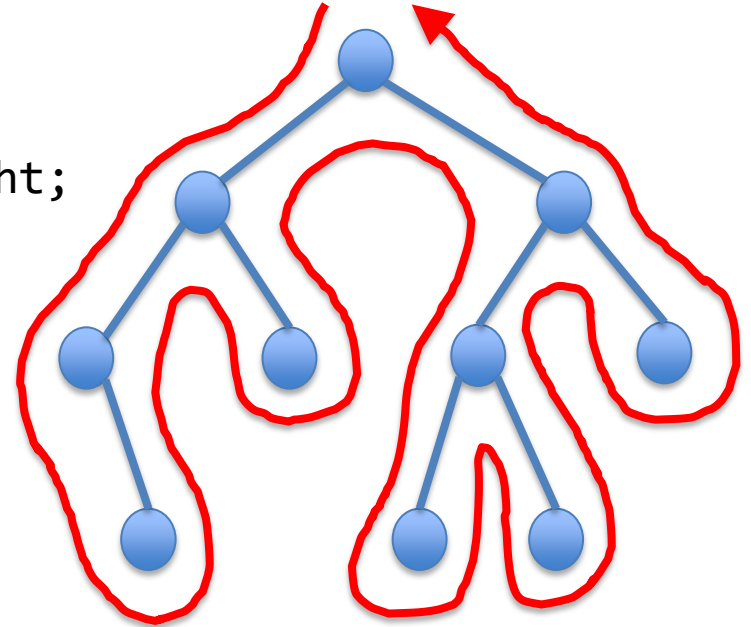
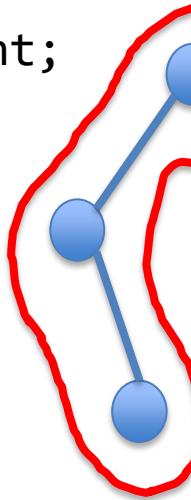
Binary Trees

Non-recursive traversals of a binary tree.



Binary Trees

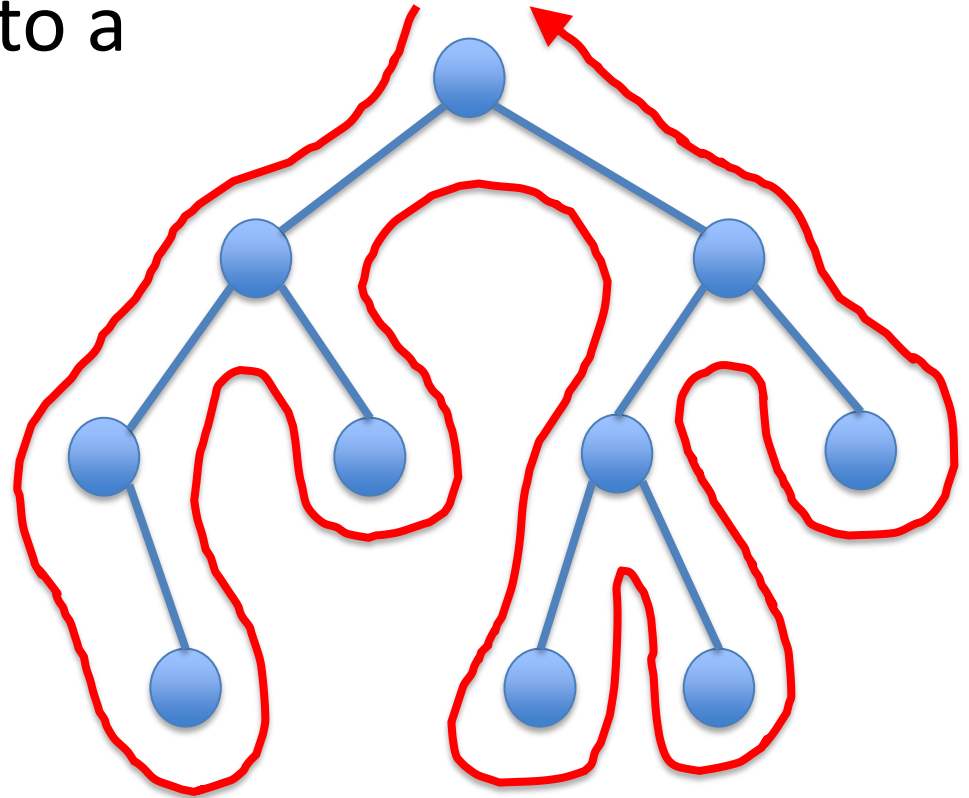
```
void traverse2() {
    Node u=r, prev=nil, next;
    while( u != nil ){
        if( prev == u.parent ){
            if( u.left != nil) next = u.left;
            else if( u.right != nil) next = u.right;
            else next = u.parent;
        } else if( prev == u.left ){
            if( u.right != nil ) next = u.right;
            else next = u.parent;
        } else {
            next = u.parent;
        }
        prev = u;
        u = next;
    }
}
```



Binary Trees

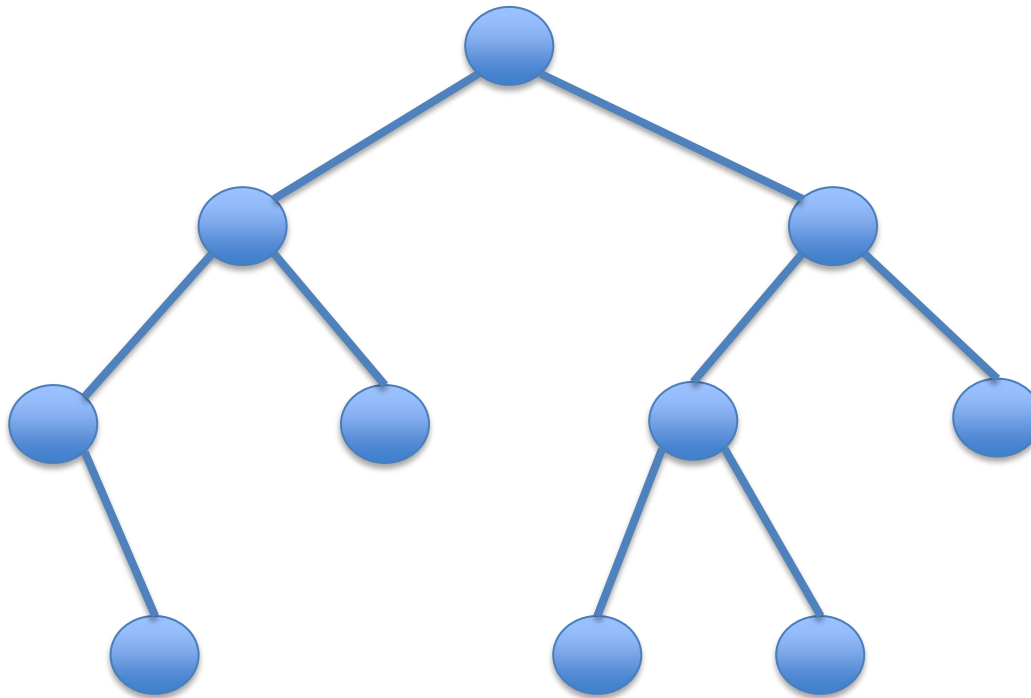
Where would you add a print statement in the `traverse2()` method so that the order of nodes printed was equivalent to a

- pre-order traversal
- inorder traversal
- post-order traversal



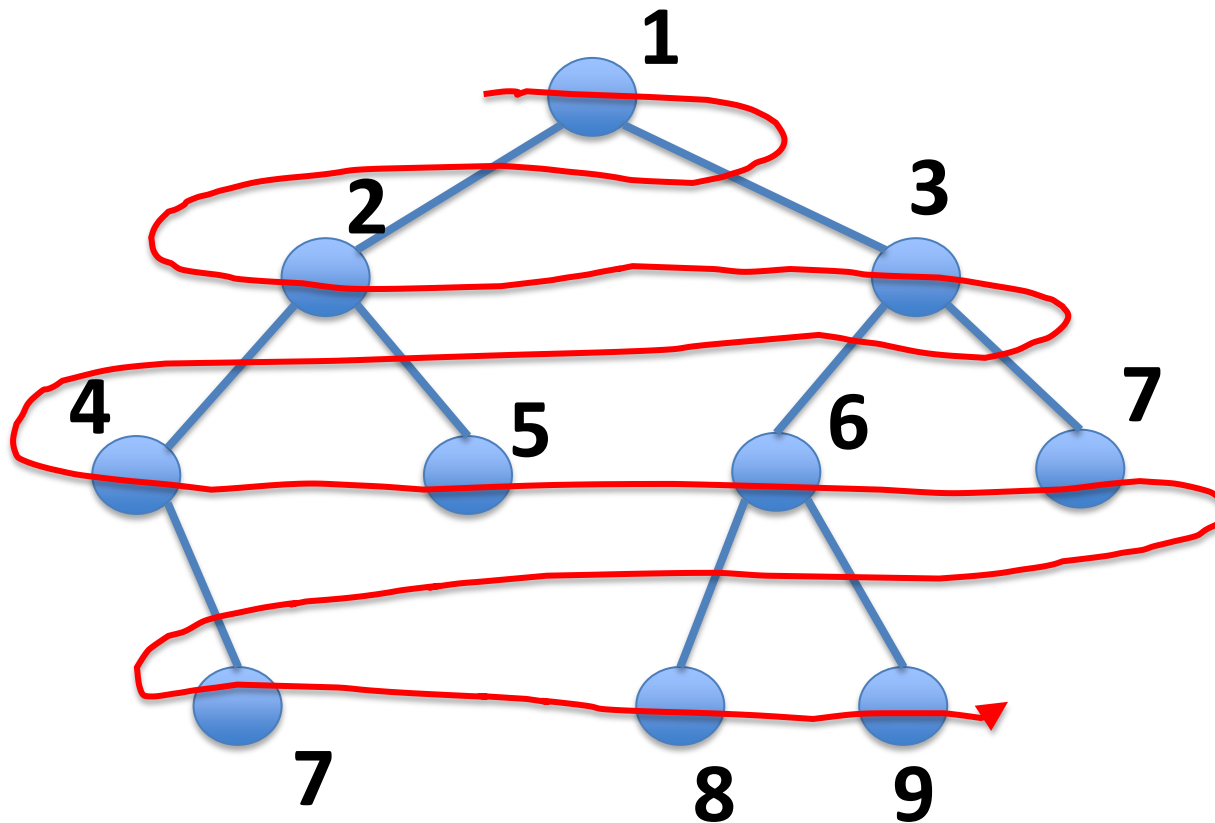
Binary Trees

A **Breadth-First traversal** is a different kind of traversal. Here, we visit all nodes level-by-level in a left-to-right manner.



Binary Trees

A **Breadth-First traversal** is a different kind of traversal. Here, we visit all nodes level-by-level in a left-to-right manner.



Binary Trees

Breadth-First traversal

```
void bfTraverse() {  
    Queue<Node> q = new LinkedList<Node>();  
    if (r != nil) q.add(r);  
    while (!q.isEmpty()) {  
        Node u = q.remove();  
        // System.out.println(u.data);  
        if (u.left != nil) q.add(u.left);  
        if (u.right != nil) q.add(u.right);  
    }  
}
```

Binary Search Trees

In a **binary search tree**, each node stores some data (in attribute x) from some **total order**

Binary Search Trees

In a **binary search tree**, each node stores some data (in attribute x) from some **total order**

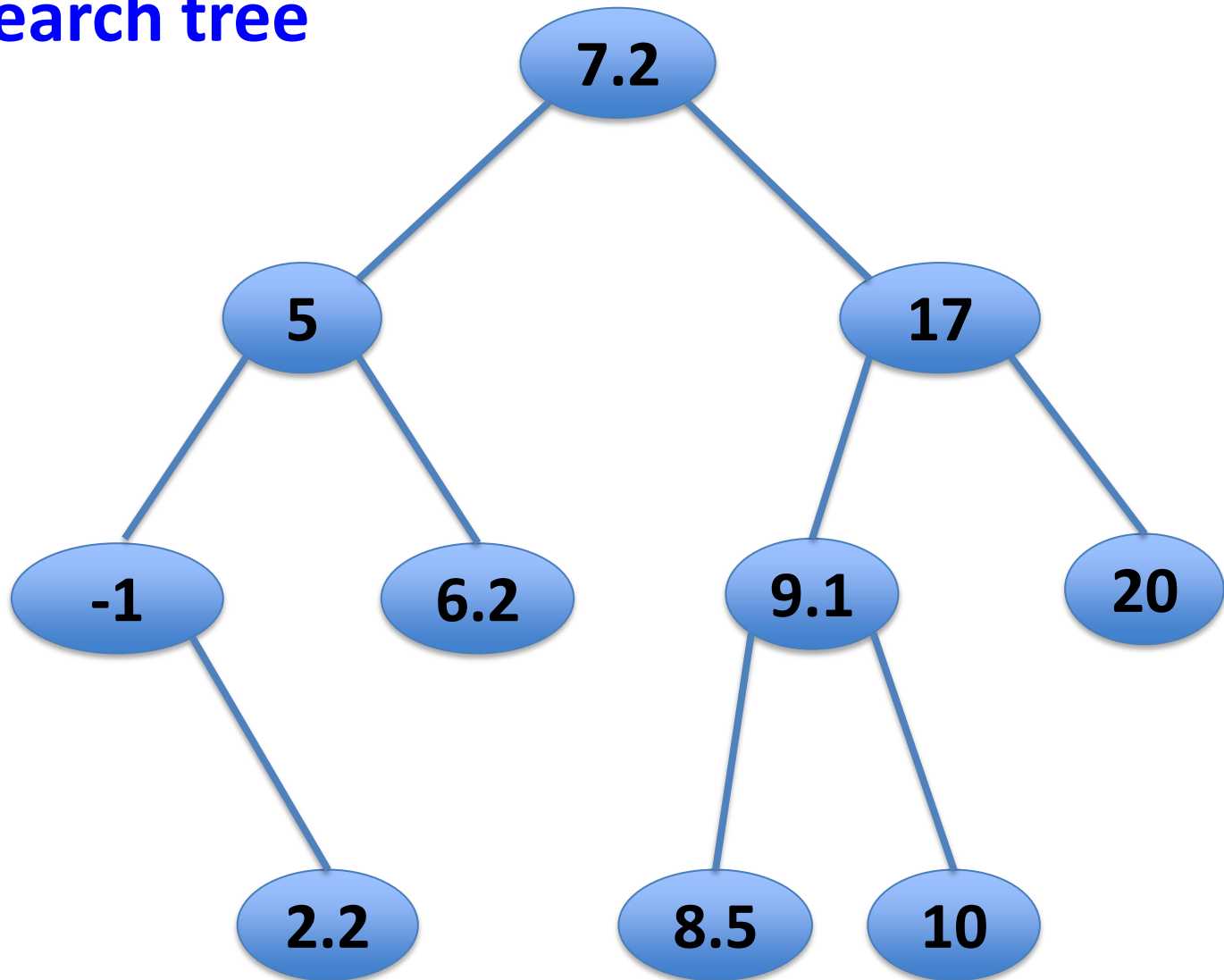
The **binary search tree property**: (with distinct items)

For any node u

1. All data values stored in the subtree $u.left$ are less than $u.x$
2. All data values stored in the subtree $u.right$ are greater than $u.x$

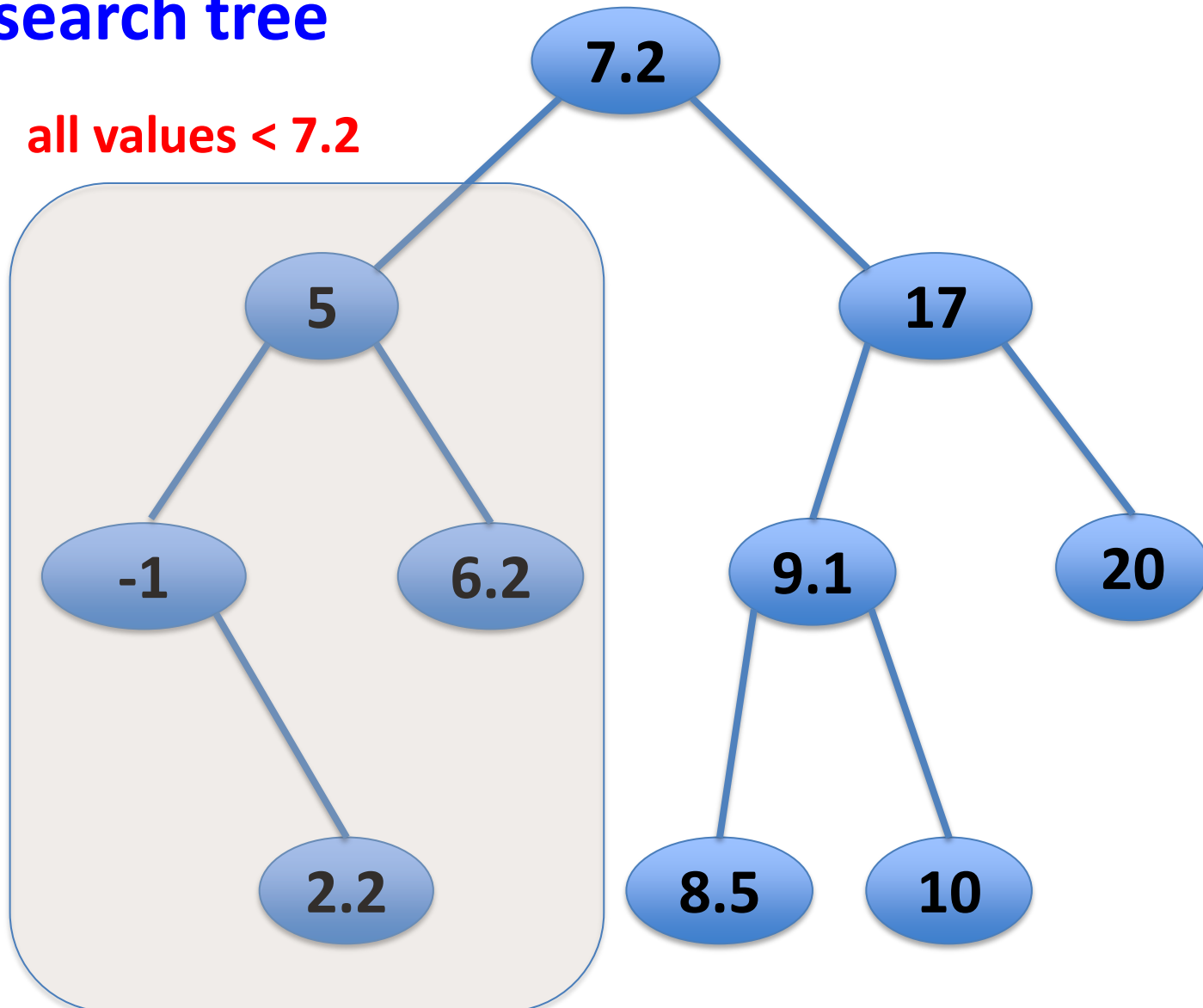
Binary Search Trees

binary search tree



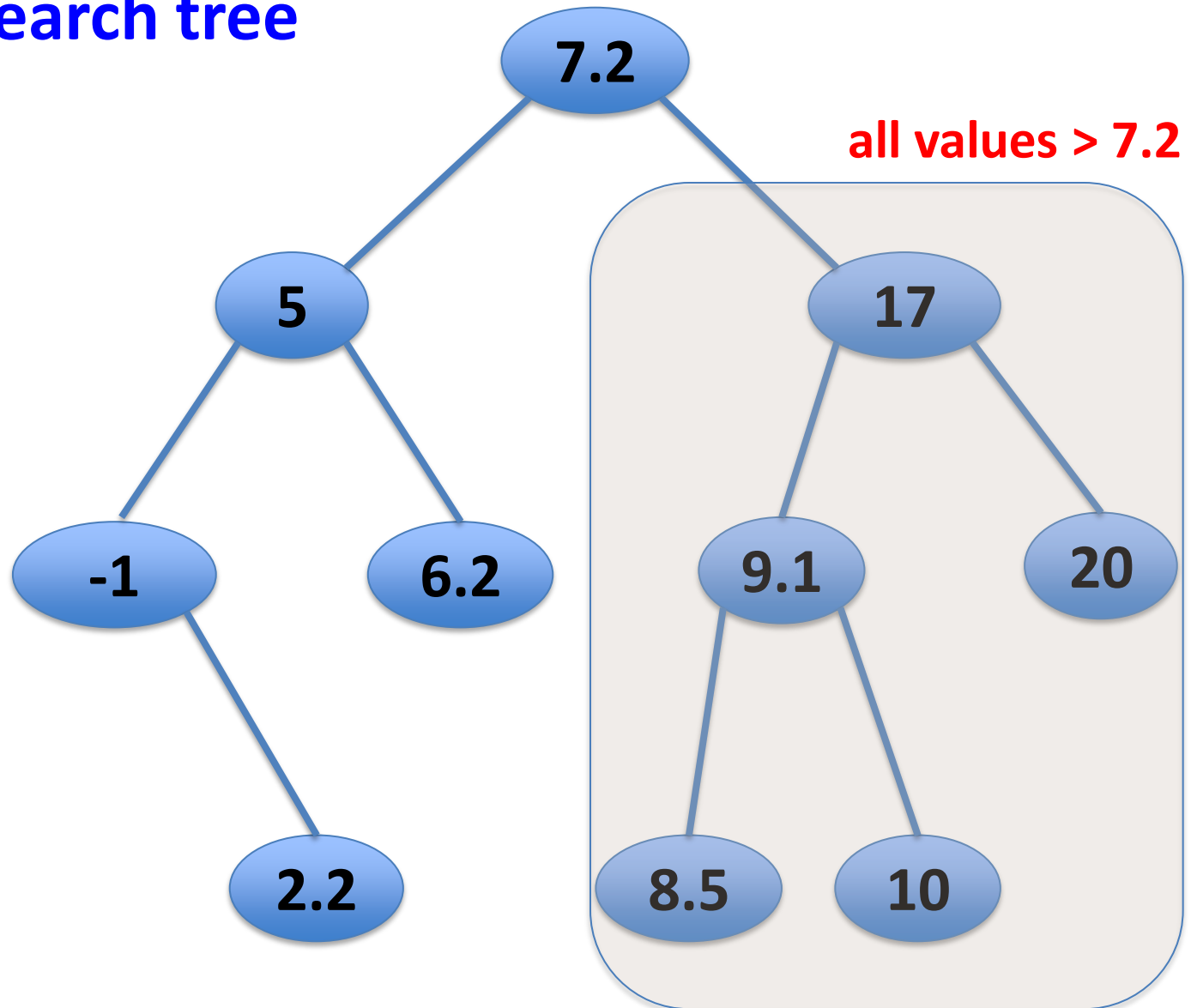
Binary Search Trees

binary search tree



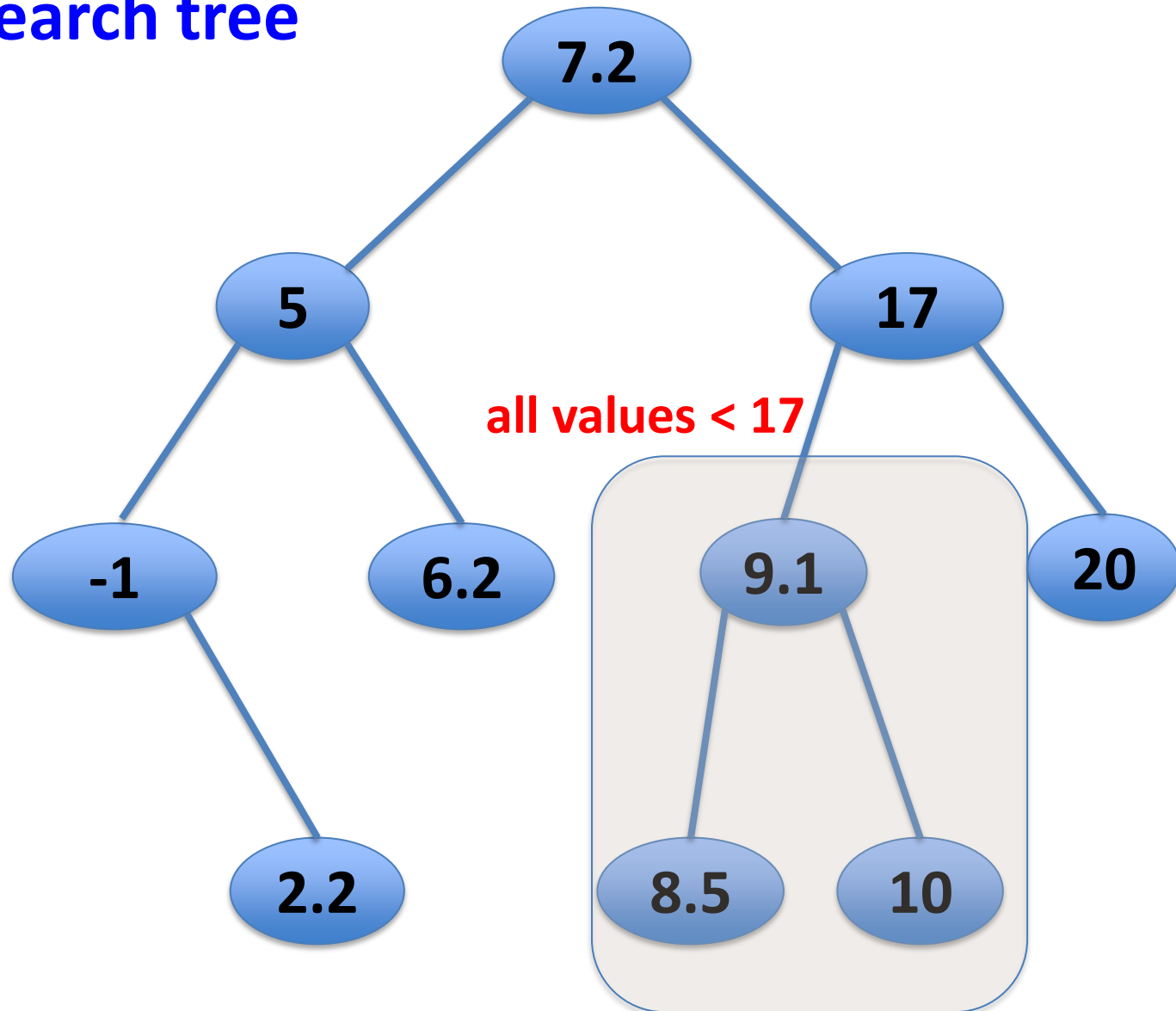
Binary Search Trees

binary search tree



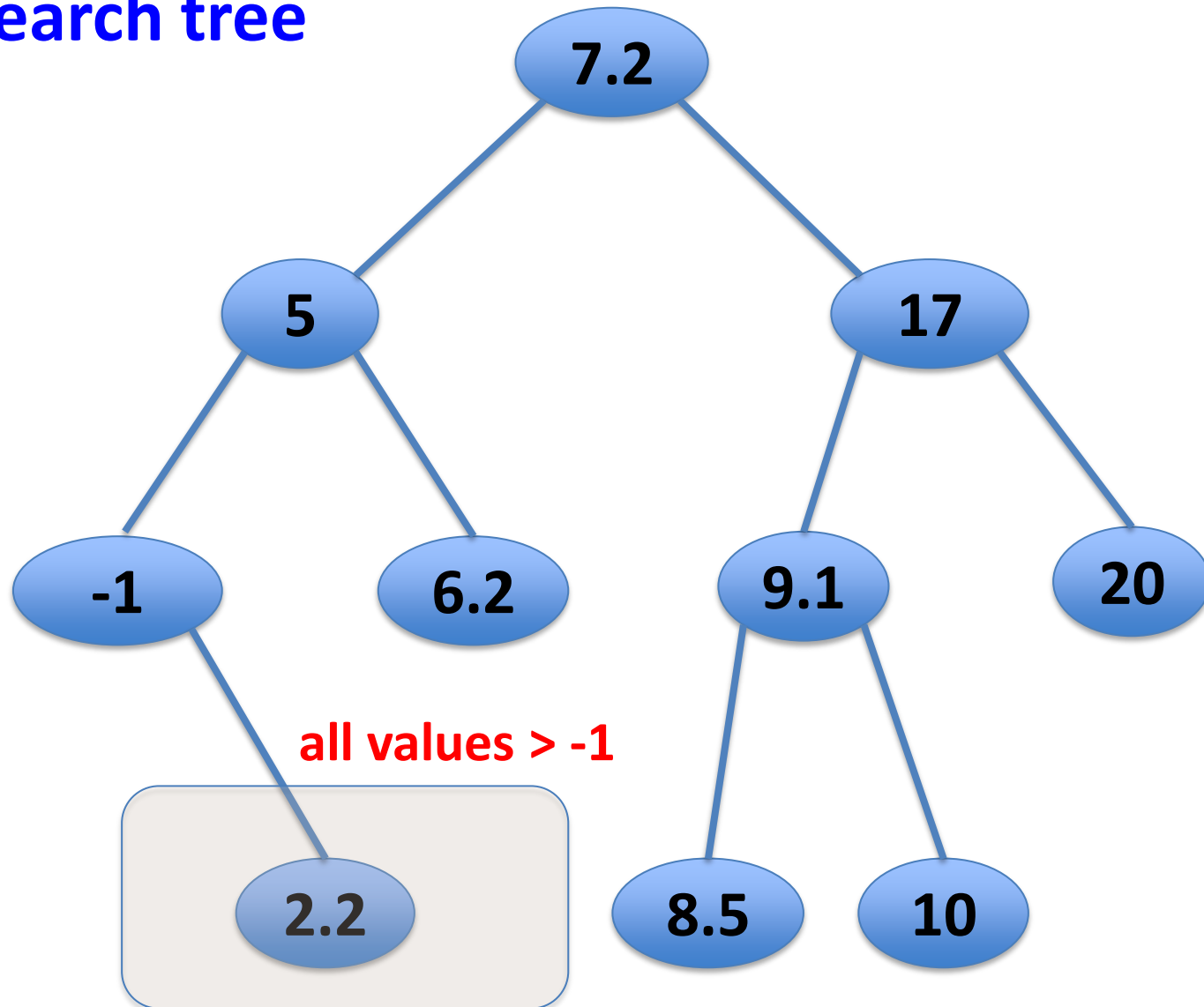
Binary Search Trees

binary search tree



Binary Search Trees

binary search tree



Binary Search Trees

Let's consider the following operations:

- Find a value in the BST
 - Exact value
 - Successor search (find smallest $y \geq x$)
- Add a value to the BST
- Remove a value from the BST

Binary Search Trees

Searching in a binary search tree.

When searching for a value x , start with the root node.

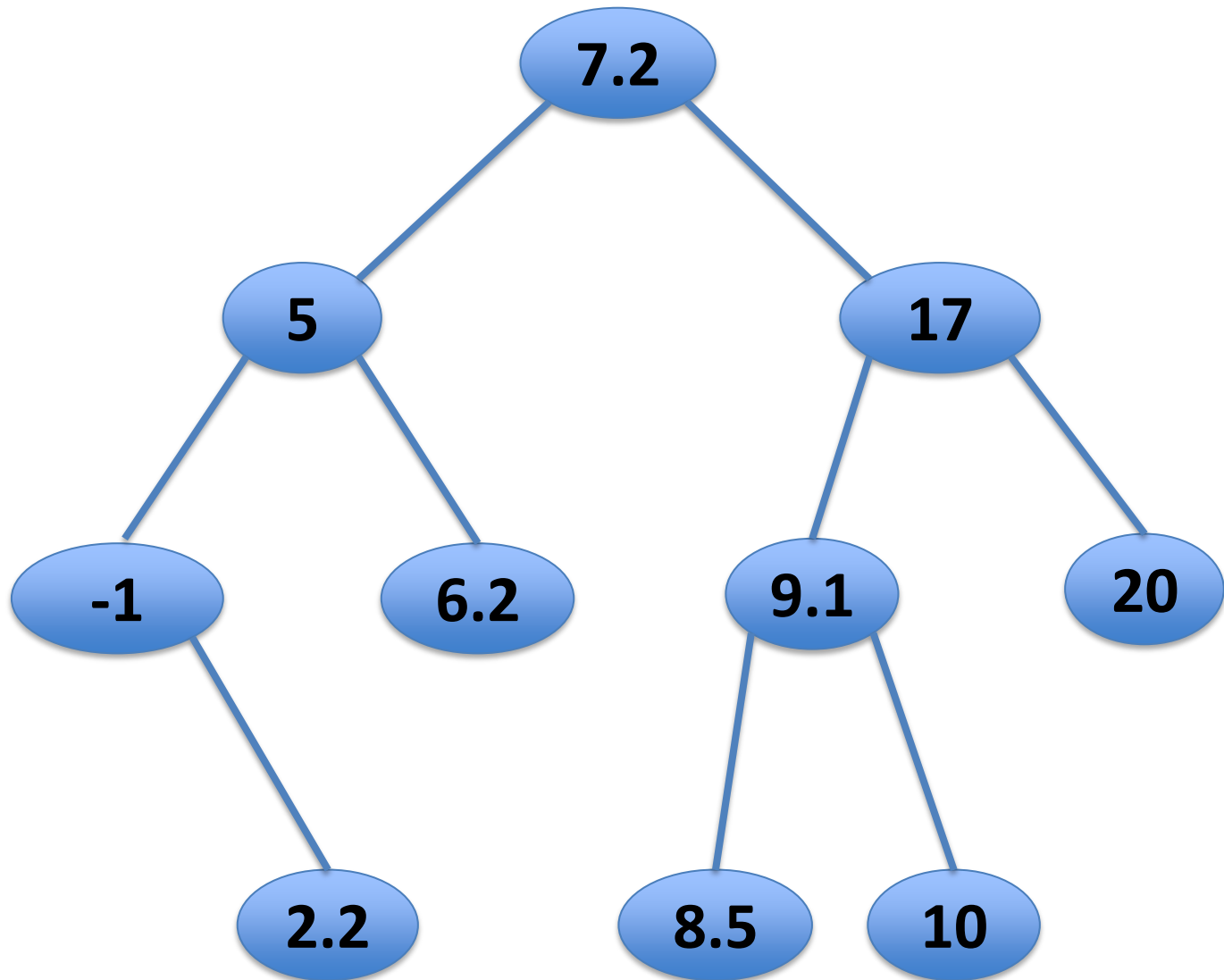
When considering a node u , there are three cases

1. if $x < u.x$, then continue with $u.left$
2. if $x > u.x$, then continue with $u.right$
3. if $x = u.x$, then we have found the node with x

If $u = nil$, then x is not in the BST.

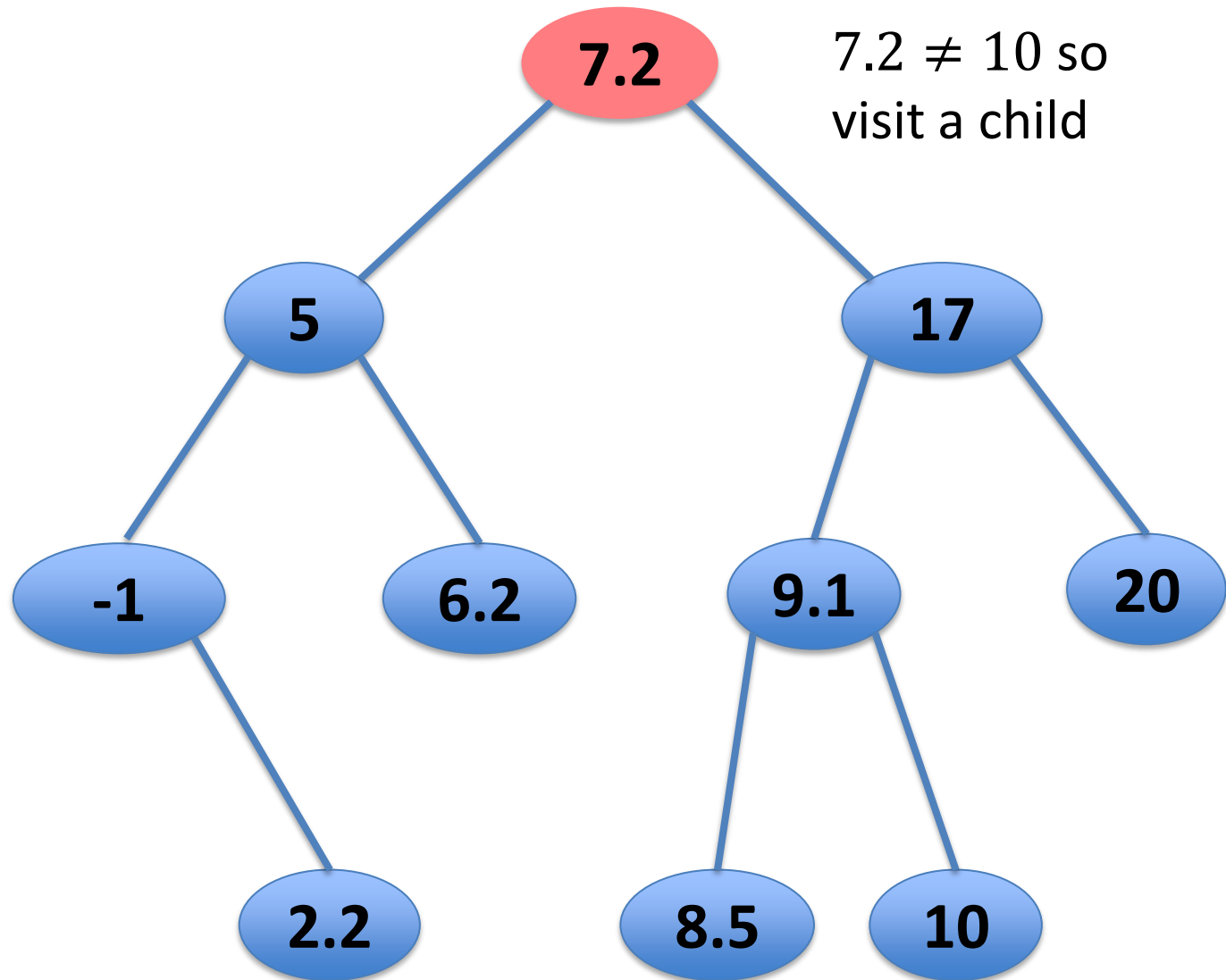
Binary Search Trees

find (10)



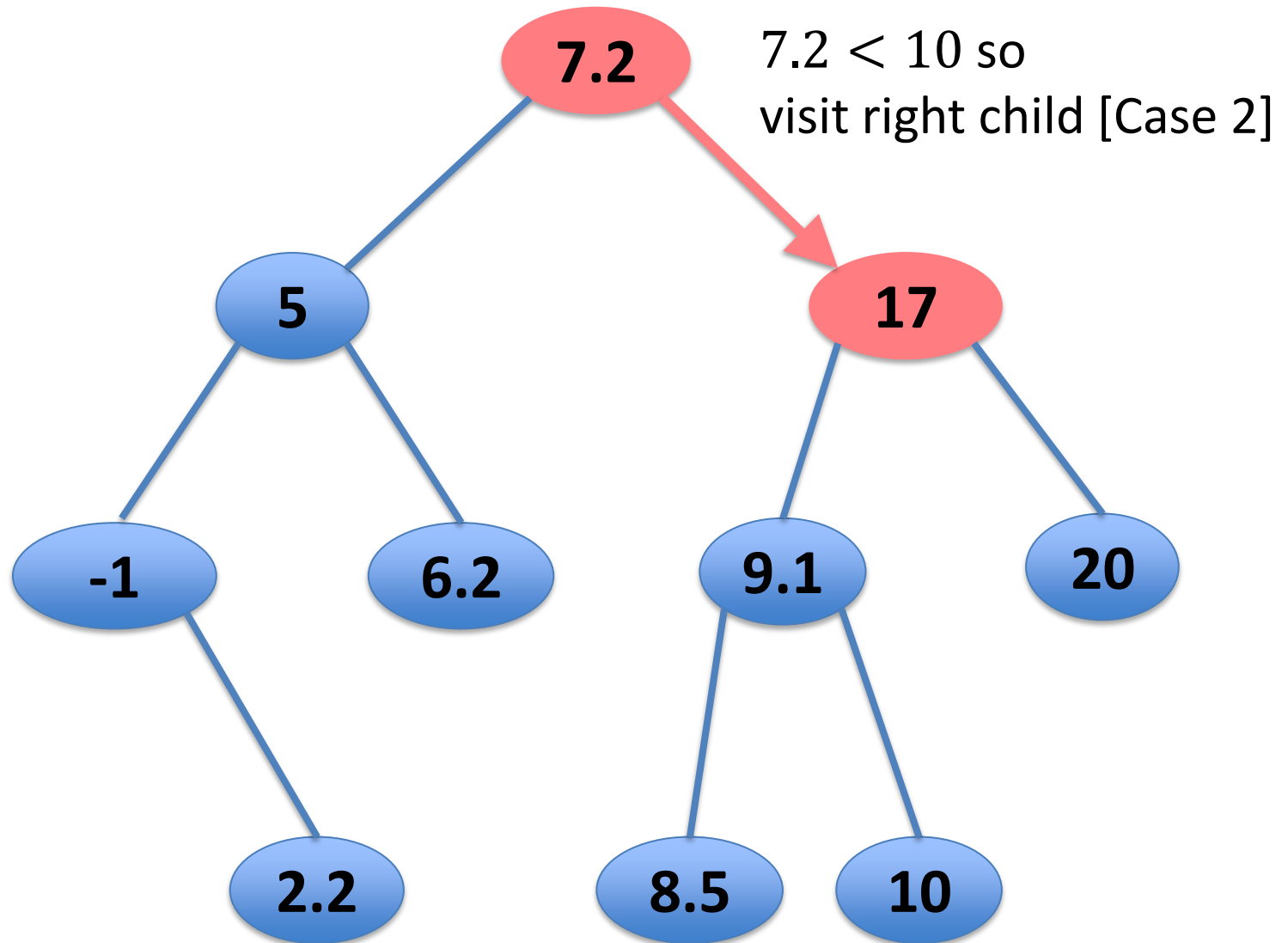
Binary Search Trees

find (10)



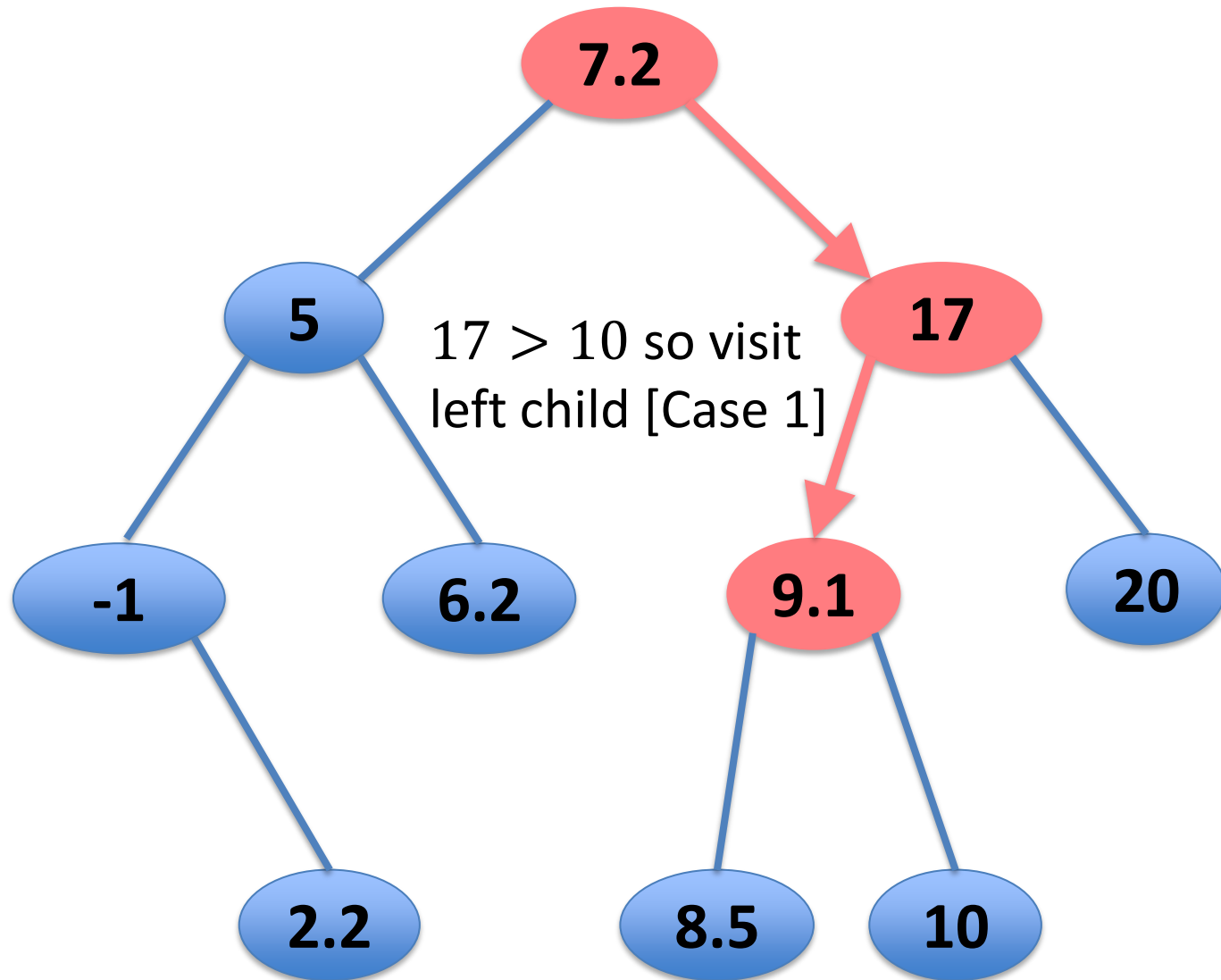
Binary Search Trees

find (10)



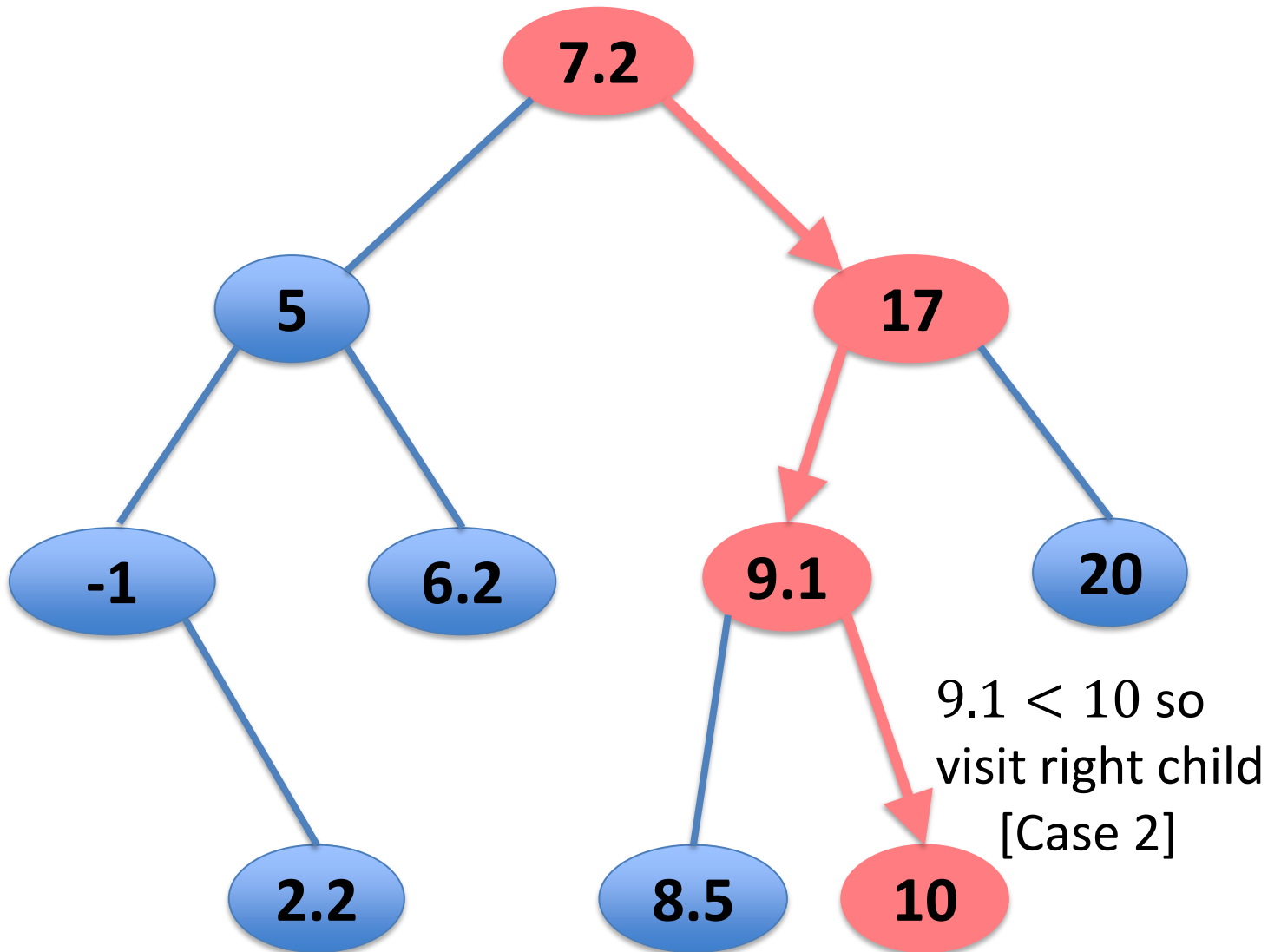
Binary Search Trees

find (10)



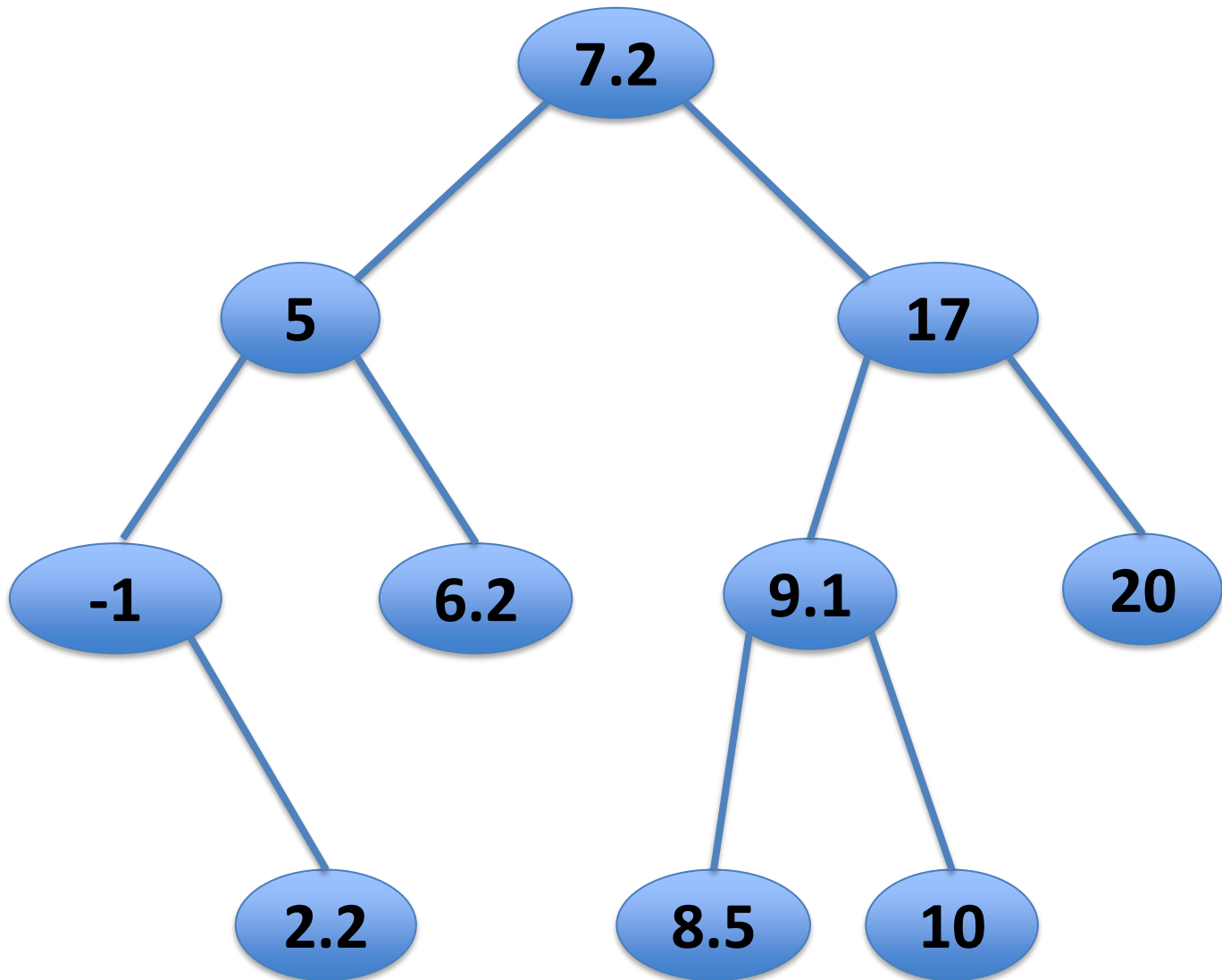
Binary Search Trees

find (10)



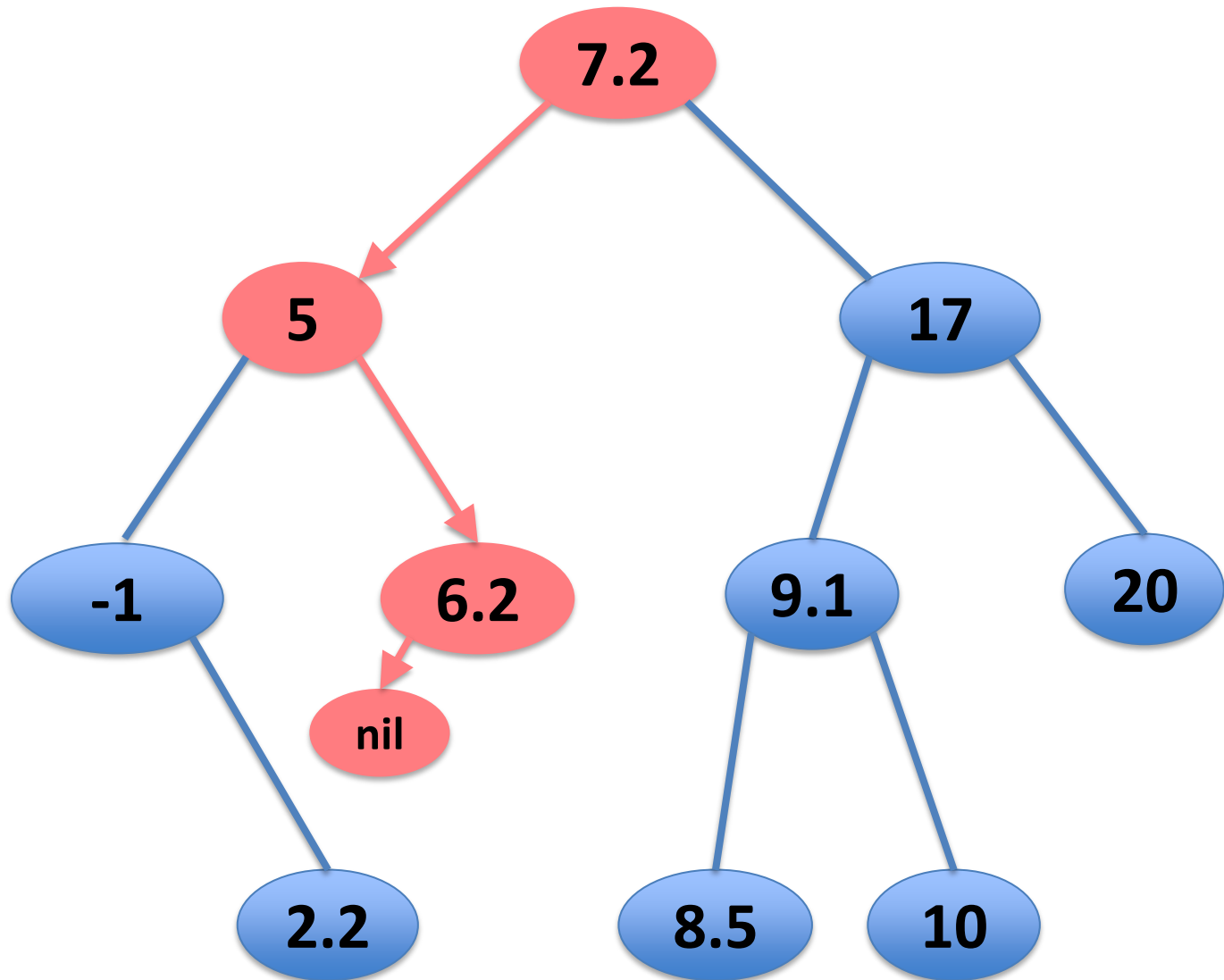
Binary Search Trees

find (6)



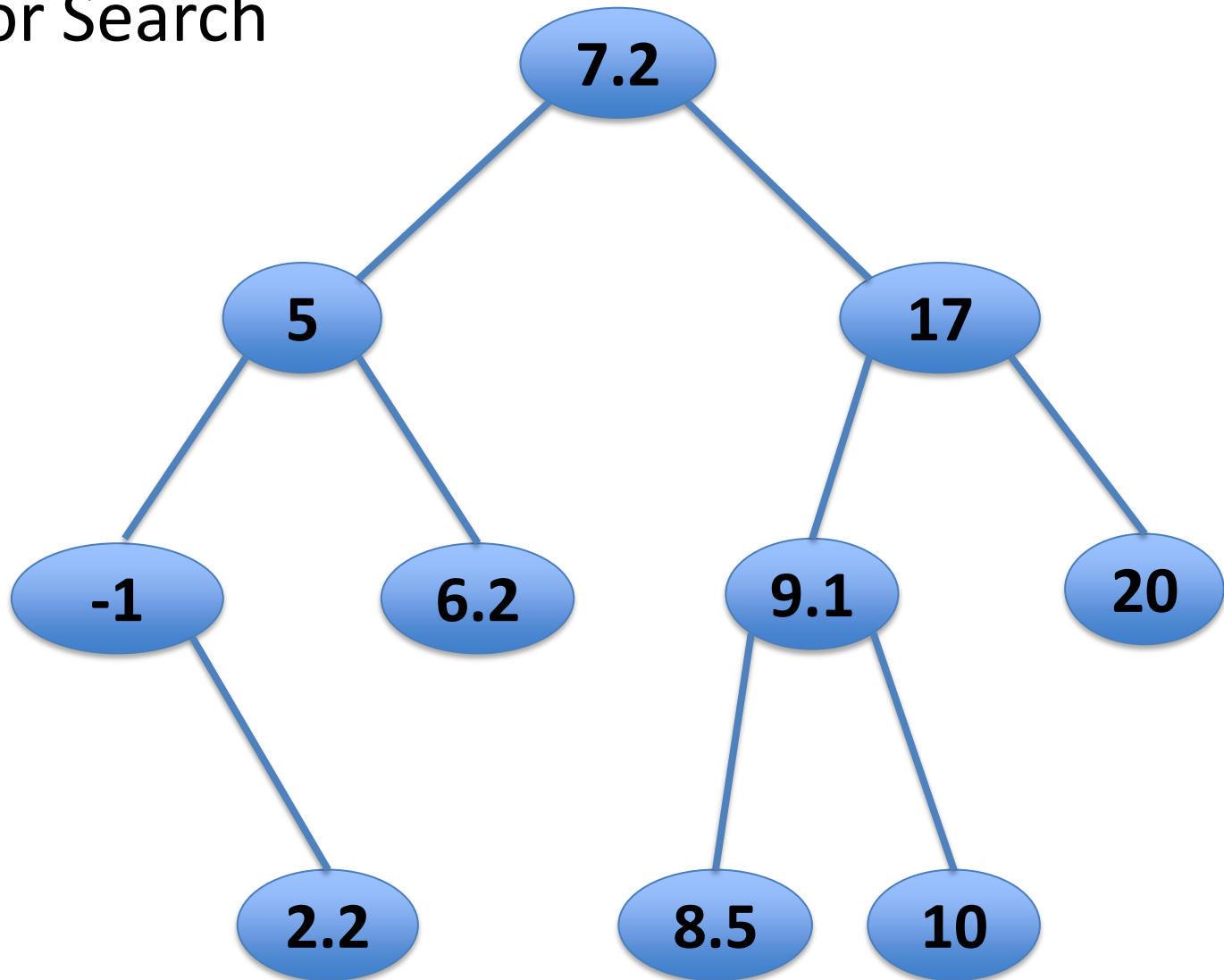
Binary Search Trees

find (6)



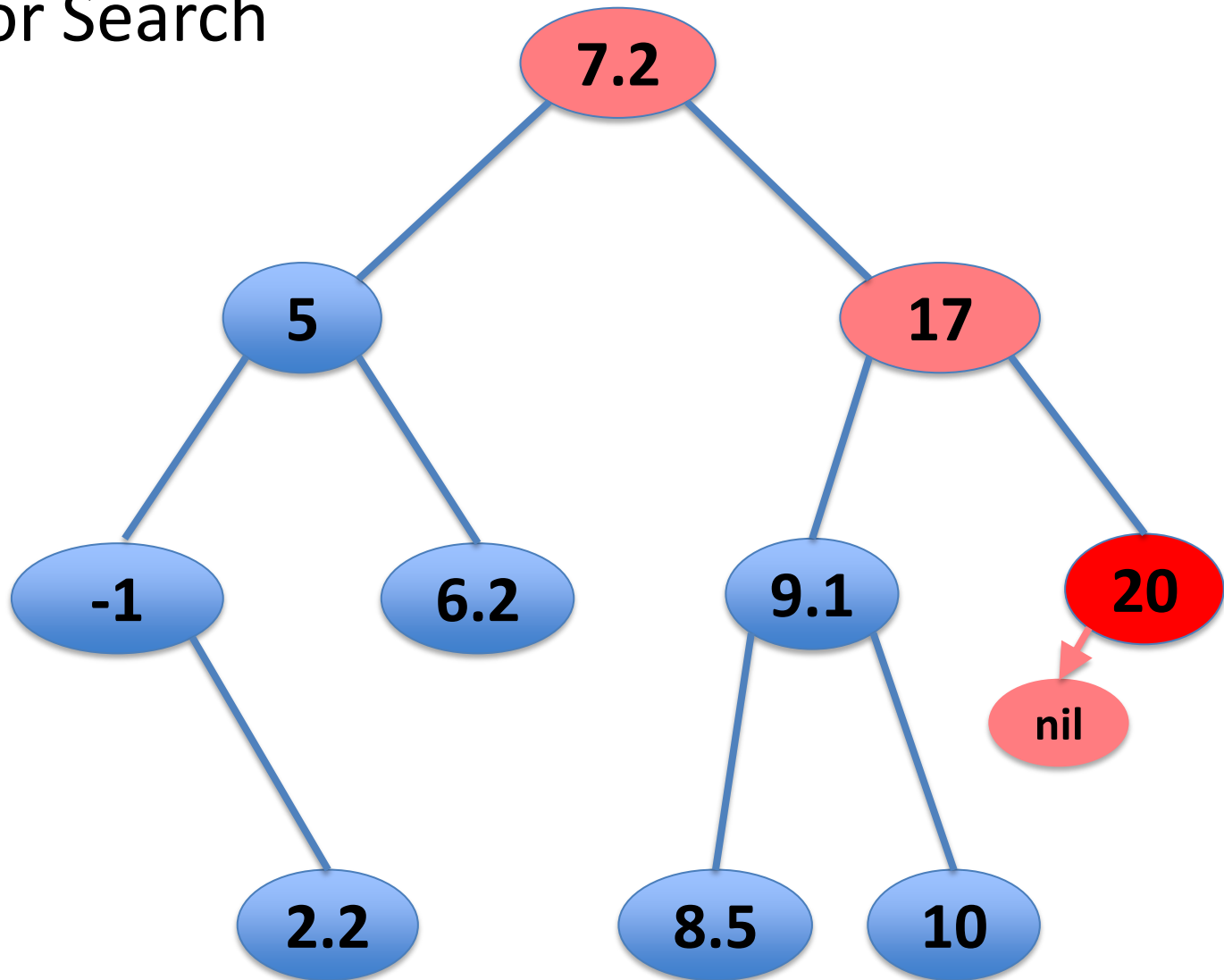
Binary Search Trees

Successor Search
find(18)



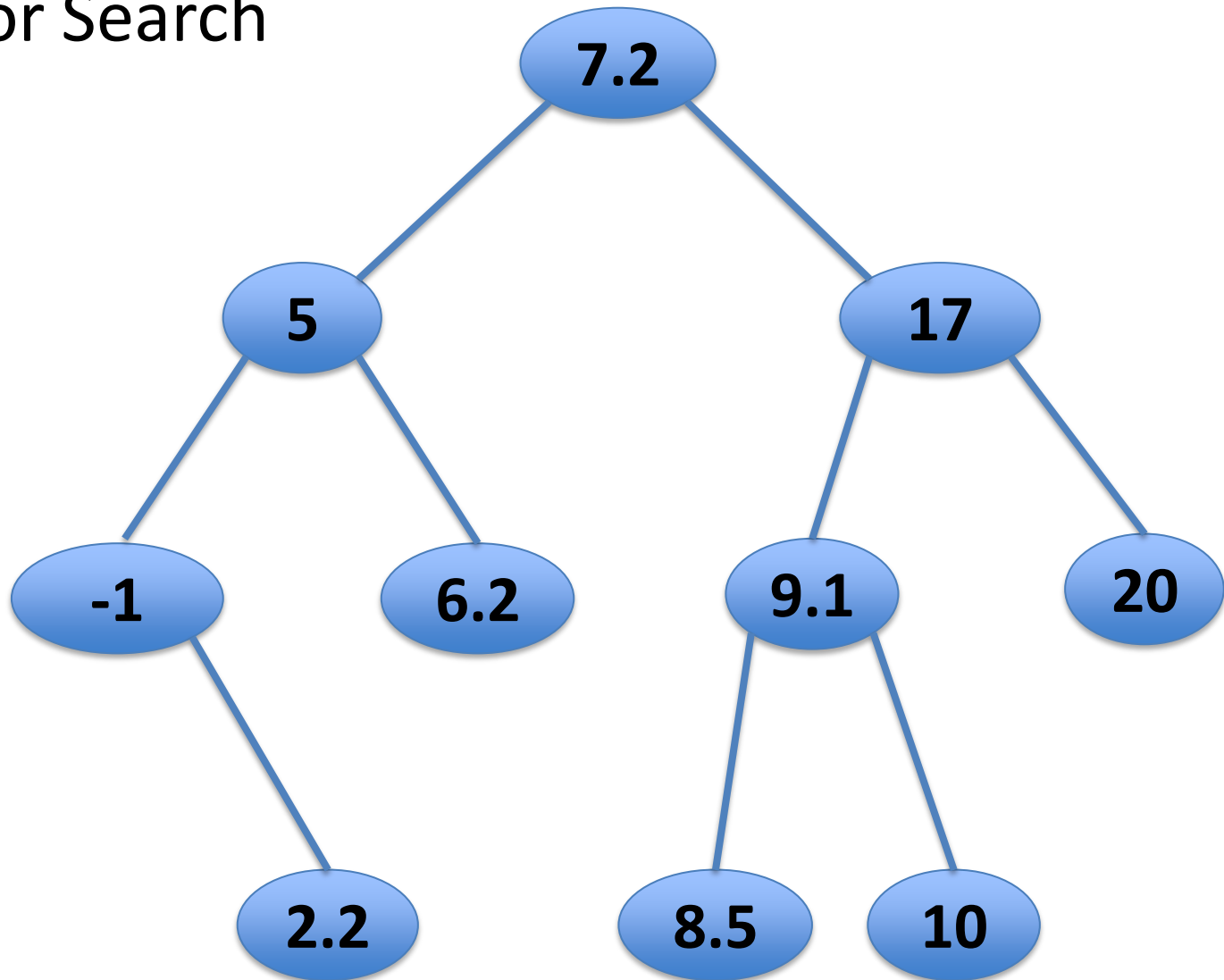
Binary Search Trees

Successor Search
find(18)



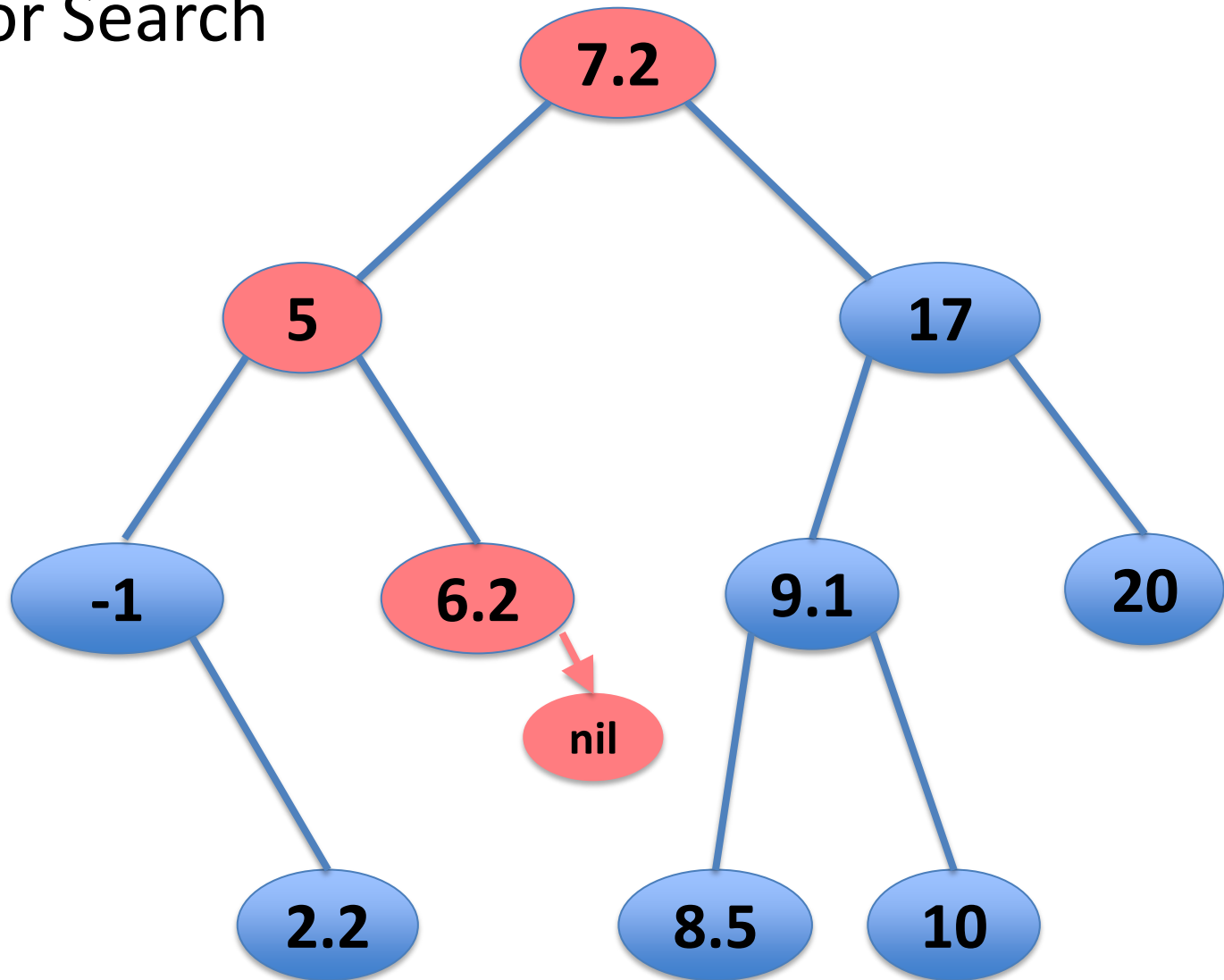
Binary Search Trees

Successor Search
find(7)



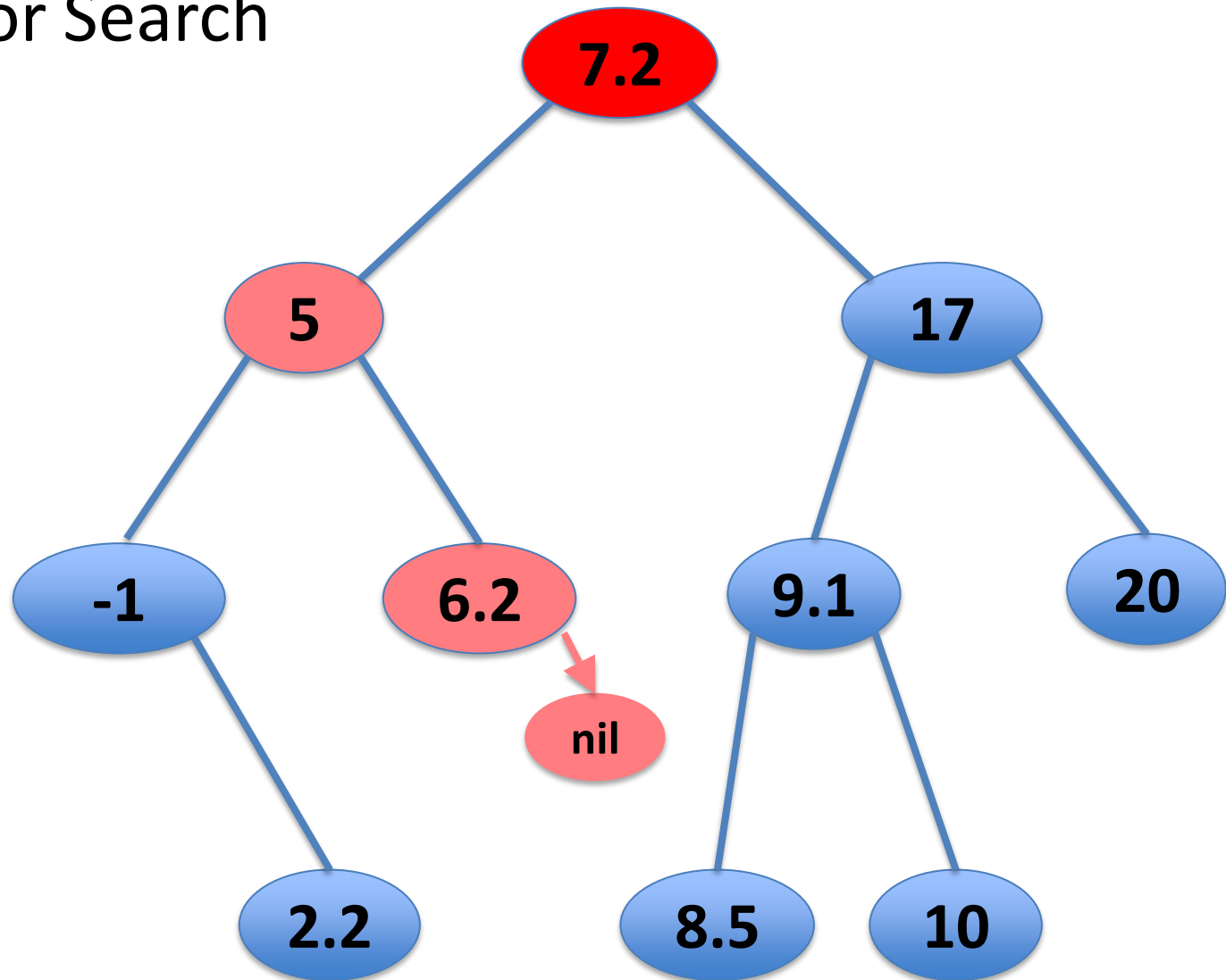
Binary Search Trees

Successor Search
find(7)



Binary Search Trees

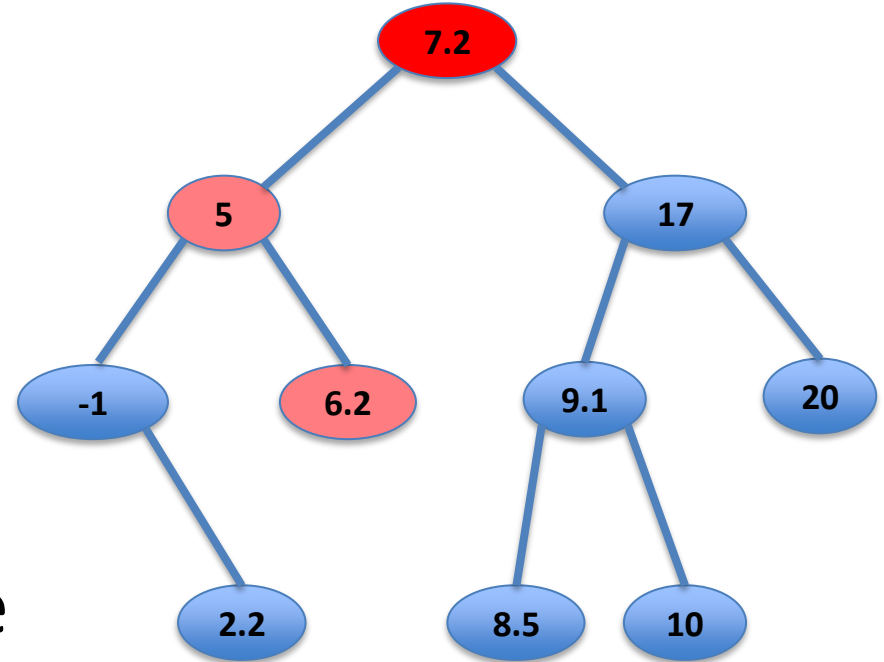
Successor Search
find(7)



Binary Search Trees

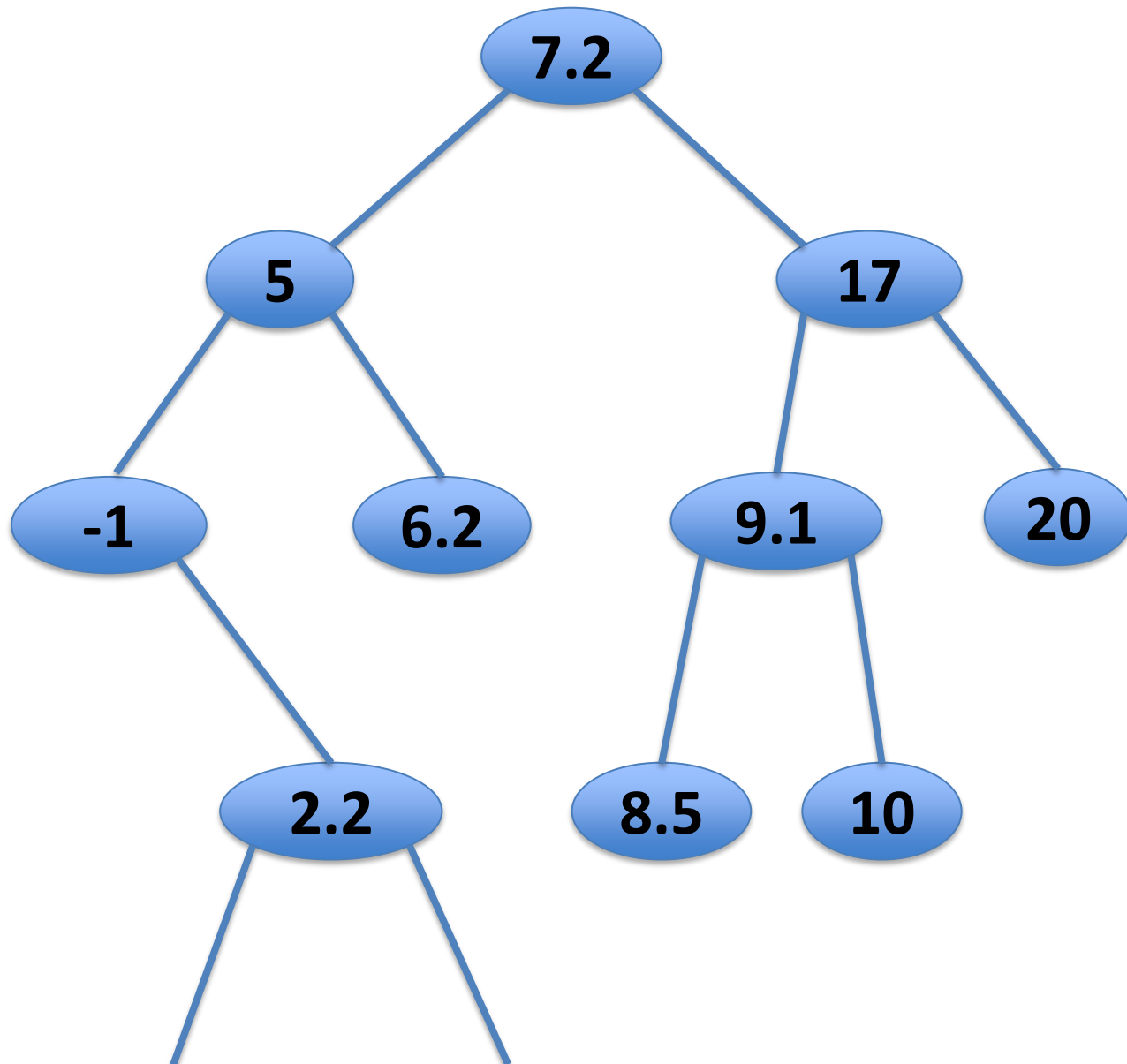
Successor Search

- if we find x return it
- otherwise, return the last node that triggered a "left" move on the path to trying to find x [Case 1]



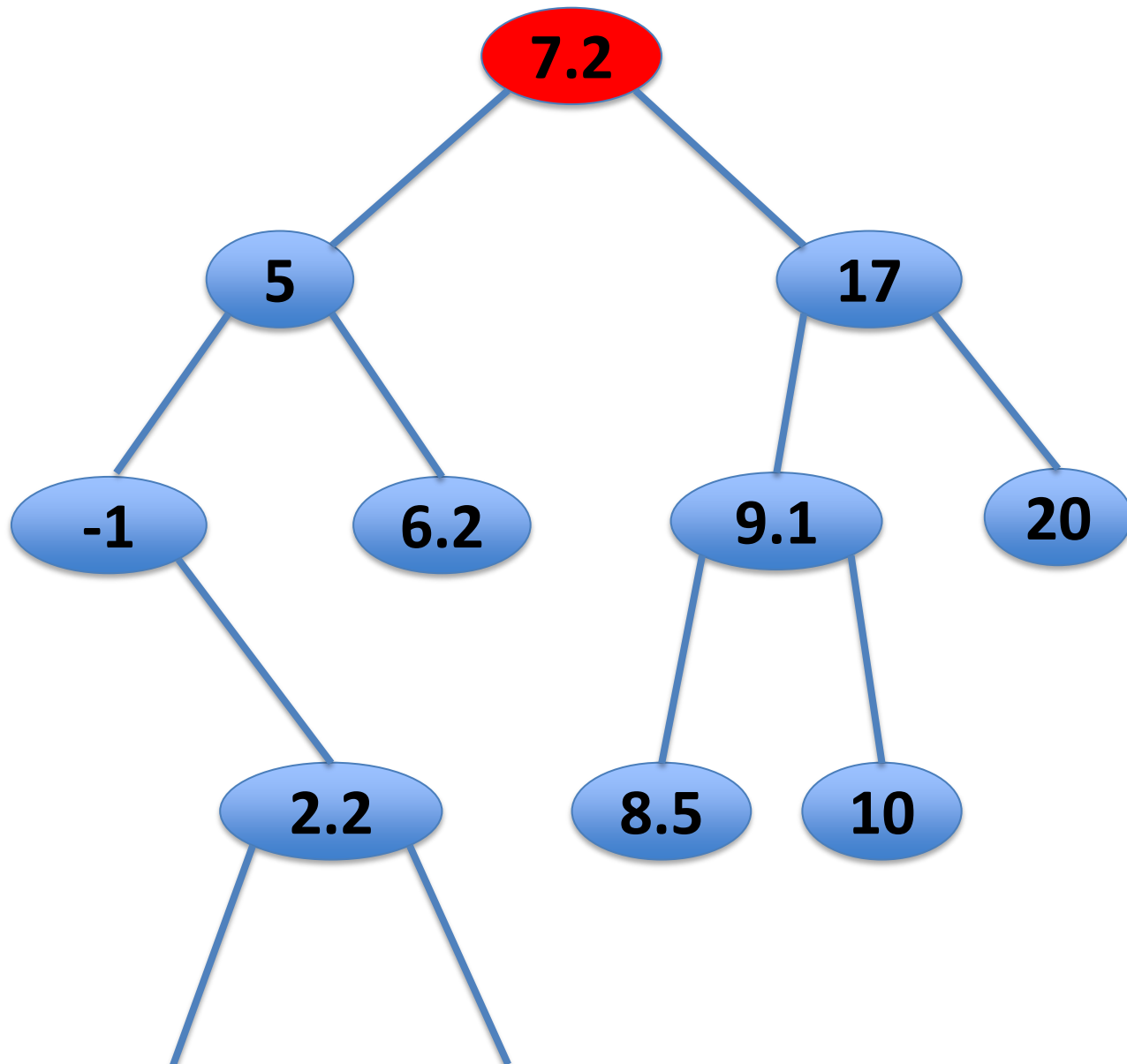
Binary Search Trees

Add(9)



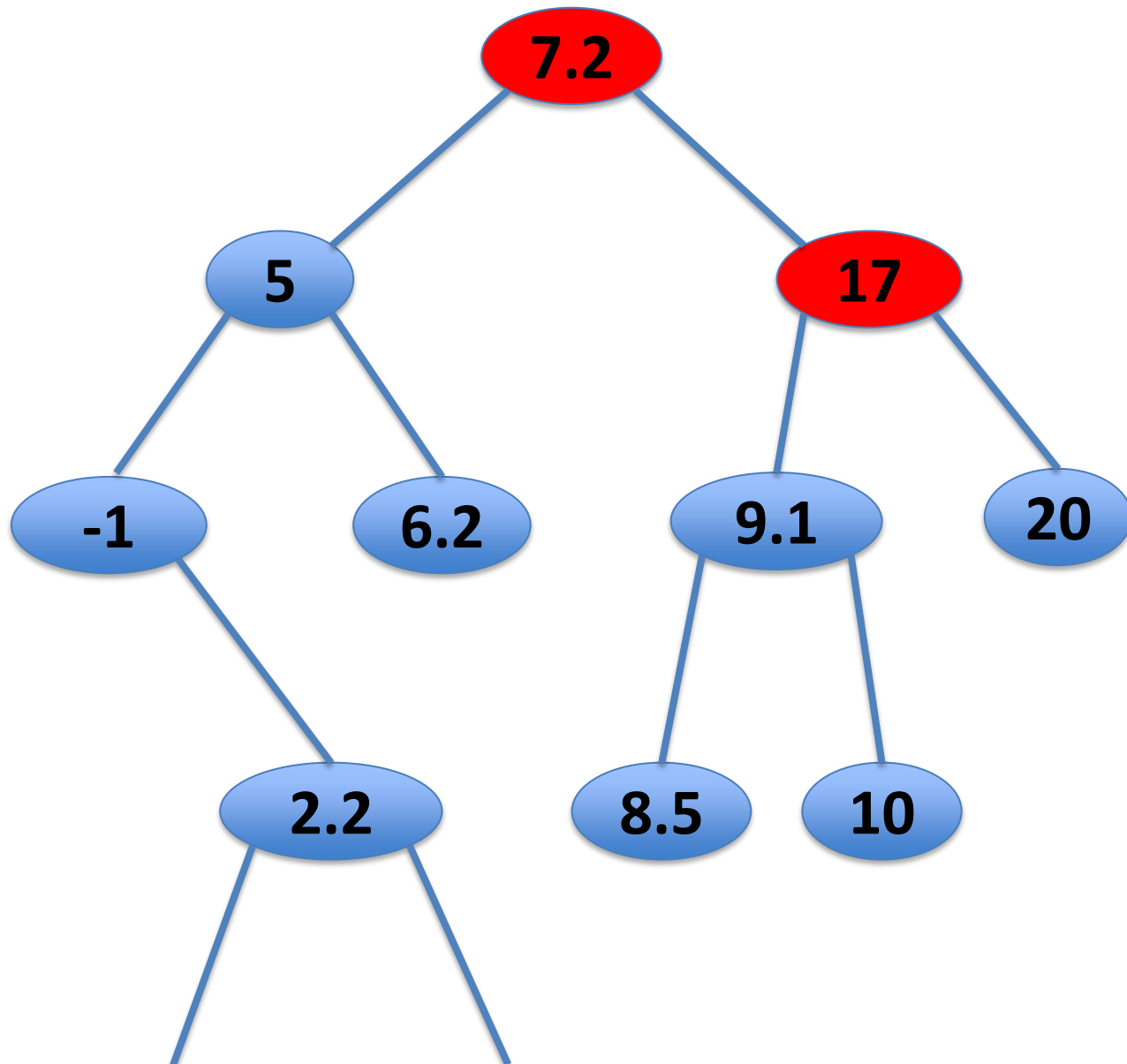
Binary Search Trees

Add(9)



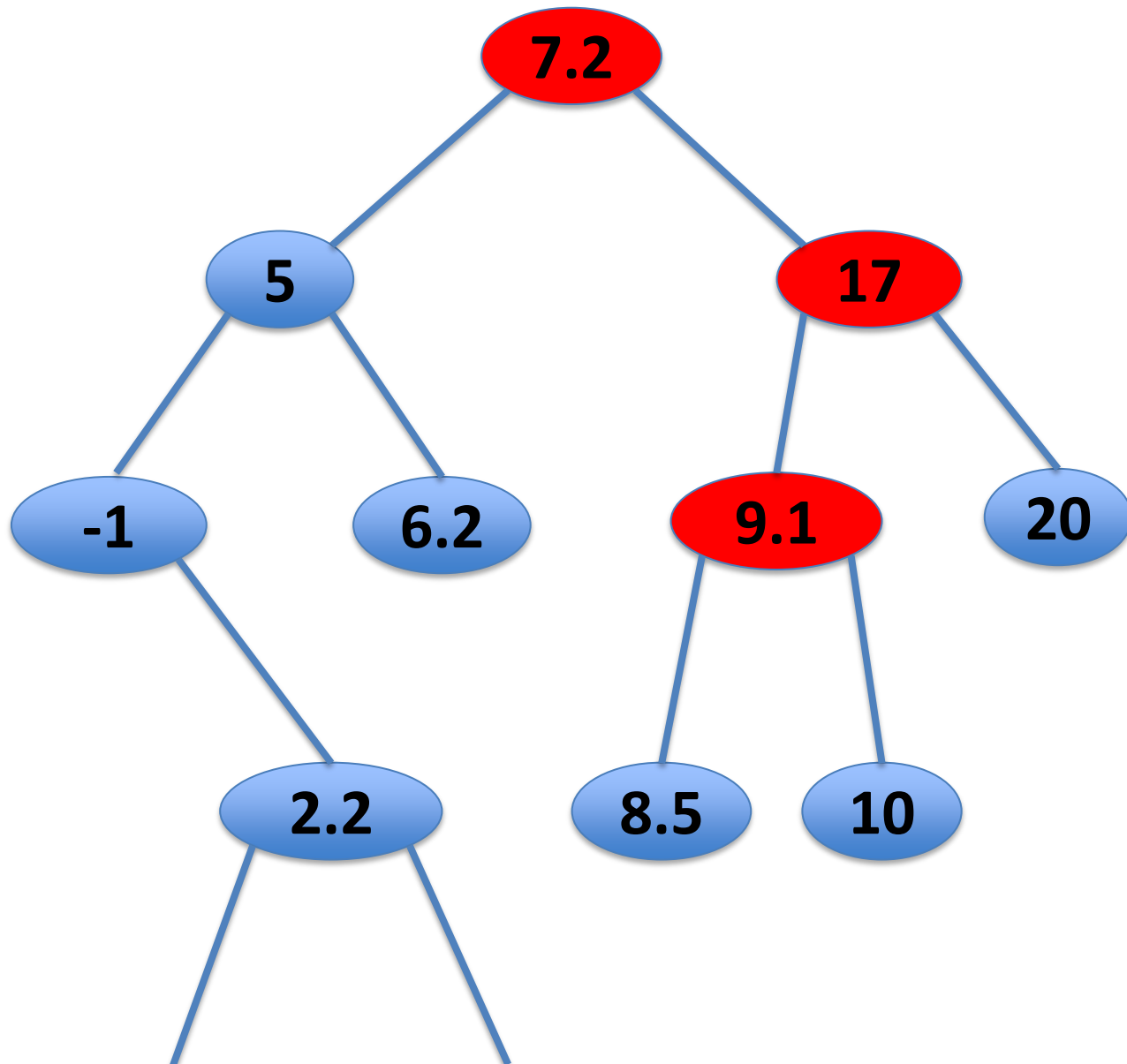
Binary Search Trees

Add(9)



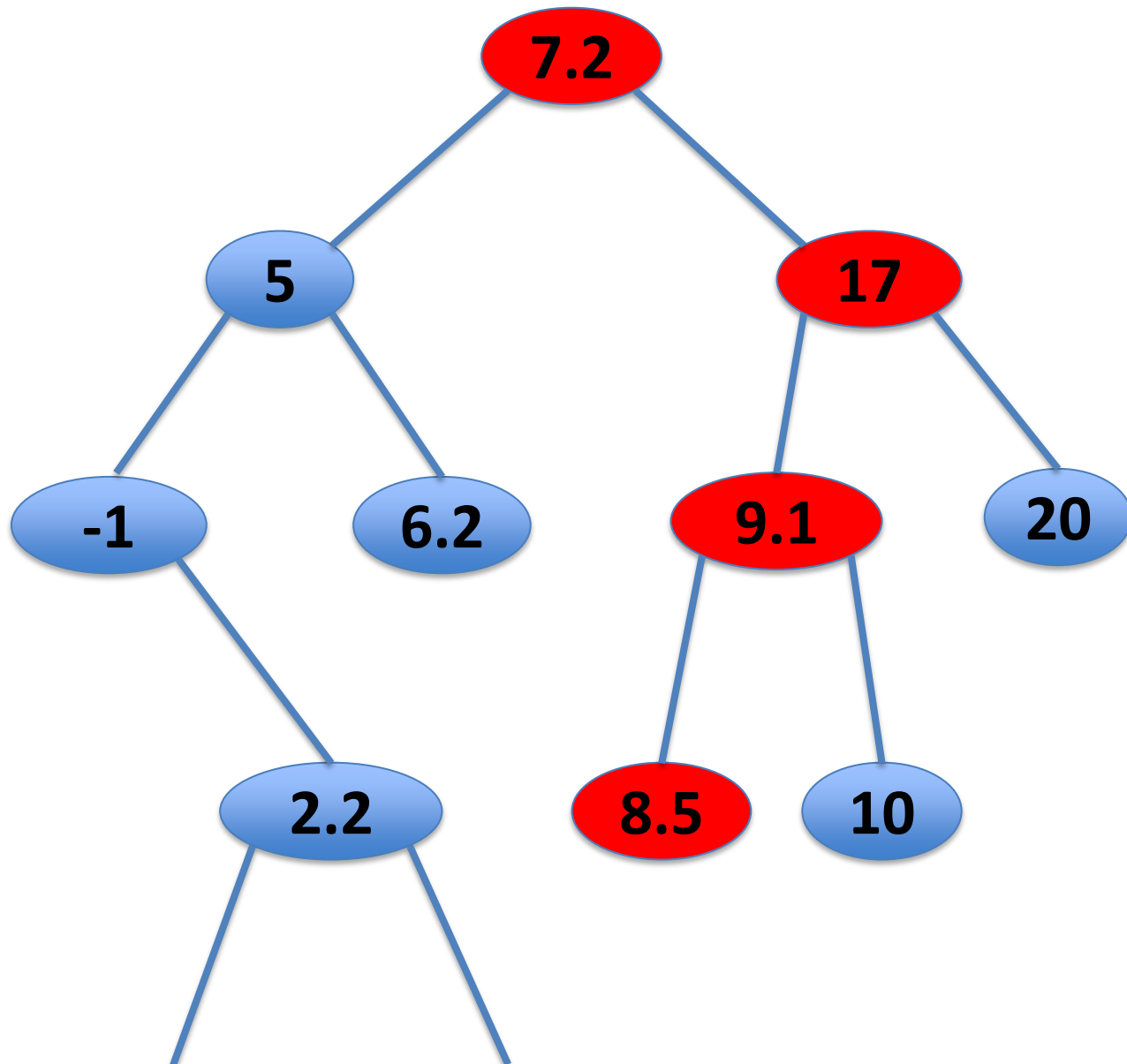
Binary Search Trees

Add(9)



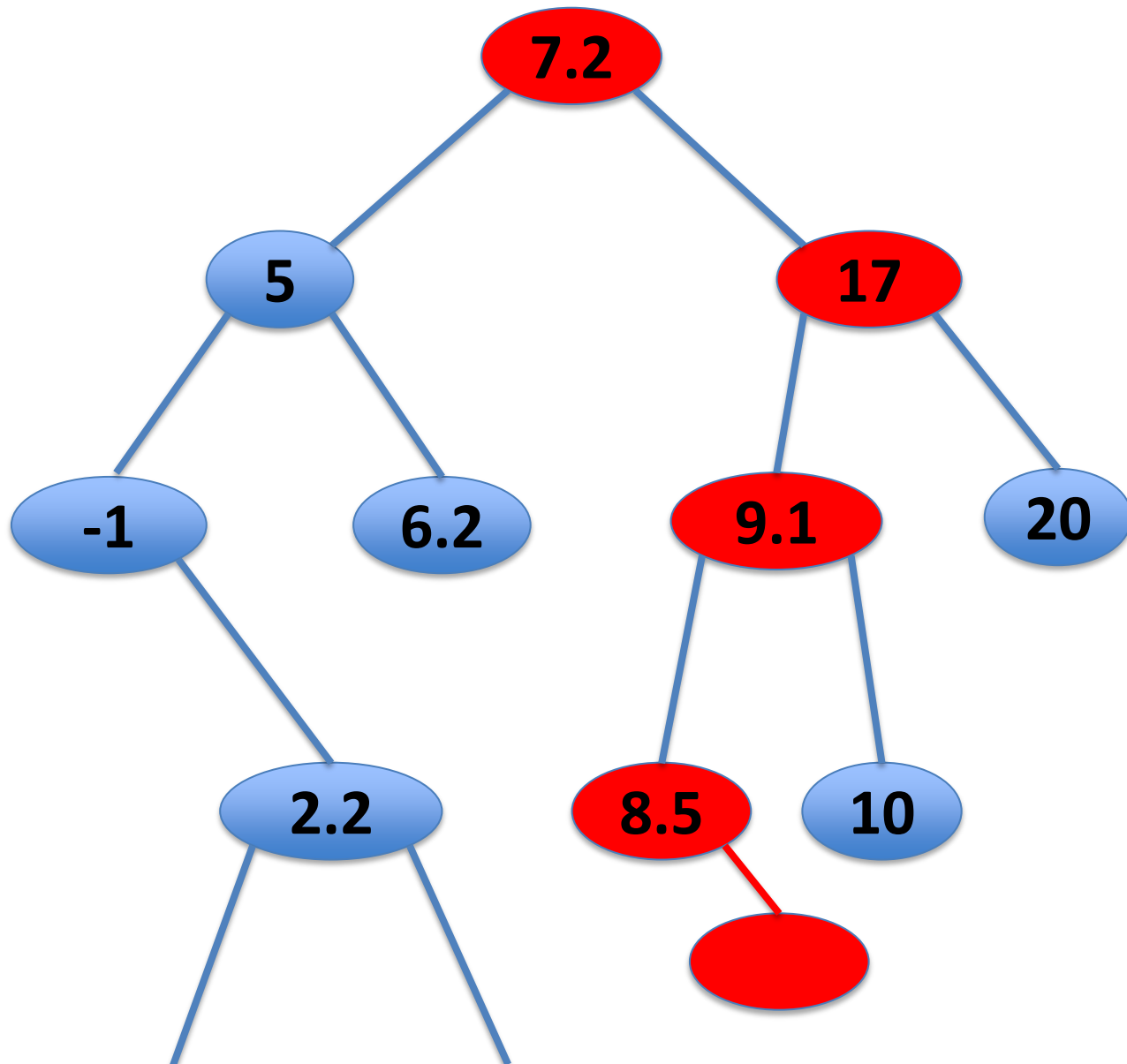
Binary Search Trees

Add(9)



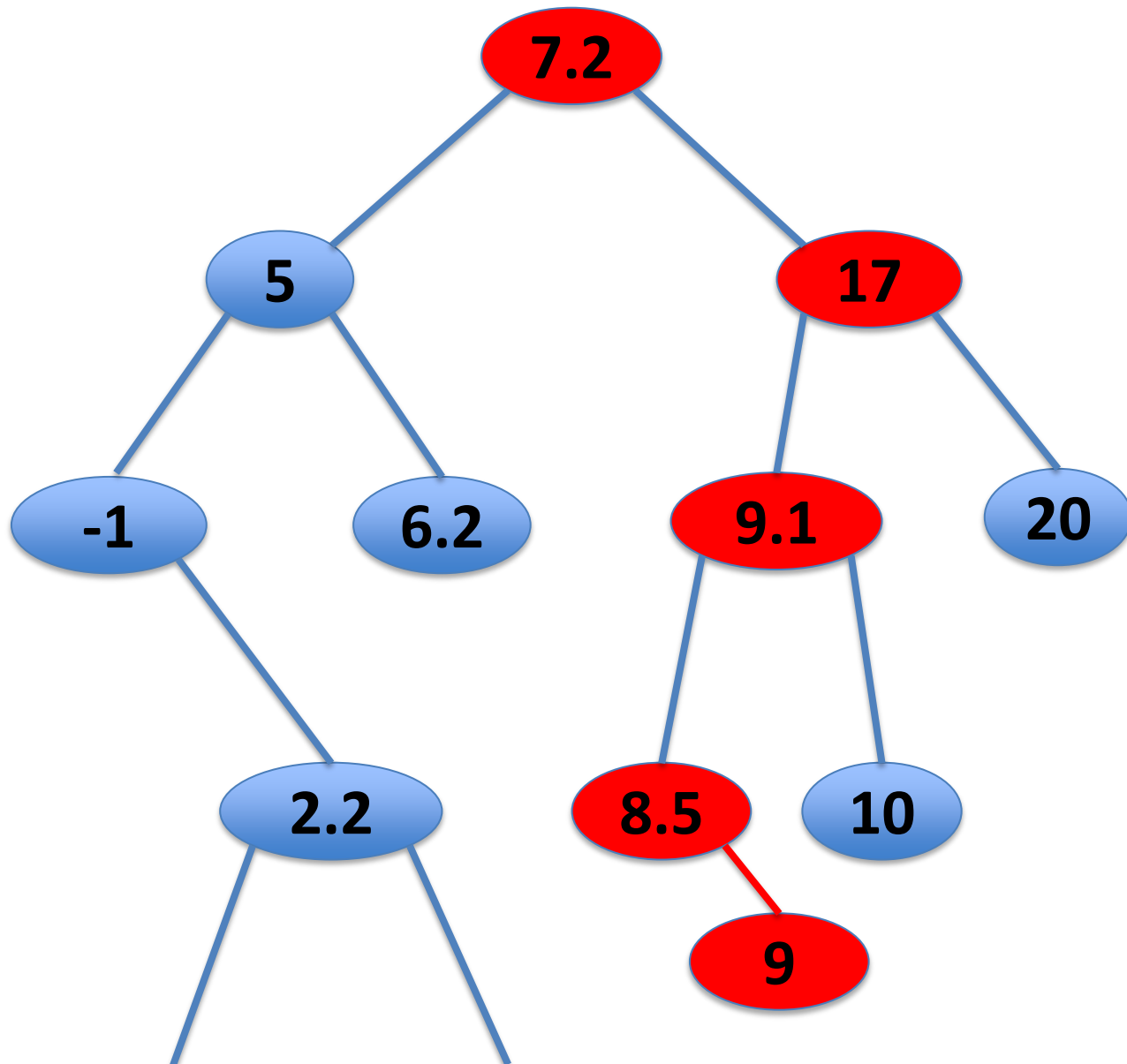
Binary Search Trees

Add(9)



Binary Search Trees

Add(9)



Binary Search Trees

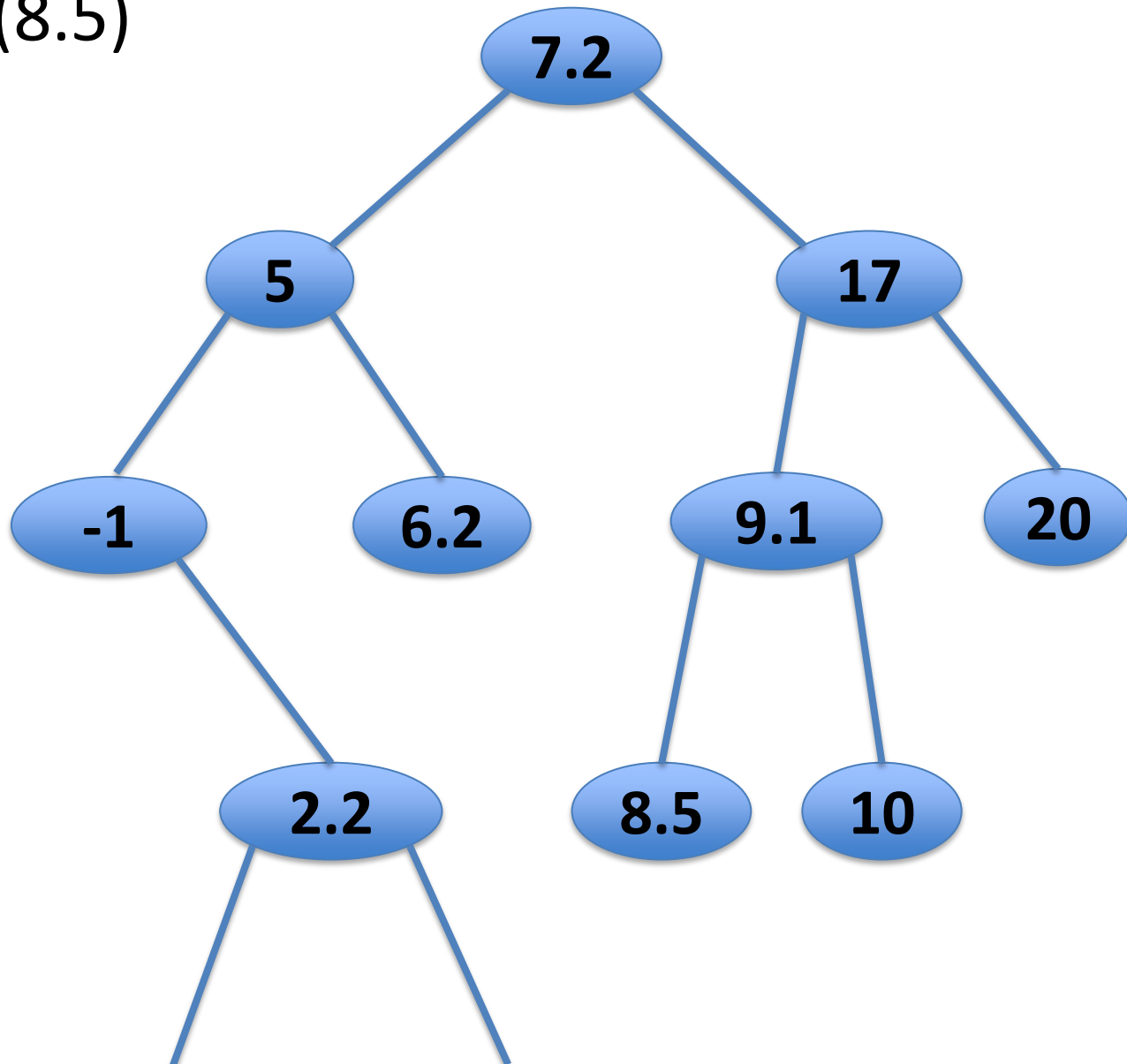
Adding a value `add(x)`

Find where the value `x` should be in the tree.

- If it is already there do nothing.
- Otherwise, add a new leaf node with the value at the place where we ran off the tree looking for it

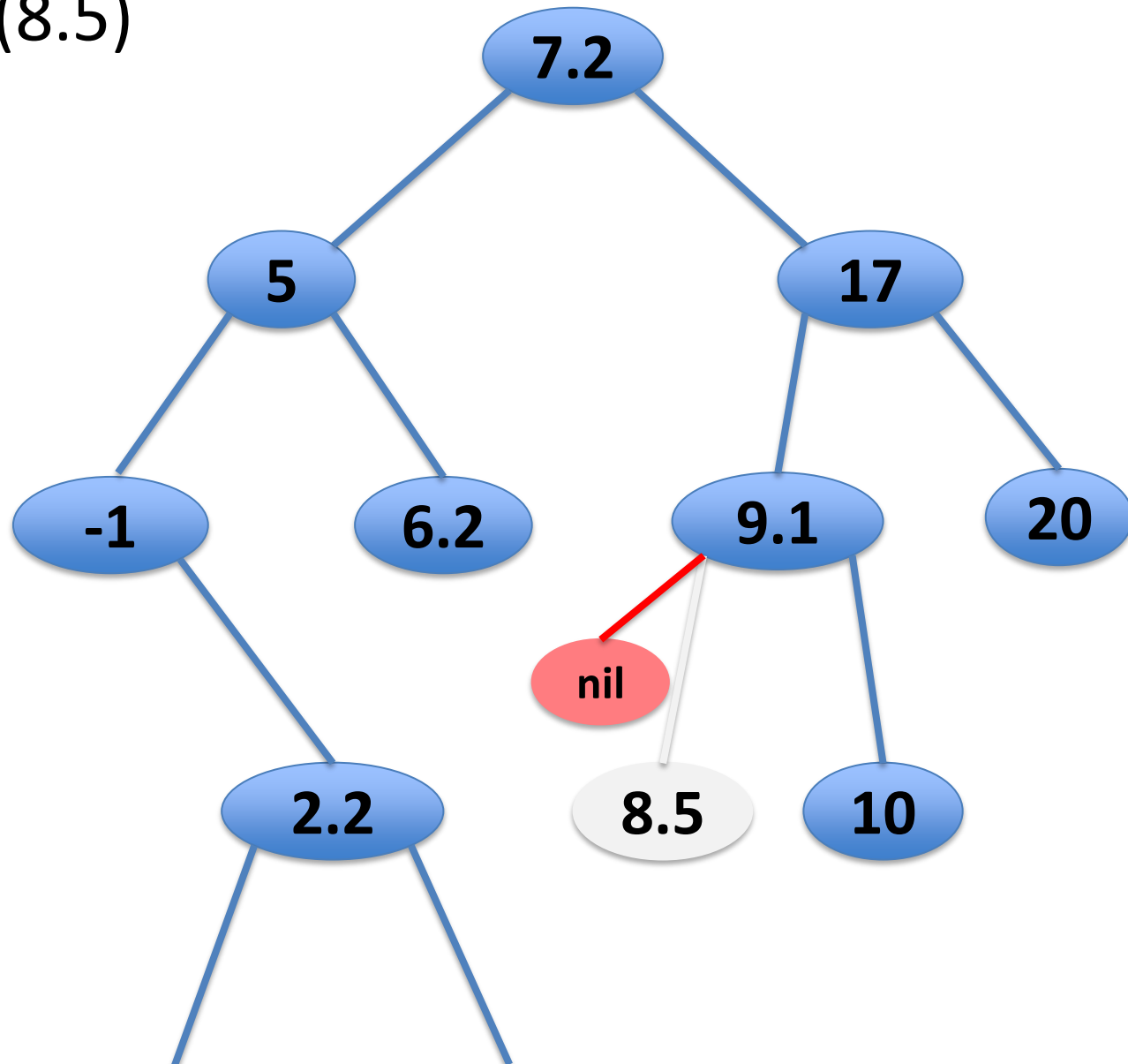
Binary Search Trees

Remove(8.5)



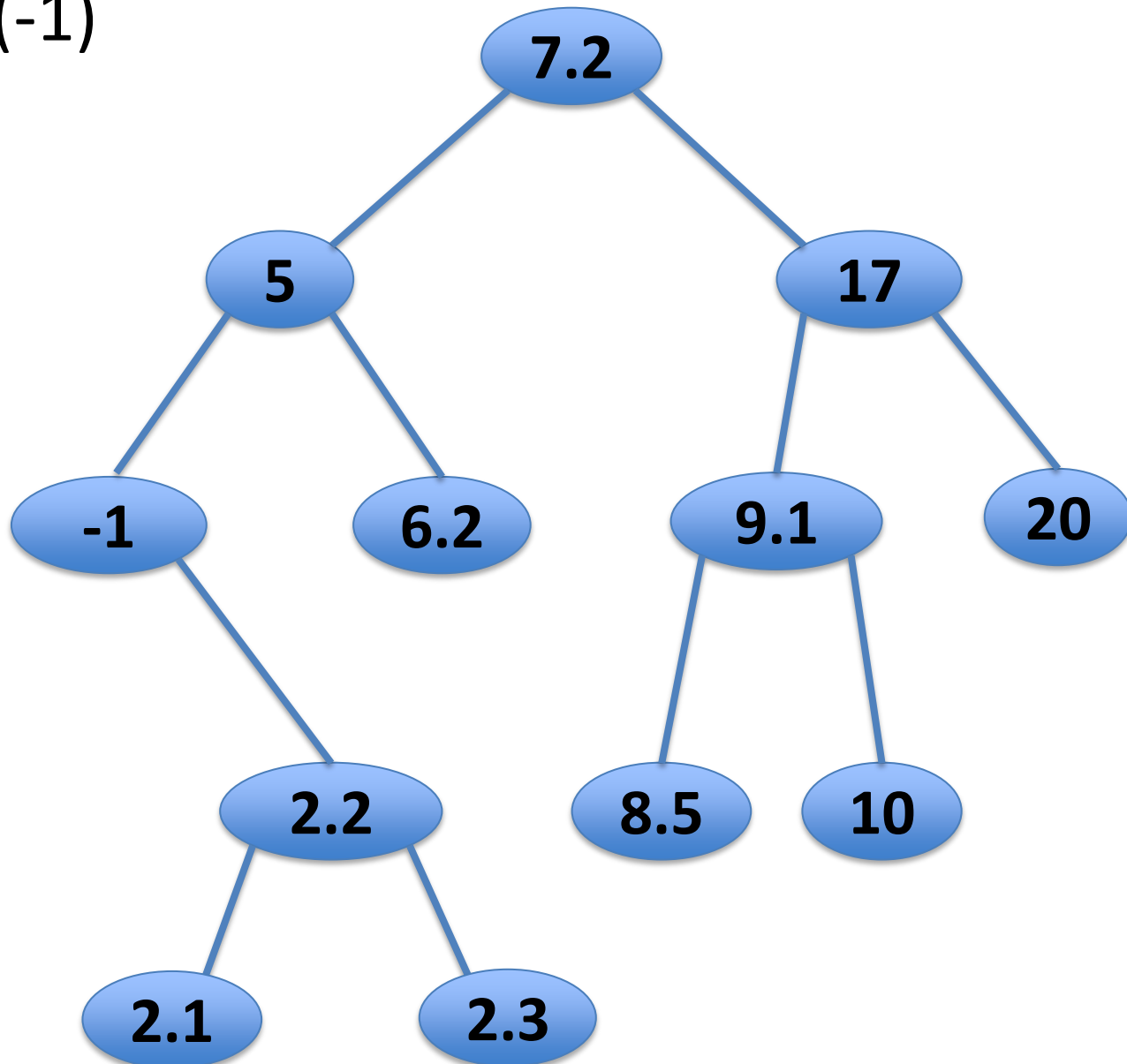
Binary Search Trees

Remove(8.5)



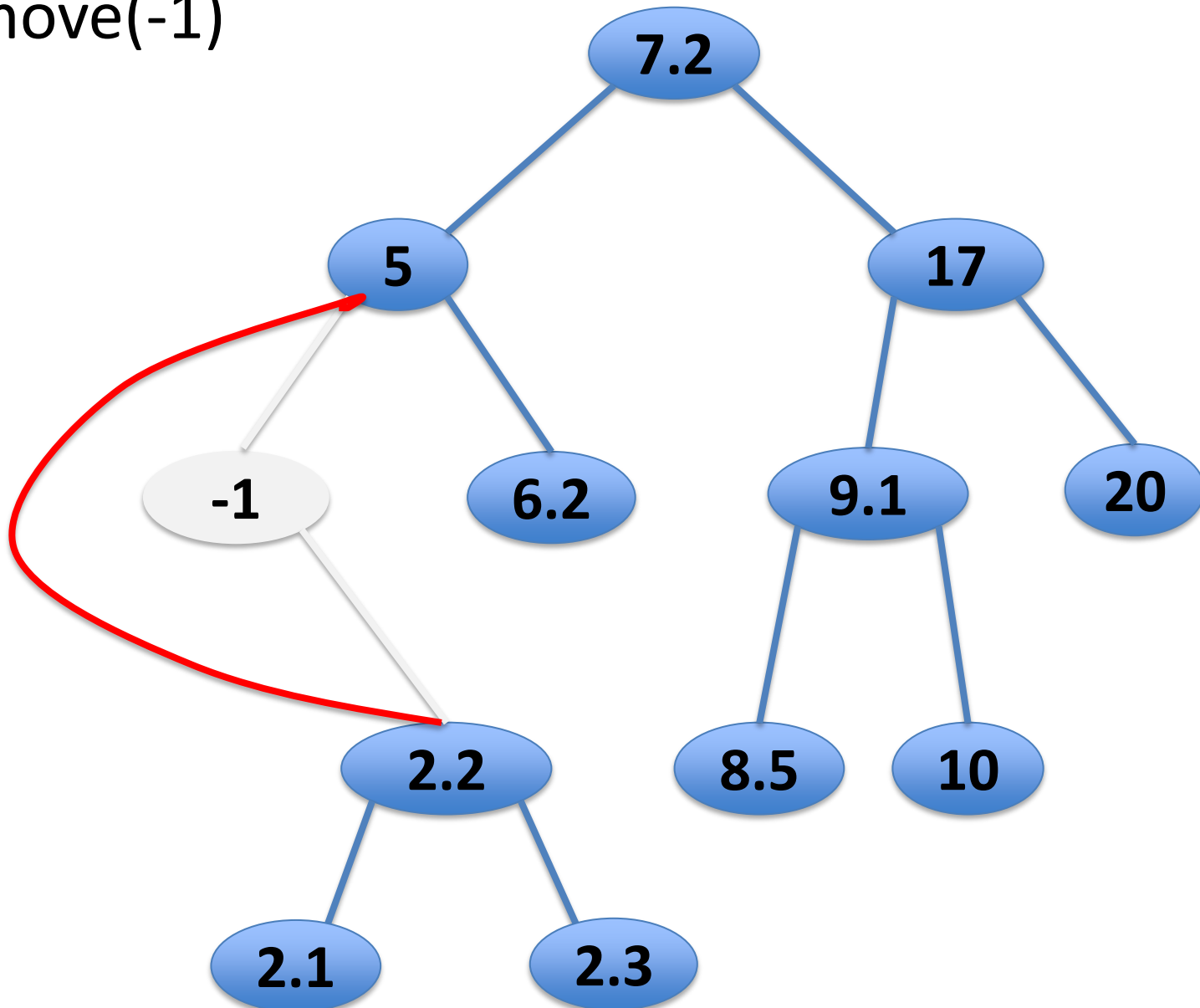
Binary Search Trees

Remove(-1)



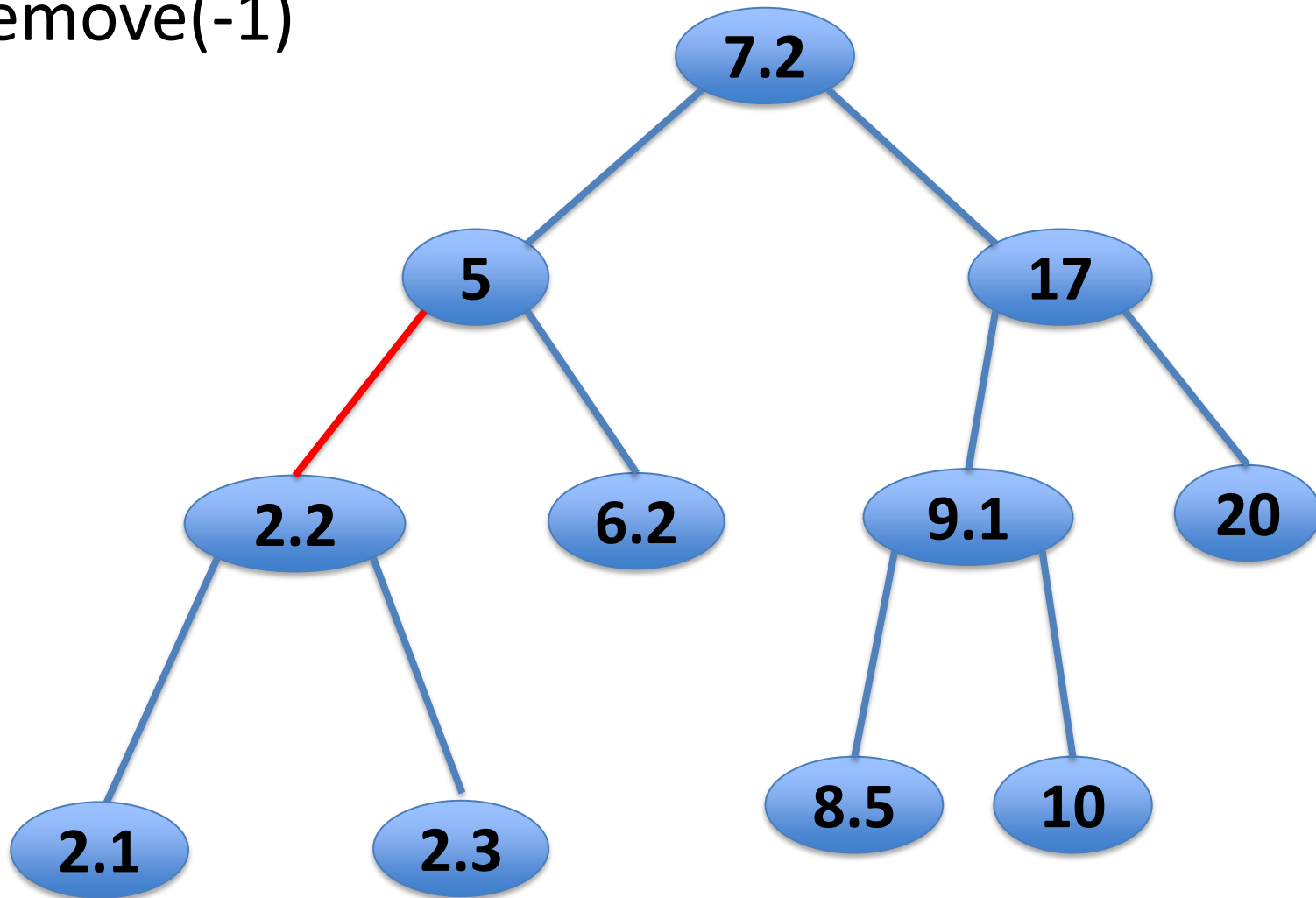
Binary Search Trees

Remove(-1)



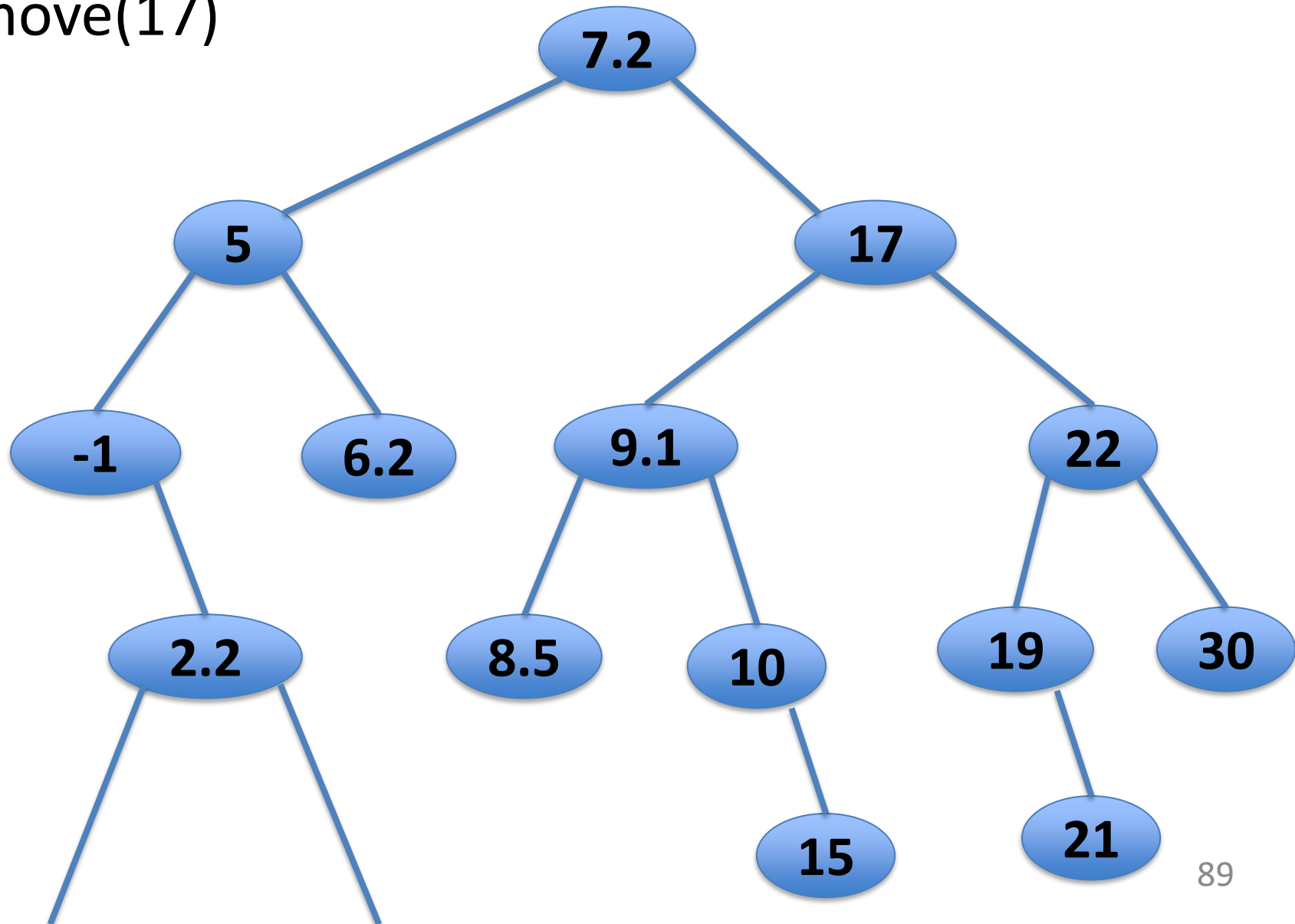
Binary Search Trees

Remove(-1)



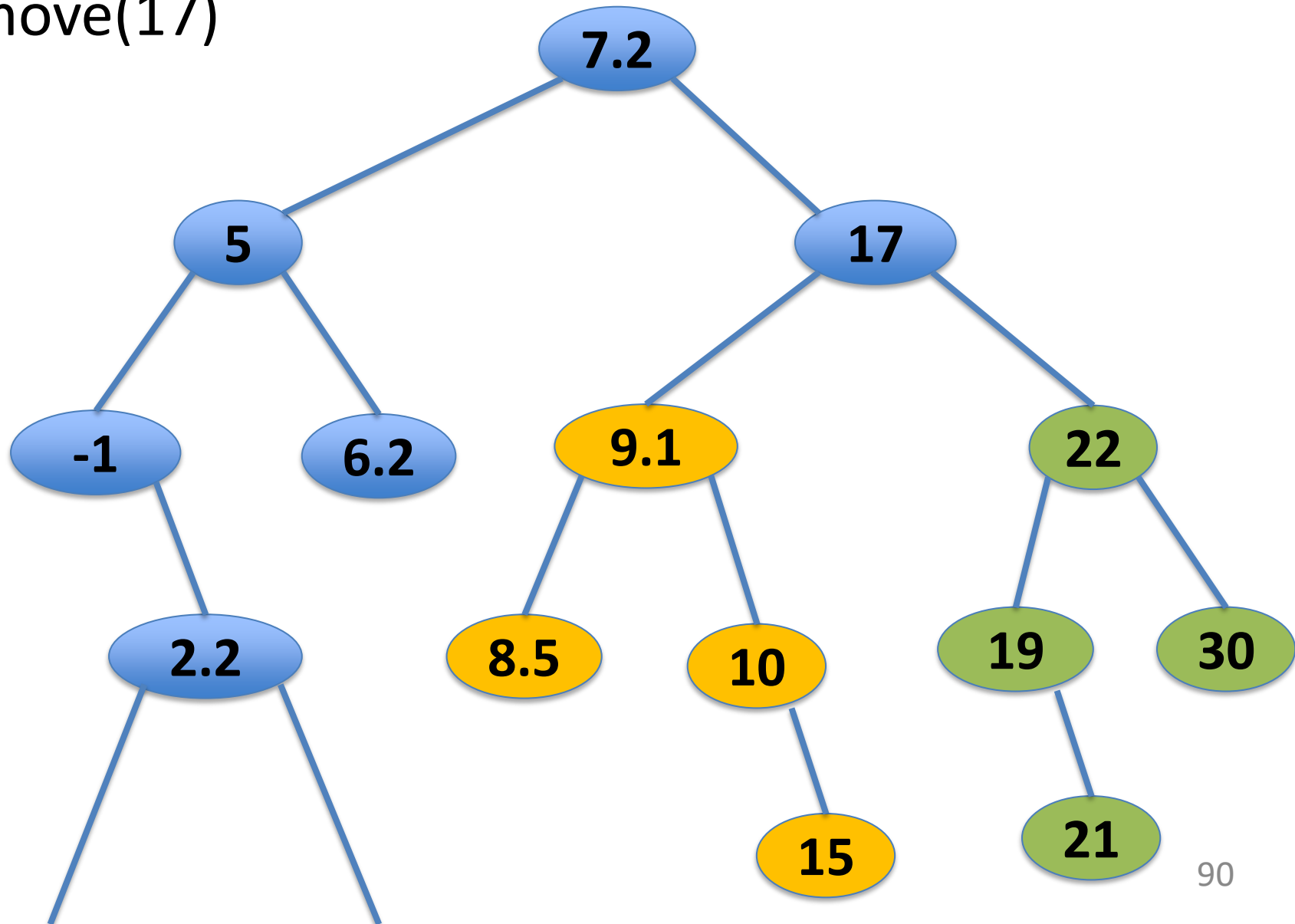
Binary Search Trees

Remove(17)



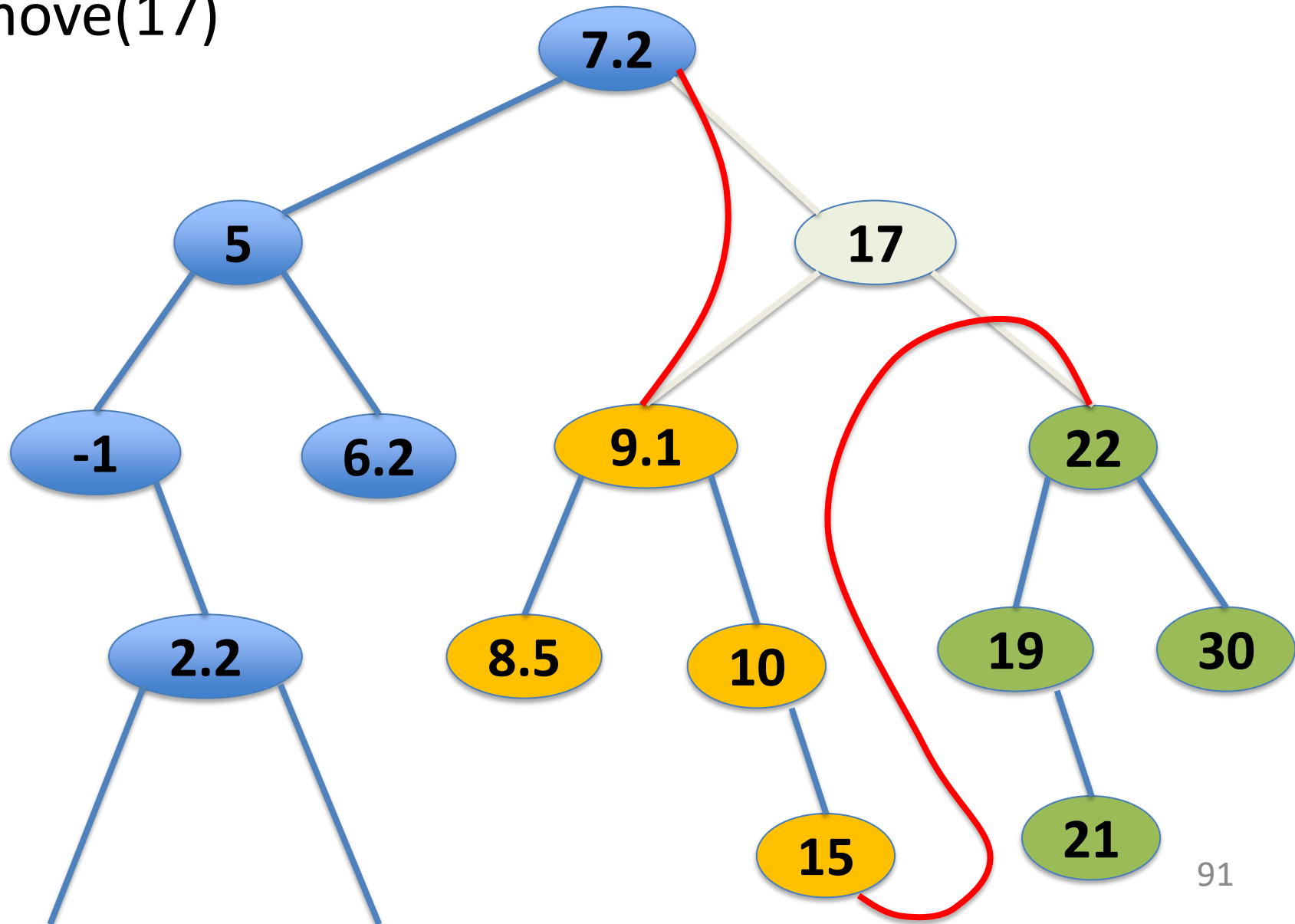
Binary Search Trees

Remove(17)



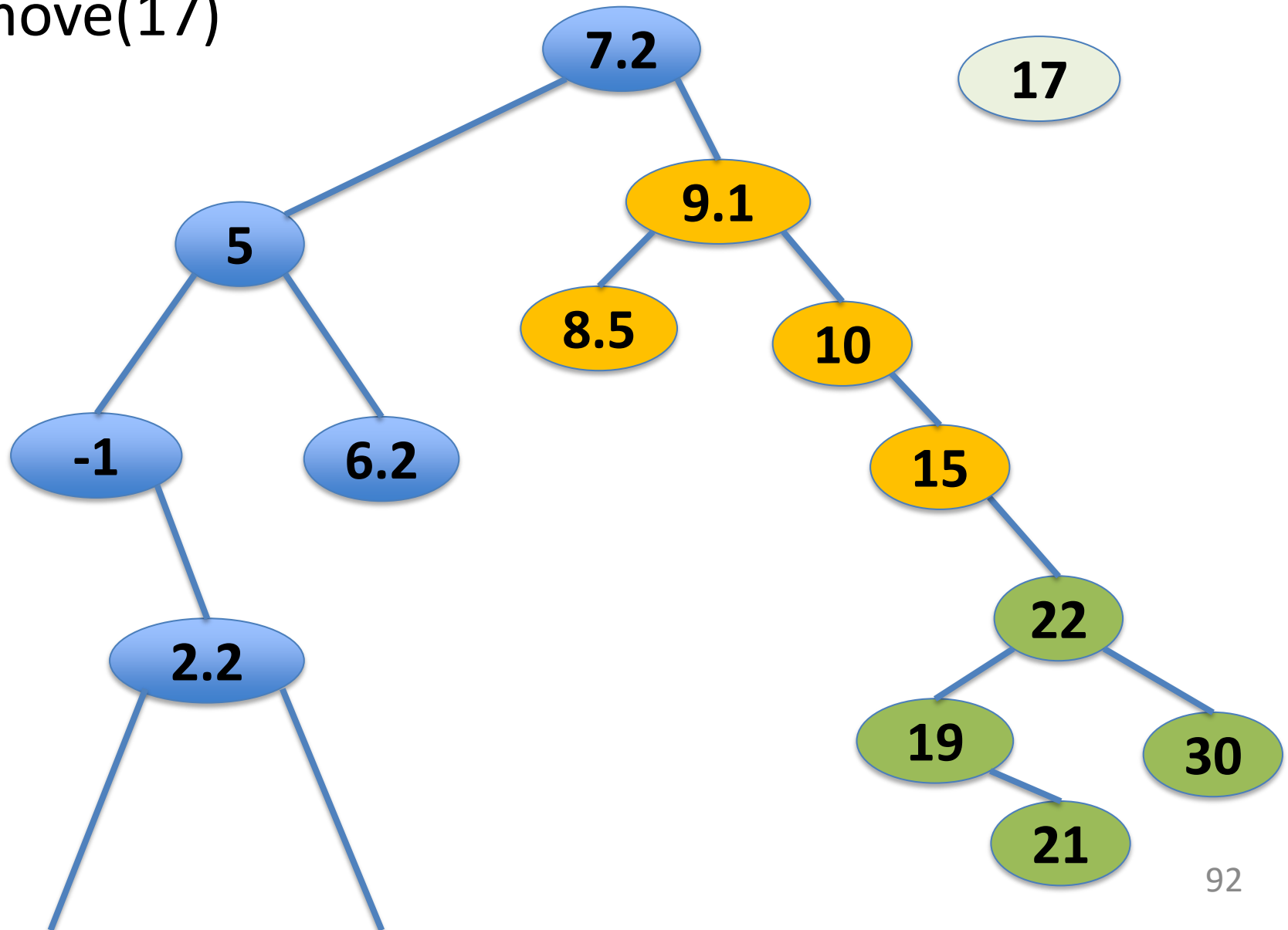
Binary Search Trees

Remove(17)



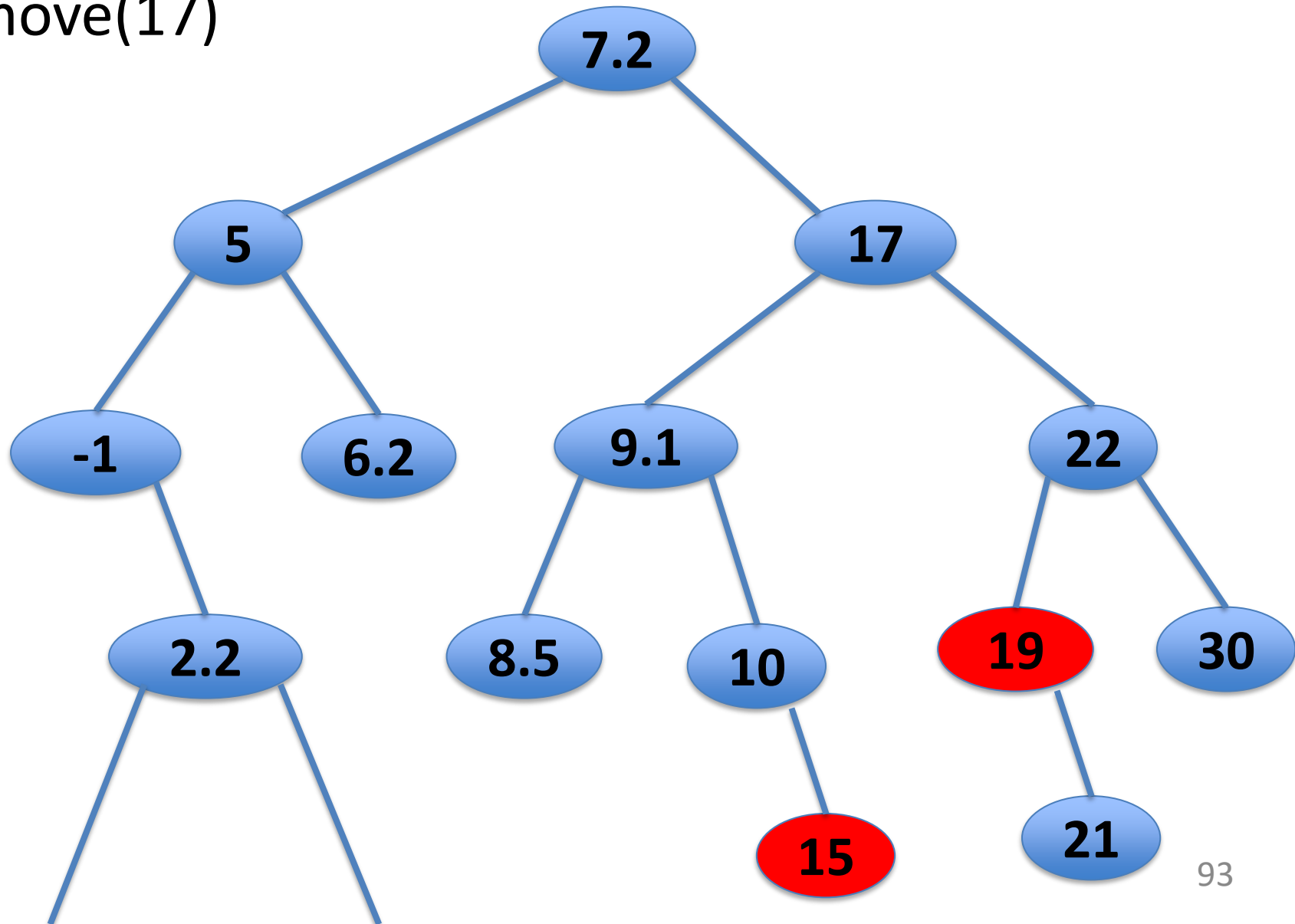
Binary Search Trees

Remove(17)



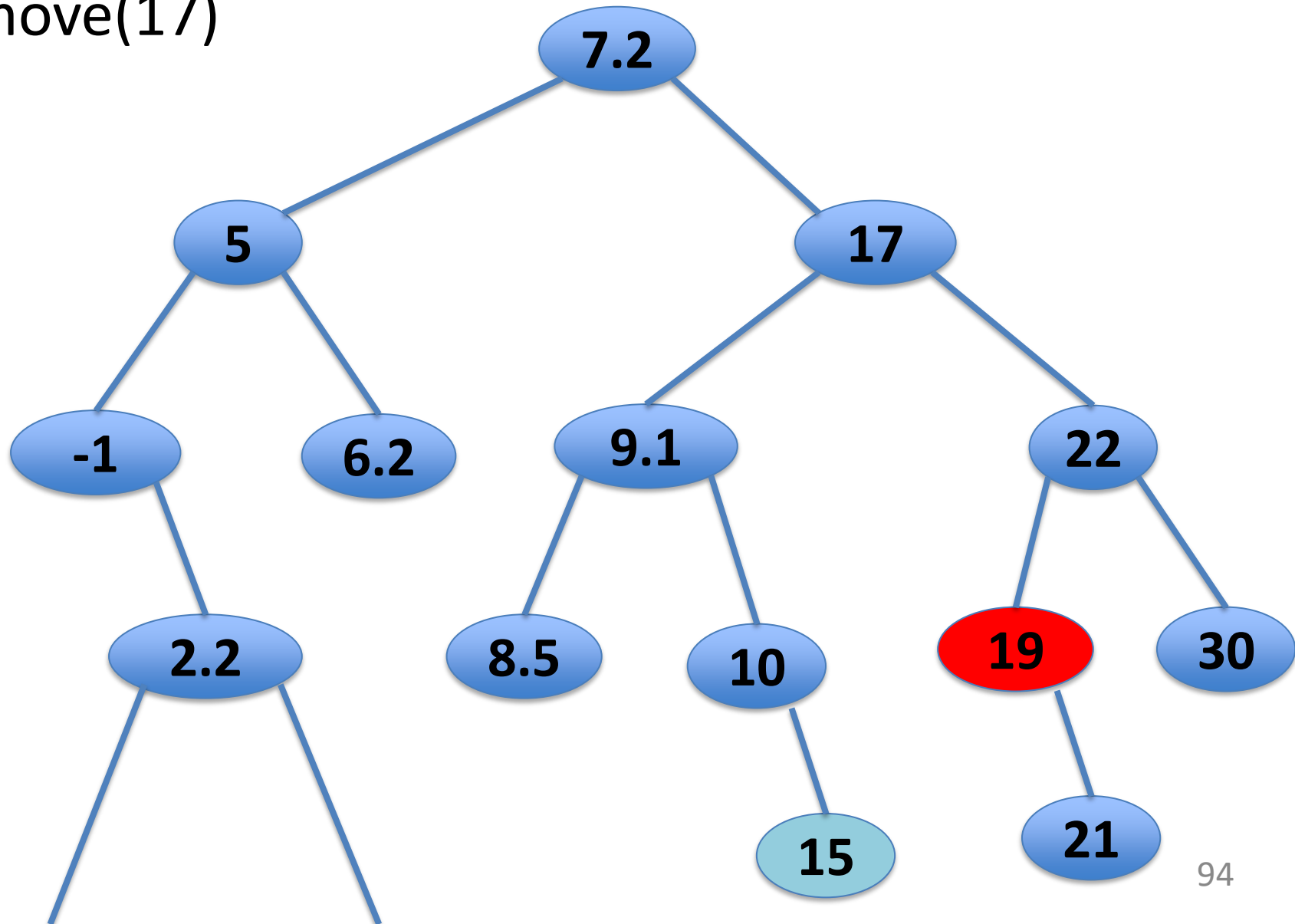
Binary Search Trees

Remove(17)



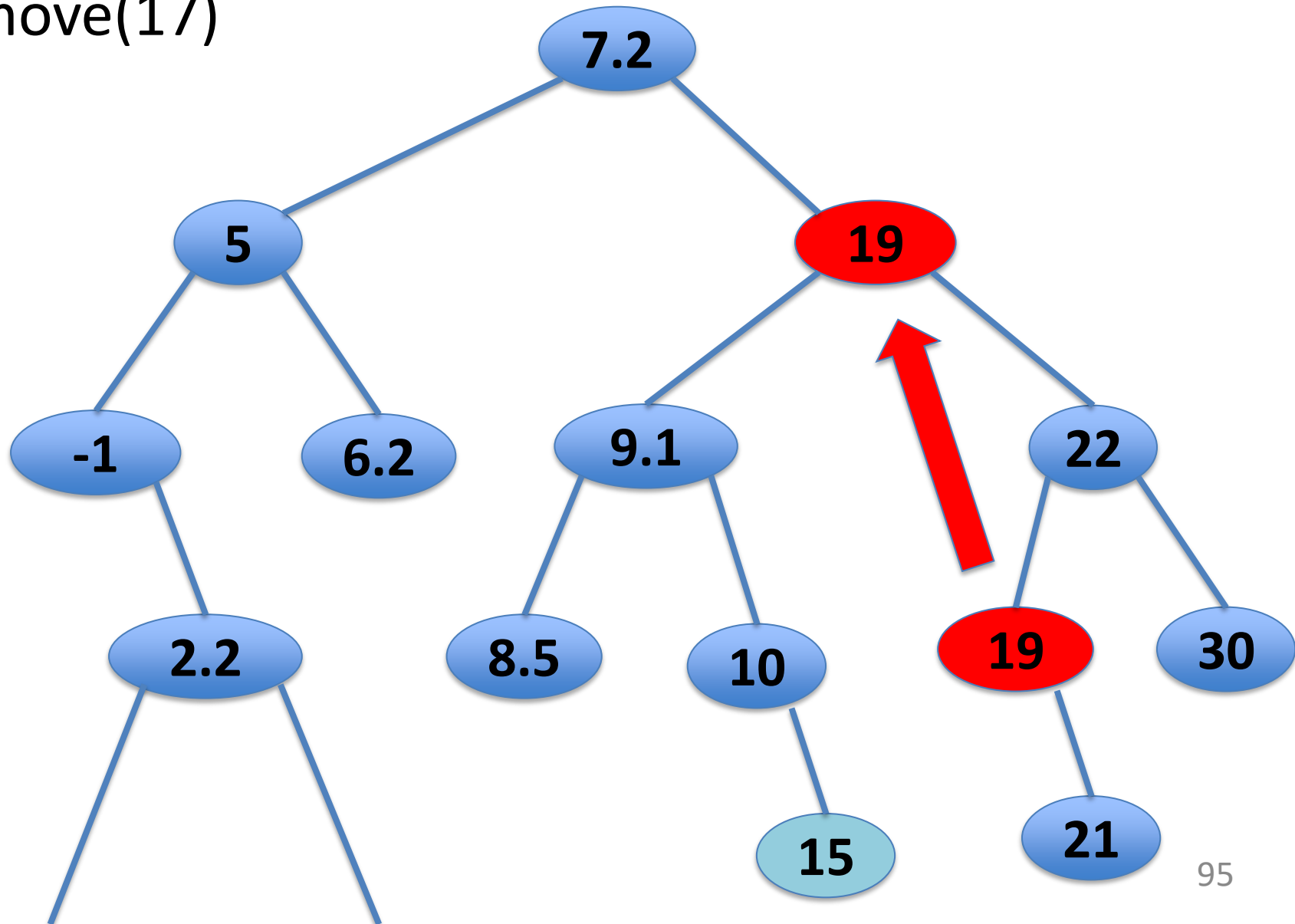
Binary Search Trees

Remove(17)



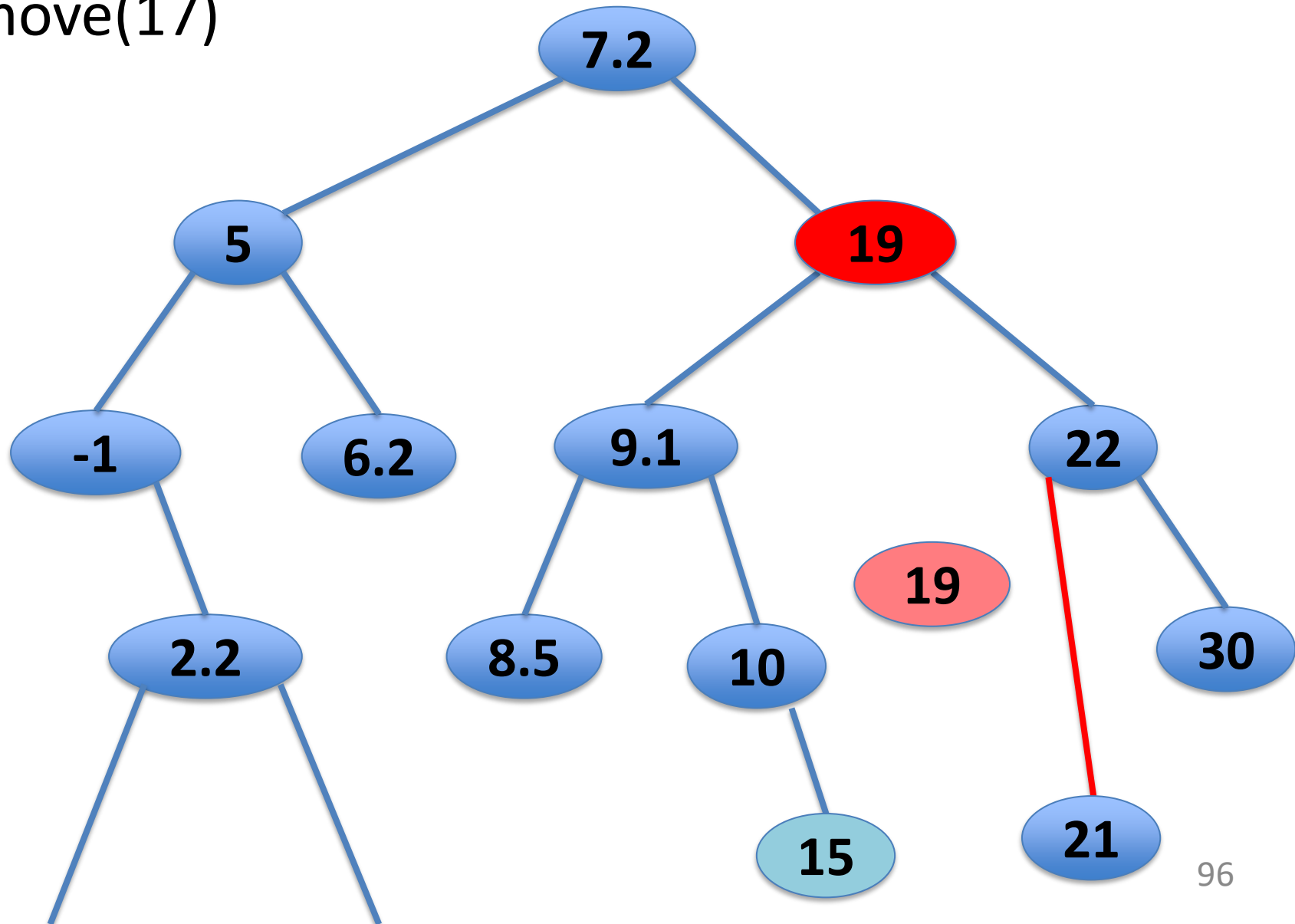
Binary Search Trees

Remove(17)



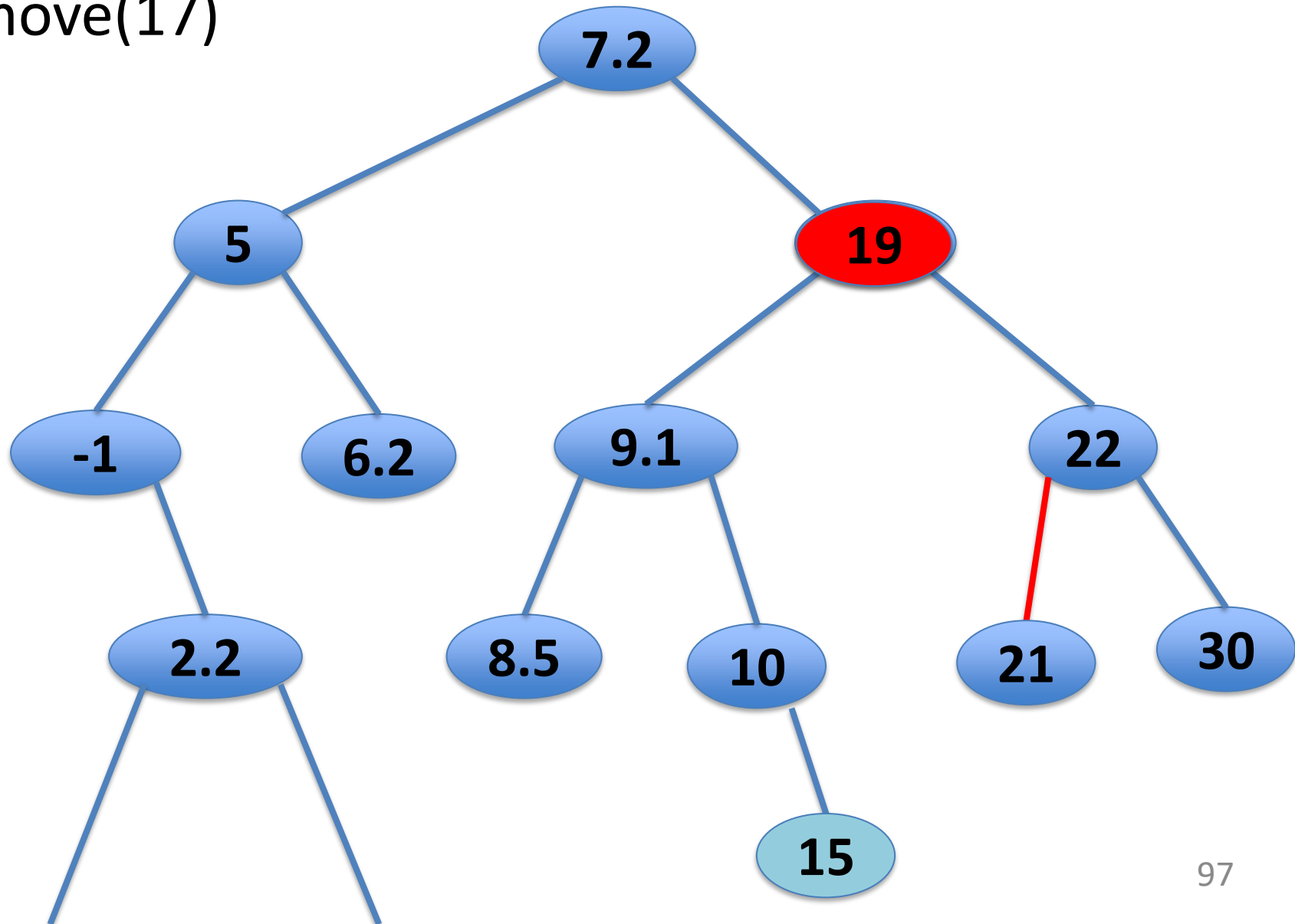
Binary Search Trees

Remove(17)



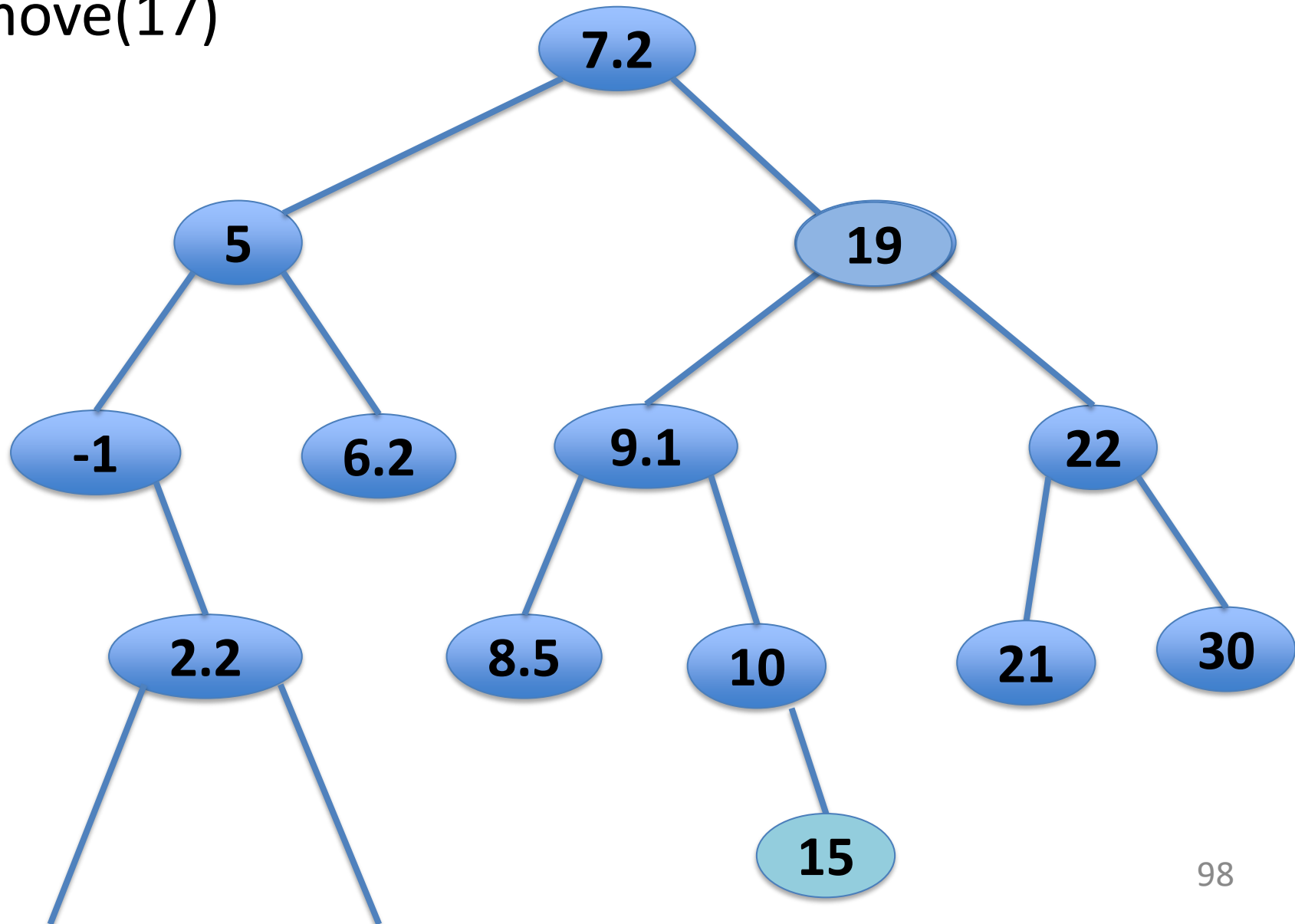
Binary Search Trees

Remove(17)



Binary Search Trees

Remove(17)



Removing Nodes

Three cases to consider when removing node u

1. If u is a leaf then simply remove it then set the reference to u from $u.parent$ to nil
2. If u is an internal node with one child then **splice** u from the tree. $u.parent$ adopts u 's child as a new child in place of u
3. If u has two children then find the smallest node bigger than u , splice it out of the tree and add this node in place of u

Binary Search Trees

BinarySearchTree implements the SSet interface and supports the operations `add(x)`, `remove(x)`, and `find(x)` in $O(n)$ time per operation.

Binary Search Trees

BinarySearchTree implements the SSet interface and supports the operations `add(x)`, `remove(x)`, and `find(x)` in $O(n)$ time per operation.

How does this compare to the skiplist?

Binary Search Trees

What is the worst case scenario for a BST?

What is the best case scenario for a BST?

Random Binary Search Trees

Consider a BST with 15 elements

Random Binary Search Trees

To create a random binary search tree (BST) we take a permutation of $0, 1, \dots, n - 1$ and add the elements one by one.

We hope that the resulting tree is good.

(We expect the resulting tree to be good.)

Random Binary Search Trees

We want binary search trees that store any data that has a total order.

Why do we just consider the numbers $0, 1, \dots, n-1$?

Harmonic Numbers

The k -th harmonic number H_k is the number

$$H_k = 1 + \frac{1}{2} + \frac{1}{3} + \cdots + \frac{1}{k} = \sum_{i=1}^k \frac{1}{i}$$

$$\log_e(k) < H_k \leq \log_e(k) + 1$$

$$\ln(k) < H_k \leq \ln(k) + 1$$

Random Binary Search Trees

Lemma 7.1: In a random binary search tree (BST) of size n , the following holds

1. For any $x \in \{0, 1, \dots, n-1\}$, the expected length of a search path for x is $H_{x+1} + H_{n-x} - O(1)$
2. For any $x \in (-1, n) \setminus \{0, 1, \dots, n-1\}$, the expected length of a search path for x is $H_{\lceil x \rceil} + H_{n-\lceil x \rceil}$

Random Binary Search Trees

What is expected runtime of searching in a random BST?

What the expected runtime of adding an element to a random BST?